基于FastMCP v2.14.5编写

Model Context Protocol

FASTMCP 2.0

服务器 开发手册

（第一版）

前言

模型上下文协议（Model Context Protocol, MCP）是一个旨在标准化 AI 模型与外部数据源和工具之间连接的开放协议。它的目标是为 AI 应用打造一个如同“USB-C 接口”般的通用连接方式，让任何 AI 模型都能通过统一的接口“即插即用”地访问各种外部资源。自 2024 年底推出以来，MCP 生态发展迅速，目前包括百度、高德在内的多家国内科技公司也已推出了基于 MCP 协议的服务。

FastMCP 是一个用于构建 MCP 应用程序的 Python 框架，提供了生产所需的一切：高级 MCP 模式（服务器组合、代理等）、企业级认证、部署工具、测试框架和完整的客户端库。它处理所有复杂的协议细节，让开发者能够快速、专注地构建应用。最重要地是，FastMCP 是“Pythonic”的——对 Python 开发者来说开发感觉自然流畅。

然而，在学习 FastMCP 开发的过程中，笔者发现官方文档对许多函数用法、参数含义和特性介绍的描述较为简略，互联网上的现有资源不仅分散，且往往不够详尽，甚至在一些参数和函数的表述上存在错误，按照这些资料难以顺利开发 MCP 服务器。笔者也曾尝试借助生成式人工智能工具辅助理解与开发，但由于相关资料匮乏，AI 的回答同样存在较多错误。此外，FastMCP 框架的源代码结构复杂，若为了理解某个函数的用法、某个参数的含义而深入底层源码，又过于耗时，严重影响开发效率。

基于上述原因，笔者在学习开发 FastMCP 服务器的过程中，按照常规的服务器代码结构顺序，系统查阅了官方文档、源代码和互联网资料，并结合实际的功能代码测试，对服务器开发中常见的 FastMCP 功能进行了详细的梳理与描述。

本开发手册适合具备 Python 开发基础、并对 MCP 协议有基本了解的读者。您可以将其作为一本“检查清单”，按照 MCP 服务器代码开发的顺序，一边编写代码，一边查阅正在编写的函数、方法、参数应如何编写、如何设置。因此，对于 MCP 的简介和原理部分，本手册仅作简要讲解，读者可通过互联网资料深入学习。对于进阶开发部分，本手册目前仅涵盖较为常用的功能，其他功能如有需要，读者可自行学习了解。

本手册编写之初，FastMCP v3.0 版本尚未正式发布，因此以当时的最新稳定版本 v2.14.5 为基础进行编写。在编写过程中，FastMCP v3.0 版本正式发布，其开发架构和特性有较大创新，旧版本代码无法直接兼容。需要特别提醒读者的是，本手册中的特性适用于 v2.14.5 版本，对于部分有更新的特性，文中有额外说明。

在这本手册的编写过程中，笔者参考了较多互联网资料，参考最多的是 FastMCP 的官方文档（https://gofastmcp.com/v2/）和其中文社区翻译版本（https://fastmcp.wiki/zh/），在此对这些文档的作者和 FastMCP 框架的贡献者表示衷心的感谢！除此以外，作者还参考了 CSDN 博客上**@ZhaoYingChao88**、

@Archy_Wang、**@m0_62919969**、**@sfdzhmr** 等的文章，在此一并表示感谢！

本手册中部分内容采用了生成式人工智能工具辅助生成与修改，例如原始文档的翻译、部分示例代码的生成、前言的撰写等。虽然笔者已进行了仔细的核查，但难免存在疏漏，请读者在阅读过程中加以自己的思考，批判性地阅读。

目前 MCP 协议和 FastMCP 框架都处于快速发展的阶段，新的特性和最佳实践正在不断涌现。本手册只是这个生态中的一个节点，笔者衷心希望读者在通过本手册入门之后，能够积极参与到 MCP 和 FastMCP 社区的建设中来——无论是提交代码、完善文档，还是在技术社区分享你的实践经验。让我们共同推动 AI 应用开发走向更加开放、互联的未来。

编著者 @wangyy04

2026 年 3 月 3 日于南京

目录

简介 1
1.1 介绍 1
1.1.1 什么是 MCP 1
1.1.2 其它 AI 调用工具标准（Function Call 和 UTCP） 1
1.2 MCP 协议架构 2
1.3 核心原语 3
1.3.1 服务端原语 3
1.3.2 客户端原语 3
1.3.3 跨领域实用原语 4
1.4 MCP 服务器 4
1.5 MCP 客户端 4
1.6 MCP 协议版本 5
MCP 协议原理 6
2.1 MCP 支持的传输协议 6
2.1.1 STDIO 传输协议 6
2.1.2 SSE 传输协议 8
2.1.3 Streamable HTTP 传输协议 9
2.1.4 MCP 传输模式小结 10
MCP 服务器基础开发 12
3.1 开发环境 12
3.1.1 Python 环境配置 12
3.1.2 FastMCP 库 12
3.2 MCP 服务器代码基本结构 13
3.3 MCP 服务器实例创建 14
3.3.1 [name] FastMCP 服务器可读名称 16
3.3.2 [host] 监听主机地址 16
3.3.3 [port] 监听端口 16
3.3.4 [log_level 和 debug] 日志级别和调试模式 17
3.3.5 [json_response 和 stateless_http] 服务器工作模式设定 18
3.3.6 [sse_path, message_path, streamable_http_path] 通信端点路径设置 19
3.3.7 [lifespan] 生命周期钩子 19
3.3.8 [include_tags 和 exclude_tags] 基于标签的组件管理 22
3.3.9 [tools] 工具列表 22

3.3.10 [strict_input_validation] 是否启用严格输入认证.....23
3.3.11 [include_fastmcp_meta] 是否省略组件元数据.....23
3.4 MCP 组件标签管理.....24
3.4.1 为组件添加标签.....24
3.4.2 组件可见性控制.....25
3.5 MCP 工具组件.....26
3.5.1 工具组件异步支持.....26
3.5.2 @tool 装饰器.....27
3.5.3 工具组件的启用/禁用.....33
2.5.4 工具组件函数的参数.....34
3.5.5 工具组件的描述注释/文档字符串.....37
3.5.6 工具组件的函数编写.....37
3.5.7 工具组件的返回值.....39
3.5.8 工具组件的错误处理.....45
3.5.9 工具组件变更通知.....47
3.5.10 从工具组件中访问 MCP 上下文（Context）.....47
3.5.11 工具转换.....48
3.5.12 工具组件重复管理与移除.....52
3.5.13 自定义工具序列化.....53
3.6 MCP 资源组件.....53
3.6.1 @resource 装饰器.....54
3.6.2 异步资源与同步资源.....56
3.6.3 资源组件的返回值.....57
3.6.4 资源模板.....58
3.6.5 资源类.....63
3.6.6 自定义资源键.....68
3.6.7 资源组件的错误处理.....69
3.6.8 资源组件变更通知.....70
3.6.9 从资源组件中访问 MCP 上下文（Context）.....70
3.6.10 资源组件重复管理.....70
3.7 MCP 提示组件.....71
3.7.1 @prompt 装饰器.....71
3.7.2 异步提示和同步提示.....72
3.7.3 提示组件函数的参数.....73
3.7.4 提示组件的返回值.....74
3.7.5 提示组件变更通知.....75

3.7.6 从提示组件中访问 MCP 上下文（Context） 76
3.7.7 提示组件重复管理.....76
3.8 部署与运行 MCP 服务器 76
3.8.1 传输协议.....76
3.8.2 常规启动方法.....77
3.8.3 异步启动方法.....77
3.8.4 自定义路由.....78
3.8.5 常规运行 MCP 服务器（直接 HTTP） 79
3.8.6 高级部署 MCP 服务器（ASGI 应用） 80
MCP 服务器的身份验证 81
4.1 简单的访问控制列表.....81
4.2 Bearer Token 82
4.3 OAuth.....82
MCP 服务器进阶开发 84
5.1 MCP 上下文（Context） 84
5.1.1 访问上下文.....84
5.1.2 上下文对象的常用功能.....85
5.1.3 运行时依赖项.....89
5.1.4 自定义依赖项.....92
5.2 中间件（Middleware） 93
5.2.1 中间件钩子.....95
5.2.2 中间件钩子结构.....97
5.2.3 中间件中的组件访问.....97
5.2.4 中间件的常见应用场景.....99
5.2.5 服务器组合中的中间件.....100
5.2.6 内置的中间件.....101
5.3 图标.....105
5.3.1 服务器图标.....105
5.3.2 组件图标.....106
5.3.3 使用数据 URI 嵌入图标.....106
附录 A MIME 类型.....107
附录 B 常用函数参数速查表 126

简介

1.1 介绍

1.1.1 什么是 MCP

模型上下文协议（Model Context Protocol, MCP）是一个连接 AI 应用与外部系统的开源标准。通过 MCP，Claude 或 ChatGPT 等 AI 应用能够连接数据源（如本地文件、数据库）、工具（如搜索引擎、计算器）以及工作流（如专用提示指令），从而获取关键信息并执行各类任务。

不妨将 MCP 想象为 AI 应用的 USB-C 接口。正如 USB-C 为电子设备提供了标准化连接方式，MCP 为 AI 应用接入外部系统提供了标准化途径。

在 AI 应用/智能体领域，MCP 提供了连接数据源、工具与应用生态的通道，从而增强能力并提升终端用户体验。例如智能体或 LLM 可以自主决定调用 MCP 服务器提供的工具（比如访问内部知识库、连接私有 API、控制本地应用程序等）并自主利用工具返回的信息等。

官方网站为 https://modelcontextprotocol.io/

官方 GitHub 页面为 https://github.com/modelcontextprotocol

1.1.2 其它 AI 调用工具标准（Function Call 和 UTCP）

在 MCP 协议之前，AI Agent 的开发如果想调用外部工具需要针对不同的 AI 大模型编写不同的代码，最常见的是 OpenAI 提供的 Function Call 逻辑。

在 Function Call 逻辑中，当调用 OpenAI Chat Completions 接口时，可以通过 tools 参数传入可供使用的外部工具。当模型运行时可根据需要调用相应的工具，与 MCP 类似。但 Function Call 不同的模型提供的接口不一样，对于每一种模型、每一个工具，都需要进行针对性地编写代码，例如为 M 个模型对接 N 个工具，则需开发 M×N 次，可迁移性弱、重构成本高。而 MCP 协议是标准化的，开发一次 MCP 服务器，则可以在所有兼容 MCP 标准的模型上使用，服务器提供的资源不必写入客户端代码，客户端连接服务器后可直接动态读取资源列表；使用工具的客户端也可按需要快速增加或移除 MCP 组件，支持热插拔。

通用工具调用协议（Universal Tool Calling Protocol, UTCP）是一种新兴的开源协议，比 MCP 更新。它的核心设计目标是消除“包装税”，即希望智能体能像人类开发者一样，直接阅读和理解工具的原始 API 文档，并据此直接调用其原生 HTTP 端点，而无需经过任何中间适配层。MCP 协议的工具在调用时需要先通过 MCP 服务器，会带来数百 ms 的延迟，在工具链调用的情况下累计延迟较大。而 UTCP 协议直接像人类一样阅读文档并调用工具 API，性能开销和额外开发负担低。但其性能依赖于工具 API 文档的完整性和模型对长文档的理解能力。此外，由于 UTCP 协议非常新，其生态不够完善，网络上参考资料、开源项目较少，不如 MCP 协议那样容易学习。

表 1.1　三种 AI 调用工具标准的对比

	Function Call	MCP	UTCP
设计定位	厂商特定的、紧耦合的模型能力。	解耦模型与工具的“连接器”标准。	回归原始 API 调用的“直接调用”协议。
核心架构	**模型原生能力。**函数定义作为提示词的一部分，模型在推理时输出结构化调用请求。	**客户端-服务器（C/S）架构。**模型是客户端，通过标准协议向 MCP 服务器请求服务。	**描述-执行架构。**工具提供 JSON 描述，客户端根据描述直接构造并发送 HTTP 请求。
优势	在单一模型生态内效率高。	已成为事实上的开放生态标准，支持动态发现和热插拔。	性能开销极低、无额外开发负担，能无缝利用现有 API 生态，理论延迟最低。
劣势	厂商锁定、扩展性差。换模型或集成新工具麻烦。	存在“包装税”。需要为每个服务开发或运行一个 MCP 服务器，增加复杂性和潜在延迟。	对工具 API 的规范性要求高，目前生态和工具支持度远低于 MCP，依赖于智能体有很强的 API 文档理解能力。
应用场景	快速原型验证，基于特定大模型构建简单功能。	企业级复杂应用，需要连接多种内部工具、考虑未来更换模型或要求严格权限管控。	性能敏感场景，或希望将现有成熟的微服务 API 体系直接、无损地暴露给 AI 智能体使用。

1.2 MCP 协议架构

MCP 协议采用了 C/S 架构，也就是服务端、客户端架构，能支持在客户端设备上调用远程 Server 提供的服务，同时也支持 stdio 流式传输模式，也就是在客户端本地启动 mcp 服务端。只需要在配置文件中新增 MCP 服务端，就能用上这个 MCP 服务器提供的各种工具，大大提高了大模型使用外部工具的便捷性。其结构如图 1.1 所示，MCP 客户端通过各个 MCP 服务器获取资源。

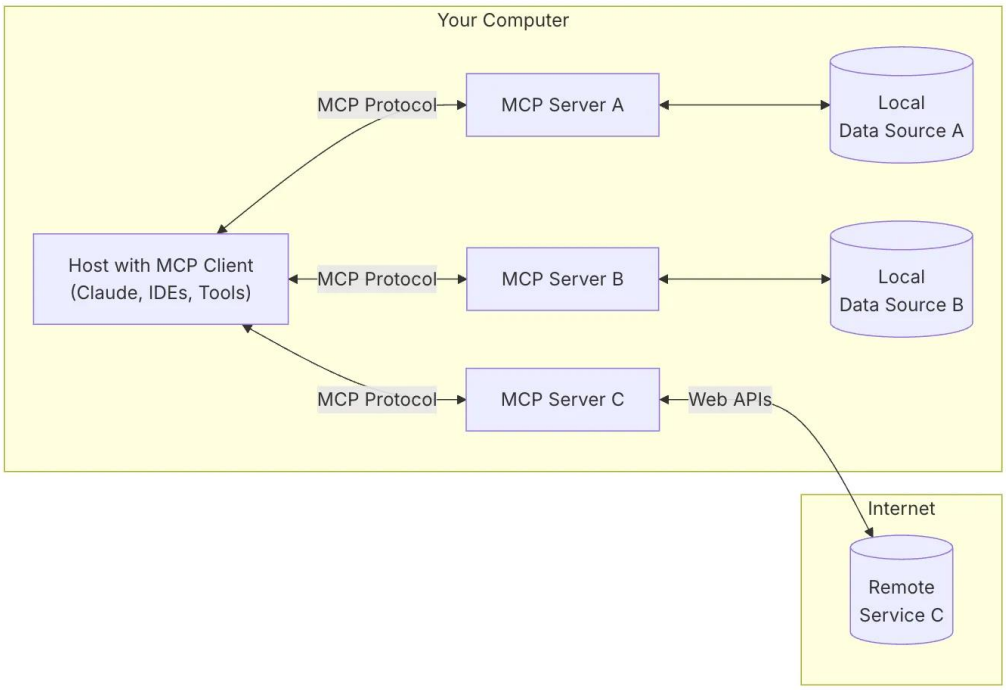

图 1.1　MCP 协议架构示意图

需要注意的是，MCP 服务器特指提供上下文数据的程序，其运行位置并不影响这一定义。MCP 服务器可在本地或远程环境中运行。例如，当 Claude Desktop 启动文件系统服务器时，由于采用标准输入输出传输协议（Stdio），该服务器实际运行于本地同一台机器上，这类通常被称为“本地”MCP 服务器。而 Sentry MCP 服务器则部署在 Sentry 平台上，采用可流式 HTTP 传输协议（Streamable-HTTP），这类通常被称为“远程”MCP 服务器。

MCP 协议由两个层次构成：

• **数据层**：定义了基于 JSON-RPC 的客户端-服务器通信协议，包含生命周期管理及核心基础构件（如工具、资源、提示指令和通知）。

• **传输层**：定义了实现客户端与服务器间数据交换的通信机制与通道，包括特定传输协议的连接建立、消息帧封装和授权验证。

从概念上看，数据层属于内层协议，而传输层构成外层封装。

1.3 核心原语

MCP 原语（Primitives）是协议中最核心的概念，定义了客户端与服务器能够相互提供的功能类型。这些原语明确了可与 AI 应用共享的上下文信息种类以及可执行的操作范围。

1.3.1 服务端原语

MCP 定义了服务器可暴露的三大核心原语：

• **工具（Tools）**：AI 应用可调用的可执行函数，用于执行具体操作（如文件操作、API 调用、数据库查询）

• **资源（Resources）**：为 AI 应用提供上下文信息的数据源（如文件内容、数据库记录、API 响应）

• **提示词（Prompts）**：用于构建与大语言模型交互的可复用模板（如系统提示、小样本示例）

每种原语类型都配有相应的发现（*/list）、检索（*/get）及执行（tools/call）方法。MCP 客户端通过*/list 方法发现可用原语，例如先列出所有可用工具（tools/list）再执行调用。这种设计使得原语列表能够动态生成。

具体示例：假设某个提供数据库上下文的 MCP 服务器，它可以暴露查询数据库的工具、包含数据库结构的资源，以及提供工具交互小样本示例的提示词。

1.3.2 客户端原语

MCP 同样定义了客户端可暴露的原语，这些原语使 MCP 服务器开发者能够构建更丰富的交互体验。

• **采样（Sampling）**：允许服务器向客户端的 AI 应用请求语言模型补全。当服务器开发者需要访问语言模型，但希望保持模型无关性且不在 MCP 服务器中集成语言模型 SDK 时，此功能尤为实用。他们可通过 sampling/complete 方法向

客户端的 AI 应用请求语言模型补全。

• **征询（Elicitation）**：允许服务器向用户请求补充信息。当服务器开发者需要从用户处获取更多信息或请求确认操作时，可使用 elicitation/request 方法向用户征询额外信息。

• **日志记录（Logging）**：使服务器能够向客户端发送日志消息，便于调试和监控。

1.3.3 跨领域实用原语

除了服务器与客户端原语外，该协议还提供了跨领域实用原语，用于增强请求的执行方式：

• **任务（Tasks）**：[该原语是一个实验性的功能] 作为持久化执行封装器，可为 MCP 请求实现延迟结果获取与状态追踪（适用于耗时计算、工作流自动化、批处理及多步骤操作等场景）。

1.4 MCP 服务器

MCP 服务器是通过标准化协议接口向 AI 应用对外提供特定能力的程序。

常见的示例包括用于文档访问的文件系统服务器、用于数据查询的数据库服务器、用于代码管理的 GitHub 服务器、用于团队沟通的 Slack 服务器，以及用于日程管理的日历服务器。

为了实现服务端原语，服务器通过三个构建模块提供功能，即工具（Tools）、资源（Resources）、提示词（Prompts），如表 1.2 所示。

表 1.2　MCP 服务器核心功能

功能	说明	示例	控制方
工具 （Tools）	LLM 可主动调用的函数，根据用户请求决定使用时机。工具可向数据库写入数据、调用外部 API、修改文件或触发其他逻辑。	搜索航班 发送消息 创建日历事件	模型
资源 （Resources）	提供只读访问信息的被动数据源，用于提供上下文，例如文件内容、数据库结构或 API 文档。	获取文档 访问知识库 读取日历	应用
提示词 （Prompts）	预置的指令模板，用于引导模型使用特定工具和资源。	规划假期 总结会议纪要 起草邮件	用户

在为 MCP 服务端编写代码时，可根据实际需要编写相关功能的实现方法，在实际应用中最常用的是 Tools 功能。关于 MCP 服务器的核心功能更加详细具体的案例，请访问并阅读官方文档中的 *Understanding MCP servers* 部分（https://modelcontextprotocol.io/docs/learn/server-concepts）。

1.5 MCP 客户端

MCP 客户端由宿主应用（Host）实例化，用于与特定 MCP 服务器进行通信。宿主应用（如 Claude.ai、集成开发环境和智能体）负责管理整体用户体验并协调

多个客户端，而每个客户端则独立处理与单一服务器的直接通信。理解这一区别至关重要：宿主应用（Host）是用户直接交互的应用程序，而客户端（Client）是实现与服务器（Server）连接的协议级组件。

除了利用服务器提供的上下文信息外，客户端还可向服务器提供多项功能，如表 1.3 所示。这些客户端功能使服务器开发者能够构建更丰富的交互体验。

表 1.3　MCP 客户端核心功能

功能	说明	示例
征询（Elicitation）	使服务器能够在交互过程中向用户请求特定信息，为服务器按需收集信息提供结构化途径。	旅行预订服务器可询问用户对飞机座位、房间类型的偏好或联系方式，以完成预订确认。
根目录（Roots）	允许客户端指定服务器应关注的目录范围，通过协调机制传达预期操作边界。	可为旅行预订服务器授予特定目录的访问权限，使其能够读取用户的日历信息。
采样（Sampling）	允许服务器通过客户端请求大语言模型补全，实现自主工作流。这种方式让客户端完全掌控用户权限与安全管控。	旅行预订服务器可向大语言模型发送航班列表，请求其为用户筛选最优航班。

对于简单的应用，即仅包含一次从客户端向服务器提交参数和一次从服务器向客户端返回处理结果，例如调用计算器等，MCP 客户端的功能由大模型控制，大模型会提交符合要求的参数并利用接收到的结果，开发者仅需为 MCP 服务器编写代码。关于 MCP 客户端的核心功能更加详细具体的案例，请访问并阅读官方文档中的 *Understanding MCP silents* 部分（https://modelcontextprotocol.io/docs/learn/client-concepts）。

1.6 MCP 协议版本

模型上下文（MCP）协议采用基于字符串的版本标识符，遵循 YYYY-MM-DD 格式，用于标示最近一次作出不向后兼容更改的日期。

只要更新内容保持向后兼容性，协议版本号将不会递增。这种方式允许在保持互操作性的同时进行渐进式改进。

客户端与服务器之间的版本协商在初始化阶段进行。客户端与服务器可同时支持多个协议版本，但必须就会话使用的单一版本达成一致。若版本协商失败，协议将提供相应的错误处理机制，允许客户端在无法找到与服务器兼容的版本时妥善终止连接。

MCP 协议原理

2.1 MCP 支持的传输协议

FastMCP 框架支持多种传输协议，包括 STDIO、SSE 和 Streamable HTTP，为不同部署场景提供灵活的通信方式。这些协议通过统一的抽象层实现，确保了 API 的一致性和易用性。每种协议都有专门的传输适配器实现，这些适配器负责将底层传输细节抽象化，向上层提供统一的读写流接口。这种设计模式使得核心业务逻辑与传输层解耦，提高了代码的可维护性和可扩展性。

FastMCP 的协议适配采用分层架构设计，将传输协议的具体实现与核心业务逻辑分离，其分层架构如图 2.1 所示。

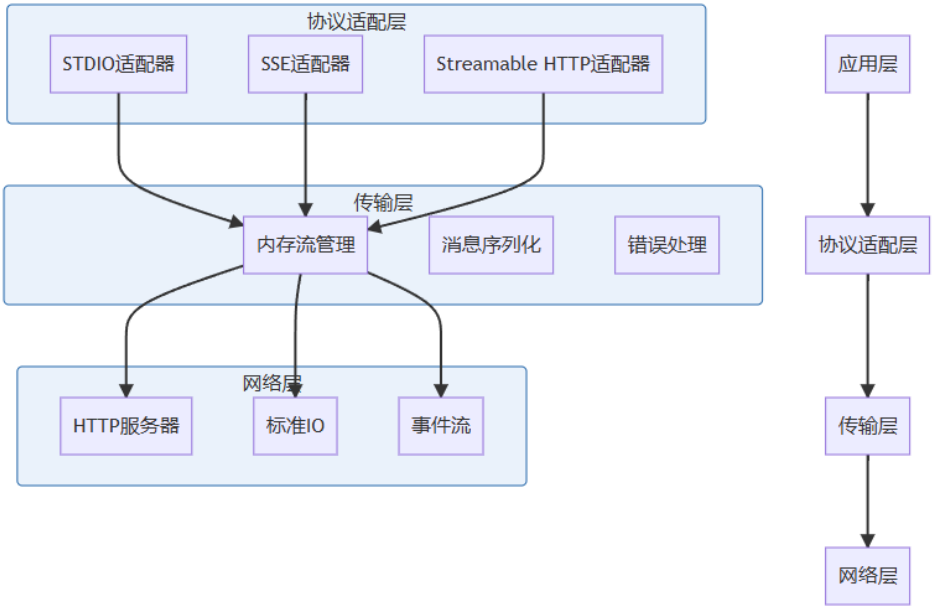

图 2.1　FastMCP 协议适配分层架构示意图

2.1.1 STDIO 传输协议

STDIO 传输协议是 FastMCP 框架中用于进程间通信的一种轻量级传输机制。它通过标准输入（stdin）和标准输出（stdout）流实现 MCP 客户端与服务器之间的消息交换。该协议特别适用于命令行工具（CLI）集成和**本地**进程间通信场景，无需网络开销即可实现高效的数据传输。

run_stdio_async 方法是 FastMCP 服务器启动 STDIO 传输模式的核心异步方法。该方法通过调用 stdio_server 上下文管理器来建立异步读写流，并将这些流传递给底层 MCP 服务器进行消息处理。该方法首先创建一个 stdio_server 异步上下文管理器，获取读取流（read_stream）和写入流（write_stream）。然后，它调用底层_mcp_server 的 run 方法，将这两个流以及初始化选项作为参数传递。

异步上下文管理器 stdio_sever 首先确保 stdin 和 stdout 以 UTF-8 编码的文本流形式存在。然后创建两个内存对象流：一个用于读取客户端消息，另一个用于向客户端写入消息。通过 create_memory_object_stream 方法创建的这两个流实现

了生产者-消费者模式[1]，使得读取和写入操作可以异步并发执行。异步上下文管理器 stdio_server 还负责启动两个协程任务：stdin_reader 用于从 stdin 读取数据并写入读取流，stdout_write 用于从写入流读取数据并写入 stdout。这两个任务在 anyio.create_task_group 中并发执行，确保了双向通信的实时性。

stdio_server 管理器的工作流程如图 2.2 所示。

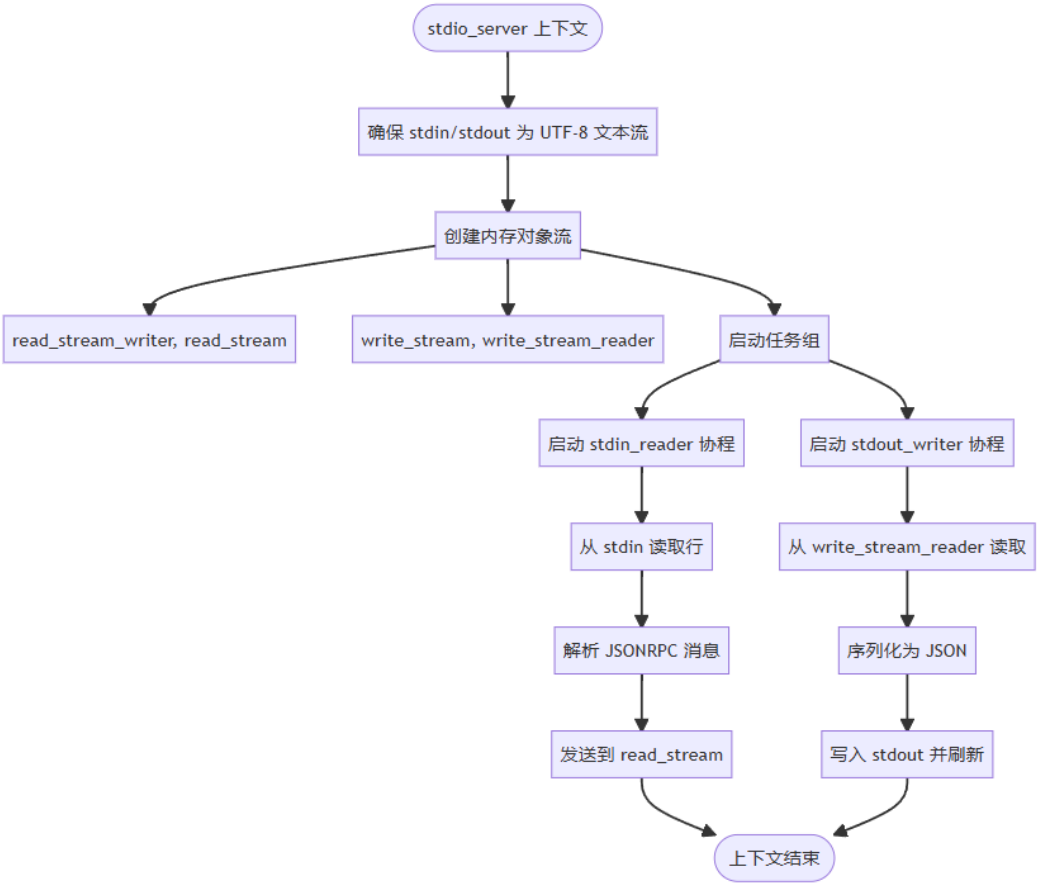

图 2.2　异步上下文管理器 stdio_server 的工作流程图

上文中已提到，stdin_reader 和 stdout_writer 是两个核心协程函数，分别负责处理 STDIO 通信的输入和输出方向。stdin_reader 协程从 stdin 异步读取每一行数据，尝试将其解析为 JSONRPCMessage 对象。如果解析成功，则创建 SessionMessage 对象并发送到读取流；如果解析失败，则将异常发送到读取流以便上层处理。该协程在 read_stream_writer 上下文中运行，确保在上下文退出时正确关闭。stdout_writer 协程从写入流读 SessionMessage 对象，将其序列化为 JSON 字符串，并写入 stdout 后立即刷新。这确保了消息能够及时传输到客户端。该协程同样在 write_stream_reader 上下文中运行，保证了资源的正确管理。

尽管 STDIO 传输协议具有简单高效的优势，但也存在一些明显的局限性。比如会话管理缺失（无会话管理功能每个消息都是独立处理的，无法维护客户端

[1] 生产者-消费者模式是一种经典的多任务协作模型：生产者负责生成数据，消费者负责处理数据，两者通过一个共享缓冲区解耦。

状态）、跨平台兼容较差（例如 Windows 与 Linux 平台之间 stdio 流存在不同难以兼容）、只能支持一个客户端连接等。**仅能在本地运行，无法远程提供服务。**

2.1.2 SSE 传输协议

SSE（Server-Sent Events）传输协议在 FastMCP 框架中实现了高效的双向通信机制。通过 SseServerTransport 类提供的两个 ASGI 应用，系统能够建立持久化的服务器到客户端消息推送通道，并接收客户端的指令请求。SSE 协议虽然底层是 HTTP 协议，但通信模式主要是服务器向客户端发送数据。客户端的请求通常在建立连接时或通过单独的通道发送。

1. SSE 协议架构

SSE 服务器必须暴露两个端点：/sse 端点由客户端长期挂起，专门用于接收服务器推送的消息（message 事件）；/message 端点则接收客户端向此地址主动发送的请求/通知。当客户端首次连接 SSE 端点时，服务器不会立即开始传输业务消息，而是首先发送一个特殊的 endpoint 事件，该事件的 JSON 格式信息中包含 data 字段，该字段告诉客户端后续的消息应当 POST 到的 URL。

服务器向客户端的信息通过始终打开的 SSE 连接，以 message 事件推送 JSON-RPC 响应或服务器请求。客户端向服务器的信息则通过 endpoint 指定的 POST URL 发送 JSON-RPC 请求/通知。其通信过程如图 2.3 所示。

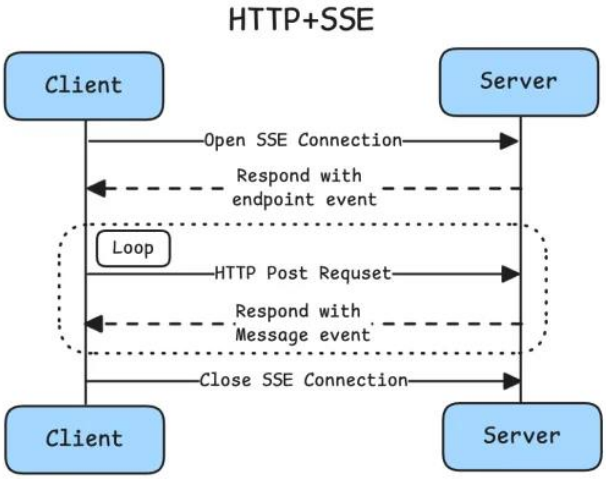

图 2.3　SSE 传输协议通信过程示意图

2. FastMCP 库 SSE 协议的实现

SseServerTransport 类是 SSE 传输协议的核心实现，提供了两个 ASGI 应用来支持双向通信。该类通过内存流（Memory Object Stream）机制管理客户端会话，并利用 UUID 作为会话标识符，确保每个 SSE 连接都有唯一的会话 ID。该 ID 通过查询参数 session_id 与客户端 POST 请求关联，实现双向通信会话绑定。

在服务器端，connect_sse 方法负责建立 SSE 连接并推送服务器消息。当客户端向服务器的/sse 端点发起 GET 请求时，该方法会创建一个新的会话，并通

过 EventSourceResponse 向客户端发送事件流。handle_post_message 方法处理客户端通过 POST 请求发送的指令。该方法会验证会话 ID，并将客户端消息传递给相应的处理流。

3. SSE 传输协议的缺点

SSE 传输方式存在一些缺点，比如需要保持长期挂起的 SSE 连接，负载均衡器必须启用 sticky session（粘性会话），否则第二个 HTTP POST 请求可能落到不同实例，导致会话丢失，对基础设施要求较高。每个客户端都需要一个专用的、常驻内存的 SSE 连接，资源开销较大。此外，该协议难以支持大数据量的双向信息交流，仅能 SSE 单向收+POST 单向发，在需要完全双向交流的场合（例如实时进度推送、需要权限认证等）无法使用 SSE 协议。

因此在**新的 MCP 协议中已经弃用 SSE 协议**，但为了兼容旧代码，目前新版本的 FastMCP 库仍然能够使用 SSE 传输模式。

2.1.3 Streamable HTTP 传输协议

Streamable HTTP 传输协议提供了一种先进的双向流式通信机制，基于 HTTP POST/SSE 混合模式实现，这种混合模式允许服务器在单个连接上持续向客户端推送多个事件，直到请求完成。POST 请求必须同时接受 application/json 和 text/event-stream 两种内容类型，确保客户端能够处理流式响应。服务器根据请求的 Accept 头部决定使用 JSON 响应还是 SSE 流模式。使用该协议的服务器仅需要提供一个挂载端点/mcp，而不需要像 SSE 协议那样提供/sse 和/message 两个端点，配置服务器更加方便，客户端连接的设置也更加方便。

1. Streamable HTTP 协议架构

协议支持真正的双向通信：客户端通过 POST 请求发送消息；服务器通过 SSE 流推送响应和通知；服务器可通过 GET 请求建立独立的 SSE 流，主动向客户端发送请求和通知。这种设计使得服务器能够在需要时主动与客户端交互，而不仅限于响应客户端请求。其通信过程如图 2.4 所示。

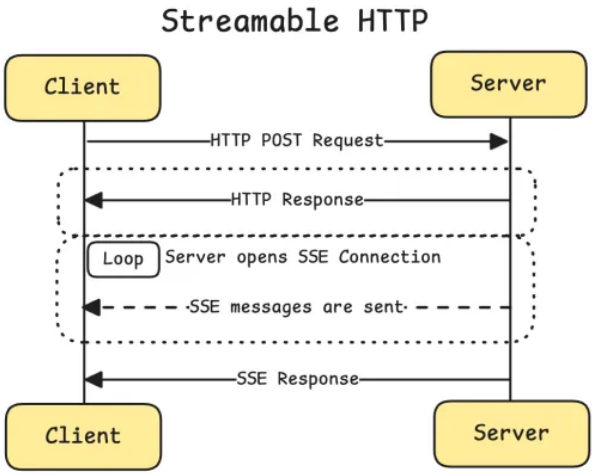

图 2.4　Streamable HTTP 传输协议通信过程示意图

Streamable HTTP 协议可选择 JSON 响应模式或 SSE 流模式。前者简单的请求-响应场景，但缺乏实时流式传输的能力；后者提供真正的流式体验，支持服务器在请求处理过程中持续发送更新。两种模式的特性对比如表 2.1 所示。

表 2.1　JSON 响应模式和 SSE 流模式特性对比表

特性	JSON 模式	SSE 模式
响应类型	单个 JSON 响应	多个 SSE 事件
实时性	低	高
适用场景	简单操作	复杂流式操作
资源消耗	低	中等
断线续传	不支持	支持

2. FastMCP 库 Streamable HTTP 协议的实现

MCP 协议通过 mcp-protocol-version 头部实现协议版本协商。客户端在请求中包含此头部以指定支持的协议版本。如果未提供，则使用默认版本。服务器验证请求的协议版本是否在支持的版本列表中。如果不支持，返回 400 Bad Request 错误，包含支持的版本列表。

MCP 协议通过 mcp-session-id 头部实现会话持久化。当客户端发起初始化请求时，服务器生成唯一的会话 ID 并通过响应头部返回。后续请求必须包含此会话 ID 以维持会话状态。会话 ID 必须仅包含可见 ASCII 字符（0x21-0x7E），确保兼容性。服务器验证每个请求的会话 ID，拒绝无效或过期的会话，这样能够有效控制服务器资源的占用。

协议支持通过 DELETE 请求实现显式会话终止。客户端发送 DELETE 请求后，服务器关闭所有相关流并释放会话资源。此后，使用相同会话 ID 的请求将返回 404 Not Found。此机制允许客户端主动清理会话状态，避免资源泄漏。服务器在终止会话后，所有相关内存流都会被关闭和清理。

EventStore 接口是实现断线续传功能的核心。它定义了两个关键方法：store_event: 存储事件以供后续检索；replay_events_after: 重放指定事件 ID 之后的所有事件。该接口的实现负责持久化事件并支持基于事件 ID 的增量重放。InMemoryEventStore 是 EventStore 接口的一个简单实现，使用内存中的双端队列存储事件。每个流最多保留指定数量的最近事件，自动移除最旧的事件以控制内存使用。当客户端断开连接后重新连接时，可通过 Last-Event-ID 头部指定最后接收到的事件 ID，它在连接恢复中起关键作用。服务器据此调用 replay_events_after 方法重放后续所有事件，确保客户端不会丢失任何消息，提高了系统的可靠性和用户体验。

3. Streamable HTTP 的特点

该协议支持会话持久化、显式会话终止和断线续传等高级功能，适用于需要可靠实时通信的分布式系统和 Web API 集成场景。

2.1.4 MCP 传输模式小结

上述三种传输协议的总结对比如表 2.2 所示。

表 2.2　三种传输协议总结对比表

协议	通信模式	典型应用场景	优势	局限性
Stdio	标准输入输出	本地开发、与代码编辑器（如 VS Code）集成	实现简单、低延迟、生命周期由客户端管理	仅限本地进程间通信
SSE	服务器发送事件	需要服务器向客户端推送实时更新的场景	支持服务器推送、适合远程工具	主要是单向通信（服务器到客户端）
Streamable HTTP	单一 HTTP 端点	云原生部署、需要双向通信和状态管理的远程服务	无状态设计利于扩展、支持双向流和断点续传	相对较新，需要特定版本的 FastMCP（如 v2.3.0+）

由于我们开发的 MCP 服务需要在远程服务器上运行，因此不可使用 STDIO 传输协议。又因为新的 MCP 协议已经弃用 SSE 协议，根据 FastMCP 官方文档 *Deployment—Running Your Server—SSE Transport (Legacy)*部分（https://gofastmcp.com/v2/deployment/running-server#sse-transport-legacy），建议仅在旧客户端只支持 SSE 协议的情况下使用 SSE 协议，在新的服务器中使用 Streamable HTTP 协议替代 SSE 协议，所以**建议使用 Streamable HTTP 传输协议开发 MCP 服务器**。在本地测试的时候可以使用 STDIO 传输协议。

MCP 服务器基础开发

3.1 开发环境

3.1.1 Python 环境配置

本人所使用的 Python 解释器为 Python 3.12，为了正常使用 FastMCP，**必须使用 Python 3.10 及以上版本**，否则无法安装 FastMCP 库，报错内容为：

```
ERROR: Ignored the following versions that require a different python version
ERROR: Could not find a version that satisfies the requirement fastmcp (from versions: none)
ERROR: No matching distribution found for fastmcp
```

IDE 和包管理器方面，本人使用的是 PyCharm (Professional Edition) 2025.2.1 和 venv 虚拟环境[2]。

3.1.2 FastMCP 库

FastMCP 是一个专为现代 AI 应用设计的高性能 Python 软件开发工具包（SDK），支持全异步操作，专注于低延迟、高并发的连接场景。通过简洁的装饰器语法，开发者可以快速将现有函数暴露为 MCP 工具或资源，实现 MCP 服务器的快速开发。其官方网站和文档页面为 https://gofastmcp.com/。FastMCP v2.0 官方文档有中文社区翻译版本（https://fastmcp.wiki/zh/getting-started/welcome），但其翻译可能使用了生成式人工智能等工具，在翻译不清晰的情况下需要查阅原始文档或其它资料。官方 GitHub 页面为 https://github.com/jlowin/fastmcp，也被 MCP 协议组织收录在 GitHub 中（页面为 https://github.com/modelcontextprotocol/python-sdk）。**但需要注意的是，MCP 协议组织收录的是 v1.0 版本的 FastMCP，后续版本未收录，v2.0 版本新增代理和客户端采样等新功能，建议使用新版本。**

FastMCP v2.14.5 的安装和导入[3]分别使用以下代码：

```
pip install fastmcp==2.14.5
from fastmcp import FastMCP
```

MCP 协议组织收录的 FastMCP v1.0 版本安装和导入分别使用以下代码：

```
pip install mcp
from mcp.server.fastmcp import FastMCP
```

安装 FastMCP v2.0 及以上版本后虽然也可使用后者代码导入，但是，根据 MCP 协议组织收录的 GitHub 页面 README 文档中所标注，FastMCP v1.0 版本支持的 Python 版本最高仅到 3.13，因此使用高版本 Python 解释器时需要注意尽量不要导入低版本的 FastMCP。

[2] 若使用 anaconda 进行包管理，需要特别注意是否具有商用许可。
[3] 编写代码时，注意不要将自己的文件命名为 fastmcp.py，否则可能无法导入 FastMCP 库。

安装完成后，可在虚拟环境终端中使用代码“fastmcp version”查看 FastMCP 库版本和安装路径、MCP 协议版本等信息，如图 3.1 所示。

```
终端:  本地 ×  +
(.venv) PS G:\Redis_MCP\Redis_MCP> fastmcp version
FastMCP version:                                     2.14.5
MCP version:                                         1.26.0
Python version:                                      3.12.2
Platform:                           Windows-10-10.0.19045-SP0
FastMCP root path: G:\Redis_MCP\Redis_MCP\.venv\Lib\site-packages
```

图 3.1　FastMCP 库安装后基本信息查询示例

3.2 MCP 服务器代码基本结构

一个基于 FastMCP 开发的 MCP 服务器代码通常包含以下几个部分：

FastMCP 实例：服务器的入口点。

@mcp.tool()：定义可供客户端调用的函数，执行特定操作。

@mcp.resource()：定义可供客户端查询或操作的数据资源。

@mcp.prompt()：定义用于生成文本或与大型语言模型交互的提示。

示例代码如下面的代码 3.1 所示，在该服务器“Demo”中有一个加法工具、一个动态问候资源和一个用于生成问候语的提示词。

代码 3.1　MCP 服务器代码的基本结构示例

```python
from fastmcp import FastMCP

# 创建一个 MCP 服务器实例，命名为"Demo"
mcp = FastMCP("Demo" , host="127.0.0.1", port=8000)

# 添加一个加法工具
@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

# 添加一个动态问候资源
# 资源路径支持变量，例如这里的{name}
@mcp.resource("greeting://{name}")
def get_greeting(name: str) -> str:
    """Get a personalized greeting"""
    return f"Hello, {name}!"

# 添加一个 Prompt，用于生成问候语
@mcp.prompt()
def greet_user(name: str, style: str = "friendly") -> str:
    """Generate a greeting prompt"""
    styles = {
        "friendly": "Please write a warm, friendly greeting",
        "formal": "Please write a formal, professional greeting",
        "casual": "Please write a casual, relaxed greeting",
    }
```

```
    return f"{styles.get(style, styles['friendly'])} for someone named {name}."

if __name__ == '__main__':
    # 运行 MCP 服务器，使用 stdio 传输模式
    mcp.run(transport="stdio")
```

通过代码 3.1，可以看到工具、资源、提示词三大核心功能实现代码的基本结构，它们被称为“组件”。需要说明的是，这段代码仅仅用于演示基本结构，如果按照这种简要的方式编写给大模型理解的提示词，大模型可能无法理解这些工具、资源的用途和调用方式，从而无法完成正常调用流程。

接下来，将详细讲解代码的各个部分应如何编写。

3.3 MCP 服务器实例创建

创建 MCP 服务器实例时，需要使用 FastMCP()语句调用 FastMCP 类构造函数进行实例化，此构造函数可缺省全部参数，若仅提供一个参数则默认为参数“name”的值。

构造函数有新旧两种写法，旧写法将服务器的通信相关参数（详见表 3.2）在实例化时写好，在主程序入口后调用 run()方法启动服务器时仅需指定传输协议；新写法则在实例化时仅写好服务器属性相关参数，其余参数在 run()方法的参数表中写。两种写法的对比如表 3.1 所示，其中仅设置了部分参数，其余参数缺省。**注意，如果在实例化时和 run()方法中都对同一参数进行了设置，则在 run()方法中的设置会覆盖实例化时的设置。**

表 3.1　MCP 服务器实例化新旧写法对比

旧写法		新写法
mcp=FastMCP("Demo",host="127.0.0.1",port=8000) …… # 其它代码 if name ==” main ”: mcp.run(transport="sse")	→	mcp=FastMCP("Demo") …… # 其它代码 …… # 其它代码 if name ==” main ”: mcp.run(transport="sse",host="127.0.0.1",port=8000)

如果采用旧写法编写 MCP 服务器代码，则会在服务器启动运行代码时出现 warning，例如“DeprecationWarning: Providing `host` when creating a server is deprecated. Provide it when calling `run` or as a global setting instead.”即建议将 host 参数在调用 run()方法时提供。但无论采用新写法还是旧写法，理论上均不影响服务器的正常运行。由于互联网上大多数现有的资料均采用旧写法，因此本手册中在本章节中对这些参数的设定进行讲解，如果想要采用新写法，则将 DeprecationWarning 中提示的参数移到 run()方法中即可。**为了更加符合新的 FastMCP 库标准，建议采用新写法。注意，在 FastMCP v3 中，旧写法会报错。**

FastMCP 构造函数的常用参数如表 3.2 所示，可能存在部分参数未在此列出，按照新写法的 FastMCP 类构造函数中的所有参数可以查阅 FastMCP 官方文档中

The FastMCP Server 部分 https://gofastmcp.com/v2/servers/server。这些参数大多数有通过全局变量设置的方法，**但构造函数参数的设置优先级高于全局变量设置**。

表 3.2　FastMCP 构造函数常用参数表

参数	描述	数据类型	默认值	备注
name	FastMCP 服务器可读名称	str | None	None	
host	要监听的主机地址	str | None	None	
port	要监听的端口	int | None	None	
log_level	设置 FastMCP 服务器日志级别，用于筛选输出的日志信息	Literal[“DEBUG”, “INFO”, “WARNING”, “ERROR”, “CRITICAL"]	“INFO”	
debug	是否开启调试模式	bool	False	优先级高于 log_level
json_response	设置 HTTP POST 请求相应格式为 SSE（False）或 json（True）	bool	False	共同决定 FastMCP 服务器工作模式（仅适用 HTTP 模式，其它模式下这两个参数无效）
stateless_http	启用无状态/有状态 HTTP 模式	bool	False	
sse_path	客户端建立 SSE 长连接的端点路径	str | None	None	适用于 SSE 模式
message_path	客户端向服务器发送请求消息的端点路径	str | None	None	
streamable http_path	Streamable HTTP 模式下的统一通信端点路径	str | None	None	适用于 Streamable HTTP 模式
lifespan	生命周期钩子，一个用于控制服务器启动和关闭逻辑的异步上下文管理器函数	AsyncContextManager | None	None	
version	FastMCP 服务器版本	str | None	None	参数缺省则默认为 FastMCP 库版本
instructions	描述服务器用途、功能和交互方式，帮助客户端理解	str | None	None	
auth	用于保护基于 HTTP 传输的身份验证提供程序	OAuthProvider | TokenVerifier | None	None	详见《MCP 服务器的身份验证》
include_tags	仅暴露具有至少一个匹配标签的组件	set[str] | None	None	v2.8.0 版本新增 标签管理详见§3.4
exclude_tags	隐藏具有任何匹配标签的组件		None	
on_duplicate_tools	如何处理重复的工具注册	Literal["error", "warn", "replace"]	“warn”	详见§3.5.12
on_duplicate_resources	如何处理重复的资源注册		“warn”	详见§3.6.10
on_duplicate_prompts	如何处理重复的提示词注册		“warn”	详见§3.7.7
tools	工具函数列表	list[Tool|Callable] | None	None	与@mcp.tool 装饰器功能相同，**大多数情**

				况下可缺省该参数
icons	FastMCP 服务器图标	list[Icon] | None	None	v2.13.0 版本新增 详见§5.3.1
website_url	提供有关服务器更多信息的网站的 URL，显示于客户端应用程序中	str | None	None	v2.13.0 版本新增
mask_error_details	是否隐藏错误细节	bool	False	工具组件的错误管理详见§3.5.8
strict_input_validation	是否启用严格认证，控制如何验证工具输入参数	bool	False	v2.13.0 版本新增
include_fastmcp_meta	是否在组件响应中包含 FastMCP 元数据	bool	True	v2.11.0 版本新增
tool_serializer	工具序列化器，将 Python 对象转为 MCP 协议消息	ToolResultSerializerType | None	None	通常使用默认的（序列化为 JSON），要更换序列化格式时可用自定义的，详见§3.5.13

注：此表中浅灰色标明的参数即为新版本 FastMCP 建议移到 run()方法的参数。FastMCP v3 版本对旧写法会报错，错误为 TypeError 并附带迁移提示。蓝灰色标明的参数为 FastMCP v3 中不再支持写在 FastMCP 构造函数的参数，其中部分参数已不再支持设置，仍支持设置的参数会在报错同时附带迁移建议。

在下面的章节中，将对一些参数进行额外的补充说明。

3.3.1 [name] FastMCP 服务器可读名称

构造函数仅提供一个参数时默认为参数“name”的值。若不提供任何参数，则自动以“FastMCP-xxxx”的形式**随机**为创建的服务器命名，但每次运行时的名称不相同，因此建议为服务器提供一个确定的名称。

该名称为供人类阅读的可读名称，理论上其内容不影响智能体、大模型之类的工具与 MCP 服务器的连接。

3.3.2 [host] 监听主机地址

参数缺省时，默认监听地址 127.0.0.1，即本地回环地址 localhost，此时 FastMCP 服务器仅监听本机内部连接，不接受外部网络访问，常用于开发测试。

当此参数设为 0.0.0.0 时，表示“所有可用的网络接口”或“任意地址”，此时 MCP 服务器向局域网内甚至互联网上所有设备（当有公网 IP 且端口开放、正确设置防火墙时）开放，常用于生产环境。建议在允许外部访问时，检查防火墙配置、身份验证、信息加密等安全措施。

3.3.3 [port] 监听端口

通常低于 1024 的端口已经被系统服务或通用软件占用，应避免使用。

建议使用 1024 到 65535 之间的端口，在 MCP 服务器这类 Web 应用开发中常用 8000、8080、9000 等端口。启动 MCP 服务前，可在命令提示符中使用命令“netstat -ano | findstr :<端口号>”检查端口是否已被占用。如果在同一服务器上运行多个 FastMCP 服务时，host 参数可以相同，但 port 参数不可相同。

3.3.4 [log_level 和 debug] 日志级别和调试模式

FastMCP 服务器在启动后会自动记录运行日志，日志条目被分为 5 个级别“DEBUG”、“INFO”、“WARNING”、“ERROR”、“CRITICAL”，DEBUG 级条目主要记录服务器向客户端发送的调试信息和自定义的调试信息，INFO 级条目主要记录网络连接信息，WARNING 级条目主要记录警告，ERROR 级条目主要记录故障信息，CRITICAL 级条目记录导致服务器完全停止运行的致命错误。图 3.2 为一个 FastMCP 服务器日志的样例，包含 DEBUG、INFO、ERROR 级条目，日志条目级别在每个条目的最左侧注明。

在图 3.2 所示的 FastMCP 服务器工作过程中，先是启动了服务器，接着使用 MCP Inspector 测试工具连接了该服务器，然后获取了工具列表并调用了一个工具，在该工具的函数实现中包含向客户端依次发送 INFO、WARNING、ERROR 级的调试信息，但对于服务器端而言，所有发送出去的调试信息全部记录为 DEBUG 级。再然后 MCP Inspector 关闭了与服务器的连接。最后关闭 FastMCP 服务器，结束进程。在本章节中仅展示各种级别的日志信息，如何使用工具调试服务器、如何向客户端发送调试信息等在后续章节中介绍。

```
[02/06/26 14:15:37] INFO     Starting MCP server 'Demo_logging'  server.py:2580
                             with transport 'streamable-http' on
                             http://127.0.0.1:8000/mcp
INFO:     Started server process [23856]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     127.0.0.1:50114 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:50114 - "POST /mcp HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:50114 - "GET /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:50118 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:50133 - "POST /mcp HTTP/1.1" 200 OK
[02/06/26 14:15:52] DEBUG    Sending INFO to client:            context.py:1044
                             my_info_log 1 and 2
                    DEBUG    Sending WARNING to client:         context.py:1044
                             my_warning_log 1 and 2
                    DEBUG    Sending ERROR to client:           context.py:1044
                             my_error_log 1 and 2
INFO:     127.0.0.1:50133 - "DELETE /mcp HTTP/1.1" 200 OK
INFO:     Shutting down
ERROR:    Cancel 0 running task(s), timeout graceful shutdown exceeded
INFO:     Waiting for application shutdown.
INFO:     Application shutdown complete.
INFO:     Finished server process [23856]
```

图 3.2　一个 FastMCP 服务器完整日志样例

log_level 用于设置 FastMCP 服务器记录的日志级别，共有 5 个级别，从详细到简略分别是“DEBUG”、“INFO”、“WARNING”、“ERROR”、“CRITICAL”，参数缺省时默认为“INFO”级。将 debug 参数设置为 True 与将 log_level 设置为

DEBUG 效果相同，**但 debug 参数的设置优先级高于 log_level 参数的设置**。

设定日志级别后，仅严重程度高于或等于设定级别的日志信息会被记录。例如当 log_level 设为 WARNING 后，仅有 WARNING、ERROR、CRITICAL 级的日志信息会被记录，DEBUG、INFO 级的信息不会被记录在日志中。

需要注意的是，一个 FastMCP 服务器中可能包含多个日志记录器，记录“服务器本身情况”的和记录“向客户端发送调试信息”的是两个记录器，仅将 log_level 设置为 INFO 或更简略的级别不能停止记录“向客户端发送调试信息”这类 DEBUG 级日志。可通过对指定的记录器设置日志级别实现精确控制。

在生产环境中，通常将 log_level 设定为“INFO”或“WARNING”。在开发测试环境中，可不改变 log_level 的设置，而将 debug 设为 True。

3.3.5 [json_response 和 stateless_http] 服务器工作模式设定

json_response 参数为 True 时，向客户端返回标准 JSON 对象。参数为默认的 False 时，使用 SSE 流式响应 HTTP POST 请求，适用于需要实时推送（如工具执行进度）的场景。

stateless_http 参数为 True 时开启无状态 HTTP 模式，此模式下每个请求都会创建一个全新的 StreamableHTTPServerTransport 实例，请求处理完毕后，该实例立即被销毁，这种模式适用于无状态的微服务或需要在多节点间负载均衡的场景。当参数为 False 时，使用默认的有状态 HTTP 模式，此模式下会话状态（如服务器内部的连接流）在会话 ID 存在期间被持久化——当请求包含有效的 MCP-Session-ID 时，说明是现有会话，复用已有的 StreamableHTTPServerTransport 实例来处理请求；当请求不包含 MCP-Session-ID 时，说明是新会话，要创建一个新的 StreamableHTTPServerTransport 实例，并生成一个唯一的会话 ID。

无状态 HTTP 模式常用于生产环境服务器部署，因其不用维护会话 ID 和上下文信息，对服务器资源的占用较少，适合高并发和负载的服务器场景。

这两个参数仅适用于 Steamable HTTP 传输模式，当使用传统的 Stdio 传输模式和 SSE 传输模式时，这两个参数的设置会被忽略。

参数 json_response 和 stateless_http 组合，共同决定了 FastMCP 服务器的工作模式，如表 3.3 所示。有些服务器托管服务或智能体平台等对 MCP 服务器的工作方式有要求，需要按照这些服务的官方文档设置这两个参数。

表 3.3　FastMCP 服务器工作模式

服务器工作模式	参数设置		特点	应用场景
	json_response	**stateless_http**		
有状态流式传输	False	False	有会话，支持 SSE 流与进度通知	需要交互和反馈的 AI 工作流
有状态 JSON 传输	True	False	有会话，但每次请求返回完整 JSON	需要维持状态但客户端不支持 SSE 的场景

无状态流式传输	False	True	无会话，但使用 SSE 流	无
无状态 JSON 传输	True	True	无会话，每次请求独立并返回 JSON	服务器部署、OpenAI API 集成、简单工具调用

表 3.3 中，“无状态流式传输”工作模式存在内部矛盾，“无状态”要求服务器不保存任何会话信息，而 SSE 流式传输又常用于交互反馈（例如工作进度通知），特性和用途是矛盾的，无法发挥优势。这种模式下无法实现包含会话信息的交互反馈，无需使用 SSE 传输。虽然参数如此设定不会产生报错，但是性能较差并且没有实际应用场景，不推荐。

3.3.6 [sse_path, message_path, streamable_http_path] 通信端点路径设置

传统 SSE 模式下，sse_path 参数用于指定客户端监听服务器事件的路径，参数缺省时默认为“/sse”。message_path 参数用于指定客户端发送请求的路径，参数缺省时默认为“/messages”。假设两者均缺省，host 参数设为 127.0.0.1，port 参数设为 8000，则需提供给 AI 应用的 MCP 服务器 URL 为 http://127.0.0.1:8000/sse。这两个参数仅在使用 SSE 传输模式时有效，其它传输模式下这两个参数的设置会被忽略。

在 Streamable HTTP 模式下，客户端与服务器间通信仅需提供一个路径，参数缺省时默认为“/mcp”。假设两者均缺省，host 参数设为 127.0.0.1，port 参数设为 8000，则需提供给 AI 应用的 MCP 服务器 URL 为 http://127.0.0.1:8000/mcp。该参数仅在使用 Streamable HTTP 传输模式时有效，其它传输模式下该参数的设置会被忽略。

按照新版本 FastMCP 的规范，这三个参数的设置应移到 run()方法中。此外，v3.0 版本中 run()方法中的参数 path 可替代这三个参数，可用于指定所有传输方式的端点路径，更加方便。并且 **run()方法中 path 参数的优先级高于上述三个参数**。因此**不建议使用这三个参数指定端点路径，而应当在 run()方法中使用 path 参数**，具体使用方法在关于 FastMCP 服务器的运行的 § 3.8.1 小节会再介绍。

3.3.7 [lifespan] 生命周期钩子

lifespan 是一个异步上下文管理器（Async Context Manager），用于管理服务器的启动和关闭逻辑。当此参数缺省时，FastMCP 实例会使用默认的 lifespan 实现，它只是简单地返回一个空字典，不执行任何特殊操作。

自定义 lifespan 的典型应用场景包括数据库连接池初始化、缓存预热、外部服务注册等。lifespan 函数的返回对象为一个包含应用上下文对象的迭代器，可在后续工具函数中通过 ctx.request_context.lifespan_context 访问。

如果想要自定义 lifespan，则需要定义一个异步上下文管理器，将其作为 lifespan 参数传递给 FastMCP 构造函数。

先要使用以下代码从 contextlib 库中导入 asynccontextmanager：

```
from contextlib import asynccontextmanager
```

在定义该异步上下文管理器时应使用@asynccontextmanager 装饰器。

代码 3.2 是一个管理数据库连接的 lifespan 函数定义与作为参数调用示例，先定义了一个异步上下文管理器，然后在 FastMCP 构造函数中将其作为参数的值传入 lifespan 参数中。

代码 3.2　MCP 服务器生命周期钩子定义与使用示例

```
 1. @asynccontextmanager
 2. async def app_lifespan(server: FastMCP) -> AsyncIterator[AppContext]:
 3.     # 启动时初始化
 4.     db = await Database.connect()
 5.     try:
 6.         yield AppContext(database=db)
 7.     finally:
 8.         # 关闭时清理
 9.         await db.disconnect()
10.
11. mcp = FastMCP("My App", lifespan=app_lifespan)
```

我们逐行分析一下代码 3.2。第 1 行为@asynccontextmanager 装饰器，用于标识该函数为异步上下文管理器，将函数转换为管理器。第 2 行定义了异步函数 app_lifespan，接收 FastMCP 服务器实例作为参数，返回一个异步迭代器（产生的值类型为 AppContext）。代码的执行流程分为 3 个阶段：①启动阶段，在应用启动阶段执行，使用第 4 行代码初始化以建立数据库连接等；②运行阶段，使用第 6 行代码将初始化的资源（数据库连接）包装成 AppContext，该对象在应用运行期间可用，代码在此处暂停直到上下文退出；③关闭阶段，执行 finally 块中的内容，第 9 行代码关闭数据库连接，确保资源正确释放。在 FastMCP 实例化时（第 11 行），将 app_lifespan 管理器作为 lifespan 的参数值传入。

与同步上下文管理器类似，yield 之前的代码在进入上下文时执行，yield 之后的代码在退出上下文时执行，yield 语句的返回值赋给 with 语句中的变量。代码 3.2 中 app_lifespan 函数的返回值并不是传统意义上的返回值，而指的是 yield 语句的返回值，必须由 with … as …语句获取返回值，with … as …语句已被包含在 ctx.request_context.lifespan_context 的实现函数中，不需要我们写，我们可在后续工具函数中通过 ctx.request_context.lifespan_context 访问返回值。此例中，可用语句“db = ctx.request_context.lifespan_context.database”获取数据库连接。该样例参考了 MCP 协议官方 GitHub 中的样例 https://github.com/modelcontextprotocol/python-sdk/blob/v1.19.0/examples/snippets/servers/lifespan_example.py。

代码 3.3 是另一个示例，包含了工具函数中的使用案例。

代码 3.3　MCP 服务器生命周期钩子与工具函数中调用使用示例

```
1. from contextlib import asynccontextmanager
2. from fastmcp import FastMCP, Context
3.
4. # 1. 定义一个异步上下文管理器作为 lifespan
5. @asynccontextmanager
6. async def database_lifespan(server: FastMCP):
7.     # 服务器启动时执行初始化逻辑
8.     print("正在连接数据库...")
9.     db_connection = {"connection": "模拟的数据库连接"}
10.    try:
11.        # yield 一个字典，它将作为上下文传递给工具函数
12.        yield {"db": db_connection}
13.    finally:
14.        # 服务器关闭时执行清理逻辑
15.        print("正在关闭数据库连接...")
16.        # 在实际应用中，这里会关闭数据库连接
17.
18. # 2. 创建 FastMCP 服务器，使用自定义 lifespan
19. server = FastMCP(
20.     name="带数据库的服务器",
21.     lifespan=database_lifespan
22. )
23.
24. # 3. 定义一个工具，使用 lifespan 上下文
25. @server.tool()
26. def query_data(query: str, ctx: Context):
27.     # 通过上下文访问数据库连接
28.     db = ctx.request_context.lifespan_context["db"]
29. return f"使用连接 {db['connection']} 查询: {query}"
30.
31. # 4. 运行服务器
32. if __name__ == "__main__":
33.     server.run()
```

代码 3.3 中，database_lifespan 函数未显式注明 yield 返回值类型，但在注释中说明了其为 dict 型，”db”键对应的值为数据库连接，如果想要注明返回值类型，则可将第 6 行写作“async def database_lifespan(server: FastMCP) -> AsyncIterator[dict]:”，即函数返回值类型记作“AsyncIterator[yield 返回值类型]”。第 28 行即为在工具函数中通过 ctx.request_context.lifespan_context 访问上下文管理器获取数据库连接。

理论上来说，使用 lifespan 的方式与传统方式相比，不需要每个请求都先连接再断开数据库，优化了性能和可靠性。但根据 FastMCP 官方 Github 中的 issue 区有报告称 lifespan 无法起效，实际上仍然每次请求都重新建立连接，“生命周期事件是在客户端建立连接时触发的，而非在应用启动或关闭时触发”（https://github.com/jlowin/fastmcp/issues/775），对此 FastMCP 的开发者称“目前是基于会话级别而非应用级别触发”，在后续的新版本 FastMCP 库中可能会对此调整，“在此之前，除非我们自行引入更高级别的生命周期管理机制，否则

FastMCP 将保持相同的行为模式”。

当使用 mount()方法组合多个服务器时，子服务器的 lifespan 会在主服务器的 lifespan 内自动执行，即主服务器启动时子服务器自动启动，主服务器关闭时子服务器自动关闭，仅需对主服务器操作即可。

3.3.8 [include_tags 和 exclude_tags] 基于标签的组件管理

对于服务器上的工具、资源、提示词，可以为每个组件打上一个或多个标签。如果连接到服务器的某客户端可以得知服务器上某工具的存在并能正常调用该工具，则称为服务器向该客户端“暴露”了该工具。这两个参数即用于根据工具等组件的标签，有选择性地向客户端暴露组件。

若参数 include_tags={A,B}，则只有标签同时包含 A 和 B 的组件可被暴露。

若参数 exclude_tags={A,B}，则标签包含 A 或 B 的组件均不被暴露。

exclude_tags 的优先级始终高于 include_tags。

具体利用标签进行组件管理的方法详见§ 3.4。

3.3.9 [tools] 工具列表

FastMCP 的工具的代码以函数形式编写，需要标识出来以便将函数形式的工具转换为 MCP 工具，tools 参数即为将函数注册为工具的方式之一。

在互联网上大多数开源的 FastMCP 代码中，均通过@mcp.tool 装饰器将工具函数转换为可被 MCP 客户端识别的 MCP 工具。但也可以通过 tools 参数将单个或多个工具一起添加到 MCP 服务器中。装饰器通常用于静态工具定义场景，而参数注册也由 Tool.from_tool()转换得到的独立工具对象（FastMCP 工具转换详见§ 3.5.11小节），适合动态加载工具的场景，两种注册方法对比如表 3.4 所示。

表 3.4 FastMCP 工具注册方法对比

使用@mcp.tool 装饰器	使用构造函数的 tools 参数
1. from fastmcp import FastMCP 2. 3. mcp = FastMCP(name="Server") 4. 5. @mcp.tool() 6. def add(a: float, b: float) -> float: 7. """Adds two numbers together.""" 8. return a + b 9. 10. @mcp.tool() 11. def greet(name: str) -> str: 12. """Greet someone by name.""" 13. return f"Hello, {name}!"	1. from fastmcp import FastMCP 2. 3. def add(a: float, b: float) -> float: 4. """Adds two numbers together.""" 5. return a + b 6. 7. def greet(name: str) -> str: 8. """Greet someone by name.""" 9. return f"Hello, {name}!" 10. 11. # 在构造函数中通过 tools 参数注册 12. mcp = FastMCP(13. name="Server", 14. tools=[add, greet] # 将函数列表传入 tools 参数)

需要注意的是，表 3.4 中所示的两种工具注册方式的 FastMCP 构造函数编写

位置不同，一个在工具函数之前，另一个则在工具函数之后。

3.3.10 [strict_input_validation] 是否启用严格输入认证

strict_input_validation 控制如何验证工具输入参数，关于输入验证模式的样例代码等可以参考 FastMCP 官方文档中的 *Tools—Validation Modes* 部分 https://gofastmcp.com/v2/servers/tools#validation-modes。

当为默认值 False 时 FastMCP 使用 Pydantic 的灵活验证，该验证会尝试自动转换兼容的输入为正确的类型（例如，对于要求为 int 的参数，将输入的"10"自动转换为 int 型的 10）。默认模式更加宽松，提高了与 LLM 客户端的兼容性，同时保持了类型安全。建议在大多数应用场景中使用默认模式。

当为 True 时，使用 MCP SDK 的 JSON Schema 验证，在将输入传递给函数之前，根据确切的模式验证输入，拒绝任何类型不匹配。适合用于需要确保输入类型绝对精确或与其他严格遵循 JSON Schema 的系统交互的场景。

关于 Pydantic 模型的重要说明：即使将 strict_input_validation 设置为 False，Pydantic 模型参数也必须以 JSON 对象（即 Python 字典）的形式提供，而不能是字符串化的 JSON。例如，{"user": {"name": "Alice"}}可以正常工作，而{"user": '{"name": "Alice"}'}将无法正常工作。

3.3.11 [include_fastmcp_meta] 是否省略组件元数据

include_fastmcp_meta 参数决定了是否省略组件元数据（例如标签、版本等）。

当为默认值 True 时，组件标签和其他 FastMCP 特定的元数据包含在每个组件的 meta 字段内的_fastmcp 命名空间中，客户端可以获取组件的这些元数据信息。此时客户端可以通过 meta._fastmcp.tags 来筛选具有特定标签的工具。

当为 False 时，将省略此元数据，从而实现与外部系统更简洁的集成。**可通过 FASTMCP_INCLUDE_FASTMCP_META 环境变量全局覆盖此设置，与其它参数设置时的而优先级不同。**

表 3.5 是设置不同的 include_fastmcp_meta 参数下，MCP 服务器针对 tools/list 方法（即客户端请求获取工具列表）的原始响应。

表 3.5 不同的 include_fastmcp_meta 参数下 MCP 服务器响应对比

include_fastmcp_meta=True	include_fastmcp_meta=False

{
"tools": [
{
"name": "add",
"description": "将两个整数相加\nArgs:\n a:第一个加数\n b:第二个加数\nReturn:\n 两个数 a 和 b 的和",
"inputSchema": {
"type": "object",
"properties": {
"a": {
	{
	"tools": [
	{
	"name": "add",
	"description": "将两个整数相加\nArgs:\n a:第一个加数\n b:第二个加数\nReturn:\n 两个数 a 和 b 的和",
	"inputSchema": {
	"type": "object",
	"properties": {
	"a": {

```
                "type": "integer"
            },
            "b": {
                "type": "integer"
            }
        },
        "required": [
            "a",
            "b"
        ]
    },
    "outputSchema": {
        "type": "object",
        "properties": {
            "result": {
                "type": "integer"
            }
        },
        "required": [
            "result"
        ],
        "x-fastmcp-wrap-result": true
    },
    "  meta": {
        "  fastmcp": {
            "tags": [
                "public"
            ]
        }
    }
  }
]
}
```


表3.5所示的样例中，该FastMCP服务器包含一个工具add，其标签为“public”。当 include_fastmcp_meta 参数为 True 时，服务器对 tools/list 的响应比参数为 False 时多一个“_meta”字段，该字段包含命名空间“_fastmcp”，里面即为该工具的标签。参数为 True 时，每个组件的响应中均会包含“_meta”字段信息。

3.4 MCP 组件标签管理

FastMCP 组件标签是附加在工具（Tools）、提示（Prompts）、资源（Resources）和资源模板（Resource Templates）这四种核心组件上的文本标记，主要用于对组件进行分类、筛选和控制访问权限。其核心功能是精确控制组件的可见性，服务器可以基于标签动态地决定向特定客户端暴露或隐藏哪些组件。

3.4.1 为组件添加标签

通常在组件的装饰器（例如@mcp.tool、@mcp.resource、@mcp.prompt 等）上添加标签，格式形如“@mcp.tool(tags={“test”})”。每个组件可以添加一个或多个标签，标签之间以逗号分隔，例如“@mcp.tool(tags={“test1”, “test2”})”。而对于不使用装饰器注册的组件，在动态注册使用的方法中存在 tags 参数，用形如

“tags={“test”}”的方式提供参数值即可添加标签，例如动态资源注册方法 mcp.add_resource、工具转换方法 Tool.from_tool 等均有 tags 参数，使用语句“mcp.add_resource(uri_template="resource://config/app",tags={"test","public"})”、“ sum_tool=Tool.from_tool(func=add,name="add",tags={"math","arithmetic","public"})”即可在注册的同时为相应组件添加标签。

当 include_fastmcp_meta=True 时，标签的内容将会被包括在 MCP 服务器响应 tools/list 之类的请求时的“_meta”—“_fastmcp”字段中，样例请详见§3.3.11 小节的表 3.5。

3.4.2 组件可见性控制

1. 服务器全局静态过滤

在§3.3.8 小节中，已经简要介绍了在 FastMCP 实例化的构造函数中，可以使用 include_tags 和 exclude_tags 参数进行全局控制。这种方式的过滤是静态的，在一次运行中不能改变过滤规则，适合在调试时使用。例如为一些测试功能添加“test”标签，在正式的版本添加将带有“test” 标签的组件过滤掉的规则。

include_tags 使用的是“与”逻辑，即若参数 include_tags={A,B}，则只有标签同时包含 A 和 B 的组件可被暴露。exluce_tags 使用的是“非”逻辑，即若参数 exclude_tags={A,B}，则标签包含 A 或 B 的组件均不被暴露。**后者优先级高于前者。**因此可以将两者结合起来使用，例如有参数设置“include_tags={"A"}, exclude_tags={"B"}”，则暴露带有标签 A 且不含标签 B 的组件。

2.运行时动态控制过滤

这是 FastMCP v3.0 的新功能，因版本较新，在使用时需要注意 FastMCP 库版本和对应的客户端版本是否支持。因为目前最新的稳定版本是 FastMCP v2.14，v3.0 仍在测试中，所以这部分内容仅略作介绍，详细关于 FastMCP v3.0 中组件的可见性控制请查阅 https://gofastmcp.com/servers/visibility。

该功能使得可以在服务器运行时，根据当前用户角色或会话状态，调用 mcp.enable() 或 mcp.disable() 方法来启用或禁用某个特定组件或带有特定标签的组件。FastMCP v3.0 规定了键（Key）的格式，使键作为各个组件的唯一标识，键的格式由组件类型决定，如表 3.6 所示。键使精准动态过滤得以实现。

表 3.6 不同组件的键的格式

组件	键的格式	样例
工具（Tool）	tool:{组件名}	tool:delete_everything
资源（Resource）	resource:{uri}	resource:data://config
模板（Template）	template:{uri}	template:file://{path}
提示词（Prompt）	prompt:{组件名}	prompt:analyze

mcp.enable() 或 mcp.disable() 方法有参数 keys 和 tags，keys 对应具体组件，tags 则对应标签，两者可以结合以实现不同精细程度的动态过滤。例如语句

“mcp.disable(keys={"tool:debug_info"}, tags={"dangerous"})”设置了组件 tool:debug_info 和带有标签“dangerous”的组件不可见。

3.5 MCP 工具组件

工具（Tool）组件是 FastMCP v2 版本中服务端三大组件中最常用的组件，通过工具组件的调用，MCP 客户端的 AI 应用可以便捷地增加功能，例如数值运算、文件管理、数据库增删查改等。

3.5.1 工具组件异步支持

在互联网上查阅资料时，可以看到有些代码的组件函数带有 async 关键字修饰，有些则不带有 async 关键字。这是因为使用 FastMCP 库开发 MCP 服务器时，组件函数既可以定义为同步形式，又可以定义为异步形式。

理论上来说，同步函数适合无 I/O 的纯 CPU 计算任务，异步函数适合需要 I/O 操作、网络请求或并发执行的函数。但同步函数在执行时可能会阻塞事件循环，此外 FastMCP 是一个异步优先的框架，在异步函数中可以调用同步函数，后者却不能调用前者，而且在同一个服务器中最好使用相同风格，因此建议在编写新的 MCP 服务器代码时优先使用异步形式。

FastMCP 官方文档中 *Tools—Async Support* 部分（https://gofastmcp.com/v2/servers/tools#async-support）提供了一种将现有的同步工具安全地转为异步形式的方法，即利用 FastMCP 内部已经包含的 anyio 库，将现有的同步工具安全地包装为异步函数，如代码 3.4 所示。

代码 3.4　使用 anyio 库将同步函数包装为异步函数示例

```
1. import anyio
2. from fastmcp import FastMCP
3.
4. mcp = FastMCP()
5.
6. def cpu_intensive_task(data: str) -> str:
7.     # Some heavy computation that could block the event loop
8.     return processed_data
9.
10. @mcp.tool()
11. async def wrapped_cpu_task(data: str) -> str:
12.     """CPU-intensive task wrapped to prevent blocking."""
13.     return await anyio.to_thread.run_sync(cpu_intensive_task, data)
```

代码 3.4 中，函数 cpu_intensive_task 为一个同步函数，通过第 10~13 行中的代码使用 anyio.to_thread.run_sync()方法异步调用该函数，安全地将其包装为一个异步函数，作为工具组件使用，避免了同步函数易阻塞的问题。

官方文档中还提到，其它替代方案包括使用 asyncio.get_event_loop().run_in_executor() 或其他线程技术来管理阻塞操作，从而避免影响服务器的响应性。例如，以下是使用 asyncer 库（FastMCP 未内置该库）创建包装同步函数的装饰器的方法示例，由@hsheth2 提供，如代码 3.5 所示。

代码 3.5　使用 anyncer 库编写装饰器将同步函数包装为异步函数示例

装饰器定义代码

```
1. import asyncer
2. import functools
3. from typing import Callable, ParamSpec, TypeVar, Awaitable
4.
5. _P = ParamSpec("_P")
6. _R = TypeVar("_R")
7.
8. def make_async_background(fn: Callable[_P, _R]) ->
Callable[_P, Awaitable[_R]]:
9.     @functools.wraps(fn)
10.     async def wrapper(*args: _P.args, **kwargs: _P.kwargs)
-> _R:
11.         return await asyncer.asyncify(fn)(*args, **kwargs)
12.
13.     return wrapper
```

装饰器使用代码

```
1. from fastmcp import FastMCP
2. import time
3.
4. mcp = FastMCP()
5.
6. @mcp.tool()
7. @make_async_background
8. def my_tool() -> None:
9.     time.sleep(5)
```

代码 3.5 里的装饰器定义代码，输入任意同步函数 fn，执行时通过语句 await asyncer.asyncify(fn)(*args, **kwargs)在后台线程调用原函数，输出异步版本的函数，签名相同但返回 Awaitable[_R]。如果装饰器的定义和使用代码不在同一文件中，则在使用时需要先导入，假设装饰器定义代码与使用代码存放在同一目录下，定义代码文件为 mcp_tools/async_utils.py，使用代码文件为 mcp_tools/system_tools.py，则在使用代码中需要添加语句“from .async_utils import make_async_background”，这一语句未在代码 3.5 中写出。如使用代码第 7 行所示，使用装饰器@make_async_background 时，**必须比@mcp.tool 装饰器更靠近工具函数**，这样才能保证工具函数先包装后注册，**顺序不可颠倒**。

鉴于装饰器方式需要额外安装 asyncer 库且写法复杂，对代码存储和导入均有要求，容易出错，建议使用代码 3.4 所示的基于 anyio 库的包装方式，对每个同步工具单独编写异步函数包装。

3.5.2 @tool 装饰器

使用@mcp.tool 装饰器[4]是最常见、最简单的注册工具组件的方法。

当工具组件被注册时，FastMCP 会自动执行以下操作：

- 将函数名称用作工具名称。
- 将函数的文档字符串用作工具描述。

[4] 也可被称为@tool 装饰器。称作@mcp.tool 是因为通常以 mcp 作为 FastMCP 实例的名称。例如若使用 server=FastMCP()语句实例化，则应当使用@server.tool 装饰器。

• 根据函数的参数和类型注解生成输入模式。

• 处理参数验证和错误报告。

需要注意的是，带有*args 或kwargs 的函数不能作为工具组件使用。**这一限制的存在是因为 FastMCP 需要为 MCP 协议生成完整的参数模式，而可变参数列表无法实现这一点。

虽然 FastMCP 会从函数中自动推断工具组件的名称和描述，但我们可以通过 @mcp.tool 装饰器的参数来覆盖这些默认值，并添加额外的元数据，@mcp.tool 的常见参数如表 3.7 所示。可能有部分参数未在此列出，可以查阅 FastMCP 官方文档中 *Tools—The @tool Decorator—Decorator Arguments* 部分（https://gofastmcp.com/v2/servers/tools#decorator-arguments）。

表 3.7　@mcp.tool 装饰器常见参数表

参数	描述	数据类型	默认值	备注
name	通过 MCP 服务器公开显示的工具名称	str | None	None	
title	工具的可读标题	str | None	None	
description	通过 MCP 服务器公开的工具描述	str | None	None	此参数覆盖函数注释（文档字符串），建议此参数缺省
tags	标签，用于对工具分类	set[str] | None	None	标签管理详见§3.4
enabled	启用或禁用工具	bool	True	工具启用/禁用详见§3.5.3
icons	工具的图标列表	list[Icon] | None	None	v2.13.0 版本新增 详见§5.3.2
meta	关于工具的可选元信息，会被传递给 MCP 客户端（**主要面向人类**）	dict[str, Any] | None	None	v2.11.0 版本新增
exclude_args	参数排除列表，列表内的工具参数不会被暴露	list[str] | None	None	v2.14.0 版本起弃用
output_schema	自定义输出模式	dict[str, Any] | None	None	v2.10.0 版本新增
annotations	可选的 ToolAnnotations 对象或字典，用于添加有关工具的额外注释（**主要面向程序**）	ToolAnnotations | dict | None	None	v2.2.7 版本新增

注：annotations 参数支持的属性未在表中列出，在下面的补充说明部分详细介绍。

参数 name、title、description、tags、icons、meta、annotations、output_schema 的内容会被包含在 MCP 服务器对 tools/list 请求的响应中。

在下面的章节中，将对一些参数进行额外的补充说明。

1. [name] 工具组件名称

name 参数可缺省，缺省情况下默认以函数名称作为 MCP 工具组件的名称。

2. [enabled] 启用/禁用工具

enabled 参数为默认值 True 时，该工具可被 MCP 服务器向客户端暴露，是

否可见取决于其它过滤设置（比如 FastMCP 构造函数参数 include_tags 和 exclude_tags 等）。enabled 参数为 False 时，该工具对客户端不可见，即使工具的标签在 FastMCP 构造函数参数 include_tags 中被设置为暴露，该工具仍然不可见。

FastMCP v2 中，工具组件的启用/禁用一共有 3 种方法，详见 § 2.5.3。

3. [meta] 可选的工具组件元信息

meta 参数的内容将通过工具对象的_meta 字段传递给 MCP 客户端，可用于自定义元数据、版本控制或其他应用程序特定的用途。

meta 的参数以键值对的形式给出，键值对的内容和数量没有限制，值可以是一个值，也可以嵌套地包含其它键值对，例如下面的两个例子，分别存在多个键值对和嵌套键值对：

```
@mcp.tool(name="test_tool",
          tags={"public"},
          meta={
              "version": "2.1.0",
              "author": "product-team@company.com",
              "rate_limit": "100req/min"
          }
)
```

```
@mcp.tool(
    exclude_args=["user_id"],
    meta={
        "excluded_parameters": {
            "user_id": "由认证中间件自动注入当前用户 ID"
        }
    }
)
```

当 MCP 服务器响应客户端的 tools/list 请求时，发送到客户端的响应中的“_meta”字段即包含 meta 参数的原始键值对和包含了标签的“_fastmcp”字段。使用上面第一个的@mcp.tool 装饰器注册的工具对象的_meta 字段如下：

```
"_meta": {
    "version": "2.1.0",
    "author": "product-team@company.com",
    "rate_limit": "100req/min",
    "_fastmcp": {
        "tags": [
            "public"
        ]
    }
}
```

MCP 客户端由宿主应用（Host）实例化，因此具体如何使用这些工具组件的元信息由宿主应用（例如大模型、智能体、AI 应用等）决定。

4. [exclude_args] 参数排除列表

exclude_args 参数数据类型为一个列表（list），该列表内的元素为 str 型，必须与工具函数的某个参数名称相同（尝试排除不存在的参数会导致警告），表示

该参数被从函数接口中排除，被排除的函数参数不会被包含在提供给 MCP 客户端的 JSON schema 中，客户端不会知道其存在。

FastMCP v2.14.0 已弃用该参数，建议使用 Depends()方法，详见 § 3.5.4 小节中的《参数排除》部分。exclude_args 参数仍可使用，但在启动服务器时会产生一个警告“DeprecationWarning: The `exclude_args` parameter is deprecated as of FastMCP 2.14. Use dependency injection with `Depends()` instead for better lifecycle management.”。

需要注意的是，被排除的函数参数必须提供默认值，以便在参数缺省的情况下正常调用工具，否则会引发 ValueError，例如以下代码：

```
@mcp.tool(exclude_args=["auth_token"])
def query_protected_api(
    query: str,
    auth_token: str = None   # 排除认证令牌（正确写法）
    # auth_token: str        # 错误写法
) -> dict:
```

常见的应用场景包括：

- **隐藏上下文状态参数**：在需要访问请求上下文或系统状态的工具中，state 参数通常由服务器自动注入，无需 LLM 干预。
- **排除认证与授权信息**：对于需要认证令牌或权限信息的工具，可以安全地隐藏这些敏感参数，例如上面的样例中的令牌。
- **内部配置参数**：工具可能需要一些内部配置参数，但这些参数对 LLM 之类的宿主应用来说没有意义，例如请求超时时间等。

exclude_args 可以与工具转换（Tool Transformation）特性结合使用，实现更复杂的参数处理逻辑。例如在工具组件中将会话 id 或用户 id 参数排除，这些参数使用转换函数直接从上下文管理器中获取并填入，保证了 id 的准确性和安全性。工具转换特性详见章节 § 3.5.11。

5. [output_schema] 自定义输出模式

对于 int、str 等基本返回类型，FastMCP 会自动将结果包装在“result”键下。开发者也可通过在@mcp.tool 装饰器中设置 output_schema 参数来完全自定义模式，但自定义模式必须满足 JSON schema 格式要求和对象类型约束（即“type”键对应的值必须是“object”），且工具返回值必须与之匹配。下面提供一个使用基本返回类型和自定义输出模式的案例。

代码 3.6 是使用了基本返回类型 int 的工具函数代码和自定义输出模式的@mcp.tool 装饰器代码，以及 MCP 客户端发起 tools/call 请求（此案例中调用工具的参数 a=1，b=2）后服务器对应的响应。

代码 3.6　工具组件自定义输出模式示例

(a)使用了基本返回

```
1. @mcp.tool()
2. async def add(a:int,b:int) -> int:
```

<table>
<tr><td>类型 int 的工具函数代码</td><td>

```
    """
    将两个整数相加
    Args:
        a:第一个加数
        b:第二个加数
    Return:
        两个数 a 和 b 的和
    """
    return a+b
```

</td></tr>
<tr><td>(b)使用基本返回类型的响应</td><td>

```
{
    "content": [
        {
            "type": "text",
            "text": "3"
        }
    ],
    "structuredContent": {
        "result": 3
    },
    "isError": false
}
```

</td></tr>
<tr><td>(c)使用了自定义输出模式的@mcp.tool装饰器的代码</td><td>

```
@mcp.tool(output_schema={
    "type": "object",
    "properties": {
        "args_1": {
            "type": "integer",
            "description": "The argument a"
        },
        "args_2": {
            "type": "integer",
            "description": "The argument b"
        },
        "ret": {
            "type": "integer",
            "description": "The sum of a and b",
            "minimum": -2147483648,
            "maximum": 2147483647
        }
    },
    "required": ["args_1","args_2","ret"]
})
async def add(a:int,b:int) -> dict:
    """
    将两个整数相加
    Args:
        a:第一个加数
        b:第二个加数
    Return:
        两个数 a 和 b 的和
    """
    return {
        "args_1": a,
        "args_2": b,
        "ret": a+b
    }
```

</td></tr>
<tr><td>(d)使用自</td><td>

```
{
```

</td></tr>
</table>

定义输出模式的响应	"content": [
	{
	"type": "text",
	"text": "{\"args_1\":1,\"args_2\":2,\"ret\":3}"
	}
	],
	"structuredContent": {
	"args_1": 1,
	"args_2": 2,
	"ret": 3
	},
	"isError": false
	}

如代码 3.6(c)的第 21 行，使用 JSON schema 自定义输出模式的情况下，工具函数返回值类型为 dict。**需要注意的是，return 中的返回值必须符合@mcp.tool 装饰器中定义的 JSON schema 格式**，否则会在调用工具时向 MCP 客户端返回文本形式的报错信息。

例如，如果将代码 3.6(c)的第 33 行改为”ret”: “test”，则响应中报错"Output validation error: 'test' is not of type 'integer'"，此时的响应如下：

```
{
  "content": [
    {
      "type": "text",
      "text": "Output validation error: 'test' is not of type 'integer'"
    }
  ],
  "isError": true
}
```

可以看到，响应中的“isError”键值变为了 true，工具未正常工作。出现这种报错时，MCP 服务器端不会有报错记录，只能在客户端调用才能发现。

6. [annotations] 工具组件注释

meta 参数是一种对工具组件的注释，但是以字符串形式描述且格式自由，主要目的是使人类阅读并理解该工具的用途、版本和其它注意事项，程序要想理解就必须依赖大模型，可能对关键信息的理解产生偏差和浪费算力。annotations 参数是面向程序的注释，仅有几个可选的注释项，包括注明指示工具是否会修改数据或系统、安全性等。这些注释项可被视为 annotations 参数的参数，见表 3.8。

表 3.8 @mcp.tool 装饰器 annotations 参数的参数表

参数	描述	数据类型	默认值	备注
title	工具的可读标题	str	None	
readOnlyHint	指示工具是否为仅读取而不做修改	bool	False	
destructiveHint	对于非只读工具，提示其更改是否具有破坏性	bool	True	
idempotentHint	指示重复的相同调用是否与单次调用效果	bool	False	

	相同			
openWorldHint	指定工具是否与外部系统交互	bool	True	

annotations 参数里的 title 参数和@mcp.tool 装饰器里的 title 参数用途相同，但是在 MCP 服务器对 tools/list 请求的响应中前者位于“annotations”字段，后者位于“title”键值的位置，两者可以内容不同。

annotations 参数需要以 dict 键值对的形式给出，例如：

```
@mcp.tool(name="my_tool",
            annotations={
                "title": "我的工具",
                "readOnlyHint": True,
                "destructiveHint": False,
                "idempotentHint": True,
                "openWorldHint": False
            }
)
```

需要注意的是，annotations 参数的参数需要以 str 形式作为键，即必须使用双引号将参数名包裹，否则会报错。

annotations 参数主要用于敏感操作注释，但需要宿主应用（比如大模型、智能体等）和 MCP 客户端根据该参数对调用工具增加安全性限制，比如调用破坏性工具前取得用户确认等。注释有助于提升用户体验，但应视为辅助性提示，它们能帮助客户端应用程序呈现合适的界面元素和安全控件，但本身并不会强制执行安全边界。此外，始终要确保注释能够准确反映工具的实际功能。

这些注释向客户端应用程序传达资源的行为方式，而不会在 LLM 的提示中消耗 token 上下文。

关于 MCP 工具组件的注释，详情可以查阅 FastMCP 官方文档中的 *Tools—MCP Annotations* 部分（https://gofastmcp.com/v2/servers/tools#mcp-annotations）。

3.5.3 工具组件的启用/禁用

在 FastMCP v2 中，工具组件的启用/禁用一共有 3 种方法实现，分别是设置 FastMCP 构造函数的参数 include_tags 和 exclude_tags、设置@mcp.tool 装饰器的参数 enabled、使用代码动态设置。

使用代码动态启用/禁用工具的方法如代码 3.7 所示。

代码 3.7　使用代码动态启用/禁用工具组件

```
1. @mcp.tool
2. def dynamic_tool():
3.     return "I am a dynamic tool."
4.
5. # Disable and re-enable the tool
6. dynamic_tool.disable()
7. dynamic_tool.enable()
```

具体来说，exclude_tags 优先级高于 include_tags，而整体上讲，include_tags

和 exclude_tags 优先级同级，都属于 FastMCP 构造函数的参数。当这三种设置方式的设置冲突时，优先级如表 3.9 所示。

表 3.9　启用/禁用工具组件方法优先级

FastMCP 构造函数参数设置	@mcp.tool 装饰器 enabled 参数设置	disable()和 enable()方法动态设置	工具启用/禁用
启用	True	enable()	**启用**
启用	True	disable()	**禁用**
启用	False	enable()	**启用**
启用	False	disable()	**禁用**
禁用	True	enable()	**禁用**
禁用	True	disable()	**禁用**
禁用	False	enable()	**禁用**
禁用	False	disable()	**禁用**

如何通过设置 FastMCP 构造函数参数 include_tags 和 exclude_tags 实现启用/禁用带有某标签的工具组件详见 § 3.4.2 小节。

除此之外，还可以通过设计中间件的方式，实现更加精细的工具启用/禁用，例如针对特定的用户禁止调用特定的工具等，详见 § 5.2 小节。

2.5.4 工具组件函数的参数

默认情况下，FastMCP 通过检查 Python 函数的签名和类型注解，将其转换为 MCP 工具。这使我们能够通过为标准 Python 类型注解的方式来定义工具组件。总体而言，该框架致力于实现“开箱即用”：像参数默认值和类型注解这类符合 Python 习惯的写法会自动转换为 MCP 模式。不过，我们也可以通过多种方式自定义工具的行为。

1. 参数类型注解和参数缺省

MCP 工具组件的参数使用标准的 Python 类型注解方式，例如：

```
1. @mcp.tool
2. def analyze_text(
3.     text: str,
4.     max_tokens: int = 100,
5.     language: str | None = None
6. ) -> dict:
7.     """分析提供的文本。"""
8.     # 实现代码...
```

此外，FastMCP 支持广泛的类型注解，包括所有 Pydantic 支持的类型，如表 3.10 所示。根据官方文档，FastMCP 支持所有 Pydantic 能够作为字段使用的类型，包括所有 Pydantic 自定义类型。

表 3.10　MCP 工具组件参数类型表

类型注解	实例	说明
基本类型	int, float, str, bool	简单标量值

二进制数据	bytes	二进制内容（原始字符串，不会自动 base64 解码）
日期与时间	datetime, date, timedelta	日期时间对象（ISO 格式字符串）
集合类型	list[str], dict[str, int], set[int]	项目集合
可选类型	float | None, Optional[float]	可为空/省略的参数
联合类型	str | int, Union[str, int]	接受多种类型的参数
约束类型	Literal["A", "B"], Enum	仅允许特定取值的参数
路径类型	Path	文件系统路径（自动从字符串转换）
UUID 类型	UUID	通用唯一标识符（自动从字符串转换）
Pydantic 模型	UserData	带验证的复杂结构化数据

需注意以下 FastMCP 特有的行为：

• **二进制数据**：bytes 参数接受原始字符串，不会自动进行 base64 解码。如需处理 base64 数据，请使用 str 类型并通过 base64.b64decode()手动解码。

•**枚举类型**：客户端发送的是枚举值（如"red"），而非枚举名称（如"RED"）。函数接收到的将是枚举成员（如 Color.RED）。

• **路径类型与 UUID 类型**：字符串输入会自动转换为 Path 和 UUID 对象

• **Pydantic 模型**：必须以 JSON 对象（字典 Dict）形式提供，而非字符串化的 JSON。即使采用灵活验证，{"user": {"name": "Alice"}}是有效的，而{"user": '{"name": "Alice"}'}是无效的。

FastMCP 遵循 Python 标准函数参数约定：未设置默认值的参数为必需参数，而设置了默认值的参数为可选参数，可选参数缺省时取默认值。

关于参数类型的验证和自动转换，详见§ 3.3.10 小节的严格输入验证模式。

2. 参数元数据

有些时候，可能需要为各个参数添加更加详细的注释，以便于理解参数的取值和用途。我们可以通过多种方式提供参数的额外元数据：

(1) **简单字符串描述**：对于基本的参数描述，可以使用 Annotated 库的便捷简写形式，比如：

```
1. from typing import Annotated
2.
3. @mcp.tool
4. def process_image(
5.     image_url: Annotated[str, "要处理的图片 URL"],
6.     resize: Annotated[bool, "是否调整图片尺寸"] = False,
7.     width: Annotated[int, "目标宽度（像素）"] = 800,
8.     format: Annotated[str, "输出图片格式"] = "jpeg"
9. ) -> dict:
10.     """对图片进行处理，可选调整尺寸。"""
11.     # 实现代码...
```

这种简写语法等同于使用 Field(description=...)，但对于简单描述的应用场景更为简洁。此简写语法仅适用于带有单个字符串描述的 Annotated 类型。

(2) **使用 Field 的高级元数据**：对于验证约束和高级元数据，可在 Annotated 中使用 Pydantic 的 Field 类，比如：

```
1. from typing import Annotated, Literal
2. from pydantic import Field
3.
4. @mcp.tool
5. def process_image(
6.     image_url: Annotated[str, Field(description="要处理的图片 URL")],
7.     resize: Annotated[bool, Field(description="是否调整图片尺寸")] =
False,
8.     width: Annotated[int, Field(description="目标宽度（像素）", ge=1,
le=2000)] = 800,
9.     format: Annotated[
10.         Literal["jpeg", "png", "webp"],
11.         Field(description="输出图片格式")
12.     ] = "jpeg"
13. ) -> dict:
14.     """对图片进行处理，可选调整尺寸。"""
15.     # 实现代码...
```

Field 提供多种验证和文档功能：

- description：参数的人类可读说明（展示给大语言模型）
- ge/gt/le/lt：大于/小于（或等于）约束
- min_length/max_length：字符串或集合的长度约束
- pattern：用于字符串验证的正则表达式模式
- default：参数省略时的默认值

关于参数元数据的原始英文版信息，可以查阅 FastMCP 官方文档中的 *Tools—Arguments—Parameter Metadata* 部分（https://gofastmcp.com/v2/servers/tools#parameter-metadata）。

3. 隐藏参数

除了 § 3.5.2 小节中@mcp.tool 装饰器的 exclude_args 参数排除列表可用于向大模型等宿主应用隐藏指定的参数，还可以使用 Depends()进行依赖注入。使用 Depends()的参数将自动从工具模式中排除，代码如下：

```
1. from fastmcp import FastMCP
2. from fastmcp.dependencies import Depends
3.
4. mcp = FastMCP()
5.
6. def get_user_id() -> str:
7.     return "user_123"  # 在运行时注入
8.
9. @mcp.tool
10. def get_user_details(user_id: str = Depends(get_user_id)) -> str:
11.     # user_id 由服务器注入，不由大语言模型提供
```

```
    return f"用户 {user_id} 的详细信息"
```

通过注入方式向工具组件提供参数进阶功能中§5.1小节 MCP 上下文部分。

3.5.5 工具组件的描述注释/文档字符串

普通 Python 代码中的函数，如果不为它编写注释并不会影响其正常运行。而 MCP 工具函数则与之不同，为函数编写指定格式的描述注释是正常调用的前提条件，宿主应用（如大模型、智能体等）依赖于注释了解工具的用途和调用方法，注释的准确性直接关系到智能体能否在合适的时候正确地调用工具。

工具的注释应当位于函数定义语句（即 def 语句）之后、实现代码之前，写在一对连续三个双引号之间。这种注释又被称为“文档字符串（docstring）”。按照规范的写法，注释应当包括工具的描述、参数的名称及意义、返回值的类型和意义。大模型可以通过函数定义时的参数表获取参数和返回值的类型，因此对于比较简单的工具，参数与返回值的类型可以省略。

如果@mcp.tool 装饰器给定了 description 参数，则会覆盖文档字符串。而文档字符串更加直观和易于编写，建议用这种方式注释，缺省 description 参数。

因为我们主要的应用场景是面向使用中文的用户、中国的 Qwen 系列大模型，所以建议使用中文编写注释。若有条件可以使用大模型测试调用情况，根据测试结果调整优化注释的表述方法。

注释有多种标准格式，理论上采用任意一种即可使大模型识别，但建议一个团队采用统一的格式。下面是一个简单的加法工具函数，注释的格式可以参考：

```
@mcp.tool()
async def add(a:int,b:int) -> int:
    """
    将两个整数相加
    Args:
        a:第一个加数
        b:第二个加数
    Return:
        两个数 a 和 b 的和
    """
    return a+b
```

如果使用针对 FastMCP 编程做过优化的 IDE，例如 PyCharm 等，在注释中漏写参数描述时会有提示“docstring 中的缺失形参”。但是如果该参数为需要隐藏的参数，也会有此提示，可以忽略。

3.5.6 工具组件的函数编写

1. 函数实现的异步写法

在§3.5.1小节工具组件的异步支持相关部分，已经建议了在新编写的 FastMCP 服务器代码中尽可能使用异步形式编写工具函数，即使是计算量较小的纯 CPU 运算工具。因此在工具的函数实现的编写过程中需要注意采用异步写法。

如果不慎忘记了为通信和数据传输之类的代码添加 await 关键字，IDE 也会提示“未等待协程”，此时在语句之前添加 await 关键字即可。

2. 服务器资源调用

由于工具组件的代码是运行在 MCP 服务器上的，使用 os 库等文件管理或系统级方法，或者读取/修改环境变量等操作，均是在服务器上执行。

工具组件中如果需要读取服务器上确定的某文件，既可以将这些文件的路径作为“资源”在服务器预先注册，也可以在工具被调用时临时传入一个路径字符串。两种读取文件资源的方法如代码 3.8 所示，其中 test_file.txt 文件位于 Python 代码文件相同目录下，内容有且仅有一行，内容为数字 3。

代码 3.8　使用临时路径字符串和注册文件资源方法读取文件示例

(a)使用临时路径字符串代码

```
1. from fastmcp import FastMCP
2.
3. mcp=FastMCP("Demo_readFile", host="0.0.0.0", port=8000)
4.
5. @mcp.tool()
6. async def add_with_file(a:int,b:int) -> int:
7.     """
8.     将两个整数相加后加上服务器指定的值
9.     Args:
10.         a:第一个加数
11.         b:第二个加数
12.     Return:
13.         a、b 和服务器指定值的和
14.     """
15.     file_path="./test_file.txt"
16.     with open(file_path,'r') as file:
17.         c=int(file.read().strip())
18.     return a+b+c
19.
20. if __name__=="__main__":
21.     mcp.run(transport="streamable-http")
```

(b)注册文件资源方法代码

```
1. from fastmcp import FastMCP,Context
2. from pathlib import Path
3. from fastmcp.resources import FileResource
4.
5. mcp=FastMCP("Demo_readFile", host="0.0.0.0", port=8000)
6.
7. file_path = Path("./test_file.txt").resolve()
8. file_resource = FileResource(
9.     uri=f"file://{file_path.as_posix()}",  # 使用 as_posix()
确保路径格式正确
10.     path=file_path,
11.     name="My fext file",
12.     mime_type="text/plain",
13. )
14. mcp.add_resource(file_resource)  # 将资源添加到 MCP 实例
15.
16. @mcp.tool(exclude_args=["doc_uri"])
17. async def add_with_file(a : int,b : int,ctx : Context,
doc_uri: str = file_resource.uri) -> int:
```

```
18.     """
19.     将两个整数相加后加上服务器指定的值
20.     Args:
21.         a:第一个加数
22.         b:第二个加数
23.     Return:
24.         a、b 和服务器指定值的和
25.     """
26.     doc_resource = await ctx.read_resource(uri=doc_uri)
27.     doc_content = doc_resource[0].content
28.     c = int(doc_content)
29.     return a+b+c
30.
31. if __name__=="__main__":
32.     mcp.run(transport="streamable-http")
```

当文件内容简单的时候，显然直接用路径字符串的写法更加简单。但是有互联网上的文章（https://blog.gitcode.com/e5415a06f48707368cdc5af411fb476a.html）称在使用某些工具的时候使用临时路径字符串的写法会引发报错。因此当遇到报错的时候可以改用注册文件资源的方法。通过将文件注册为资源，可以确保接口一致性，并能够集中管理所有资源的元数据和访问控制。

代码 3.8(b)中的第 7 行使用 Path().resolve()方法将可能来自不同操作系统的路径标准化为统一格式，第 9 行使用 as_posix()方法确保路径格式正确。路径字符串经过这两种方法的处理后提高了跨平台兼容性。

代码 3.8(b)仅作注册资源方法的示例，有关资源组件详见§3.6 章节。

3. 客户端资源调用

如果编写一个使用常规的文件管理、读写工具，运行在**远程**的 MCP 服务器上，那么宿主应用（比如智能体）控制客户端调用这个工具是无法实现客户端设备的文件管理的。工具中使用的文件路径（例如 ./docs/file.txt），是相对于 MCP 服务器进程的工作目录，而不是客户端。

使 MCP 服务器能调用客户端资源、能跨服务器调用资源，都需要将资源注册后使用，MCP 资源组件的注册与使用详见§3.6 章节。

3.5.7 工具组件的返回值

FastMCP 工具组件能以两种互补的格式返回数据：传统内容块（如文本和图像）和结构化输出（机器可读的 JSON）。添加返回类型标注时，FastMCP 会自动生成输出模式以验证结构化数据，并支持客户端将结果反序列化回 Python 对象。

通常情况下，我们使用默认的工具序列化器，将工具的返回值序列化为 JSON 形式的字符串。但还可以通过在创建服务器（即 FastMCP 实例化）时提供 tool_serializer 函数来自定义此行为，详见§3.5.13。

1. 内容块（Content Blocks）

FastMCP 会自动将工具函数的返回值转换为适当的 MCP 内容块：

- **str**：作为 TextContent 发送
- **bytes**：进行 Base64 编码后作为 BlobResourceContents 发送（位于 EmbeddedResource 内）
- **fastmcp.utilities.types.Image**：作为 ImageContent 发送
- **fastmcp.utilities.types.Audio**：作为 AudioContent 发送
- **fastmcp.utilities.types.File**：作为 Base64 编码的 EmbeddedResource 发送
- **MCP SDK 内容块**：按原样发送
- **以上任意类型的列表**：根据上述规则转换每个项目
- **None**：产生空响应

上述常见且简单的类型的返回值可以直接像普通的 Python 代码那样写在 return 语句中，运行代码时会自动转换为 MCP 内容块形式。

2. 媒体辅助类（Media Helper Classes）

上述的三个类型 fastmcp.utilities.types.Image、fastmcp.utilities.types.Audio 和 fastmcp.utilities.types.File 属于媒体形式，是用于返回图像、音频和文件的辅助类，因此它们被称为“媒体辅助类”。

正如上文中所说，当直接或作为列表的一部分返回其中一个类的实例时，FastMCP 会自动将其转换为相应的 MCP 内容块。例如，如果在 return 语句返回一个 fastmcp.utilities.types.Image 对象，FastMCP 会将其转换为具有正确 MIME 类型和 Base64 编码的 MCP ImageContent 块。

需要注意的是，辅助类仅在直接返回或作为列表（list）的一部分返回时，才会被自动转换为 MCP 内容块。对于字典（dict）这类更复杂的容器，我们需要在代码中手动将其转换为 MCP 类型，例如：

```
# ✅ 自动转换
return Image(path="chart.png")
return [Image(path="chart1.png"), "文本内容"]

# ❌ 不会被自动转换
return {"image": Image(path="chart.png")}

# ✅ 嵌套使用时手动转换
return {"image": Image(path="chart.png").to_image_content()}
```

手动转换需使用方法 Image().to_image_content()、Audio().to_audio_content()、File.to_resource_content()进行转换。此外，如果要想使用 Image、Audio、File 这些简略写法而不是 fastmcp.utilities.types.Image、fastmcp.utilities.types.Audio 和 fastmcp.utilities.types.File 这些完整写法，则需要导入 fastmcp.utilities.types 库：

```
from fastmcp.utilities.types import Image, Audio, File
```

每个辅助类的构造函数具有相同的参数表，这些参数及其含义和提供方式如表 3.11 所示。

表 3.11　媒体辅助类构造函数参数表

参数	描述	说明
path	文件路径（字符串或 Path 对象）——MIME 类型根据文件扩展名自动检测	**这两个参数必须选择其一提供，不能同时提供**
data	原始字节数据——必须提供 format 参数以指定 MIME 类型	
format	可选的格式覆盖参数（例如“png”、“wav”、“pdf”）	提供 data 参数时必须提供 提供 path 参数时可以提供
name	当使用 data 参数时，为 File 对象指定的可选名称	提供 data 参数创建 File 对象时建议提供
annotations	可选的 MCP 注释，附加额外的元数据信息	

注：需要注意不同情况下参数的必要性不同。

媒体辅助类的构造函数必须二选一地提供 path 参数或 data 参数。

提供 path 参数时，使用媒体文件的路径创建辅助类，此时自动根据文件扩展名设置 MIME 类型，可以提供 format 参数覆盖自动识别的 MIME 类型，可以提供 name 参数为辅助类指定名称（通常辅助类为 File 类时有意义）。

提供 data 参数时，使用原始字节数据创建辅助类，此时无法自动推测 MIME 类型，必须提供 format 参数设置 MIME 类型。如果辅助类是 File 类，强烈建议提供 name 参数为文件命名，以便于文件的下载、保存和识别，若 name 参数缺省则可能被保存为一个无意义的名称。

annotations 是可选的 MCP 注释，可为辅助类提供额外元数据信息，无论提供 path 还是 data 参数，annotations 参数均可提供。该参数需要提供一个 Annotations 类，必须先从 mcp.types 库导入才可使用，不能以字典 dict 的形式提供。注释的形式类似于键值对，以下为使用参数 annotations 的示例代码，为创建的 Image 类添加注释，注释内的属性 aaa 值为“123”，属性 xyz 值为“test”：

```
from mcp.types import Annotations
Image(annotations=Annotations(aaa="123", xyz="test"))
```

3. 结构化返回值

结构化的返回内容是一个 JSON 对象，它会与传统内容一同发送。当工具返回数据具有 JSON 对象表示形式时，FastMCP 会自动在生成传统内容的同时创建结构化输出。这提供了机器可读的 JSON 数据，客户端可以将其反序列化为 Python 对象。FastMCP 自动的结构化遵守以下规则：

- **对象类结果（字典 dict、Pydantic 模型、数据类）**：始终转换为结构化内容（即使没有提供自定义输出模式）
- **非对象类结果（整数 int、字符串 str、列表 list）**：仅在有输出模式对其进行验证/序列化时才会转换为结构化内容
- **所有结果**：为保持向后兼容性，始终生成传统内容块

这种自动化行为使客户端能够在获取人类可读内容的同时接收机器可读数

据，而无需为对象类返回显式定义输出模式。相关的具体描述如下，其中 MCP 服务器返回信息由 MCP Inspector 获取，转义字符已自动还原并去除换行符，实际从 MCP 服务器发出的信息应当以转义字符表示引号、换行符等。

(1) 字典（Dictionaries）与对象（Objects）

当工具返回字典、数据类或 Pydantic 模型时，FastMCP 会自动从中创建结构化内容。该结构化内容包含实际对象数据，便于客户端将其反序列化为原生对象。

以下为对象类结果自动转换的样例，工具函数以字典形式返回，FastMCP 自动创建了结构化的返回内容：

工具函数返回值部分的代码：

```
@mcp.tool
def get_user_data(user_id: str) -> dict:
    """Get user data."""
    return {"name": "Alice", "age": 30, "active": True}
```

MCP 服务器实际的返回信息：

```
{
  "content": [
    {
      "type": "text",
      "text": "{"name":"Alice","age":30,"active":true}"
    }
  ],
  "structuredContent": {
    "name": "Alice",
    "age": 30,
    "active": true
  }
}
```

通过上述样例，我们可以看到 MCP 服务器返回的信息除了自动化生成的结构化内容（“structuredContent”字段）以外，还包括原始的返回值（“content”字段），客户端可根据需要选用合适格式的信息。

(2) 基本类型（Primitives）与集合（Collections）

当工具返回基本类型（整型 int、字符串 str、布尔值 bool）或集合（列表 list、集合 set）时，需要在工具函数定义的末尾添加返回类型注解才能使 FastMCP 生成结构化内容。该注解会指示 FastMCP 如何验证并序列化返回结果。

以下分别为未添加返回类型注解和添加了返回类型注解的工具函数代码，以及对应的 MCP 服务器返回信息：

	未添加返回类型注解	添加返回类型注解
函数代码	@mcp.tool def calculate_sum(a: int, b: int): """Calculate sum without return annotation.""" return a + b # Returns 8	@mcp.tool def calculate_sum(a: int, b: int) -> int: """Calculate sum with return annotation.""" return a + b # Returns 8
MCP 服务	{ "content": [	{ "content": [

器返回信息	{
	"type": "text",
	"text": "8"
	}
	]
	}
		{
		"type": "text",
		"text": "8"
		}
		],
		"structuredContent": {
		"result": 8
		}
		}

如上述样例所示，当添加返回注解（例如 -> int）时，由于 JSON schema 要求结构化输出的根节点必须是对象类型，FastMCP 会通过将基本值包装在 {"result": ...}对象中来生成结构化内容。

(3) 类型化模型（Typed Models）

上述的返回类型注解适用于任何可转换为 JSON schema 的类型。数据类（Dataclass）和 Pydantic 模型尤其有用，因为 FastMCP 会提取其字段定义以自动创建详细模式。

代码 3.9 是一个自定义了一个数据类，且工具函数返回该自定义数据类格式的返回值的代码样例。代码 3.9 后面是 FastMCP 根据该代码自动生成的 JSON schema 以及 MCP 实际返回信息。

代码 3.9　返回自定义数据类的工具组件代码

```python
from dataclasses import dataclass
from fastmcp import FastMCP

mcp = FastMCP()

@dataclass
class Person:
    name: str
    age: int
    email: str

@mcp.tool
def get_user_profile(user_id: str) -> Person:
    """Get a user's profile information."""
    return Person(
        name="Alice",
        age=30,
        email="alice@example.com"
    )
```

自动生成的 JSON schema：

```
{
  "properties": {
    "name": {"title": "Name", "type": "string"},
    "age": {"title": "Age", "type": "integer"},
    "email": {"title": "Email", "type": "string"}
  },
  "required": ["name", "age", "email"],
  "title": "Person",
```

```
  "type": "object"
}
```

MCP 服务器实际的返回信息：

```
{
  "content": [
    {
      "type": "text",
      "text": "{name="Alice", age=30, email="alice@example.com"}"
    }
  ],
  "structuredContent": {
    "name": "Alice",
    "age": 30,
    "email": "alice@example.com"
  }
}
```

代码 3.9 中，Person 数据类将转换为输出模式（JSON schema），用于描述预期的格式。执行时，客户端将在 MCP 服务器实际的返回信息中同时收到“content”和“structuredContent”字段，可根据需要选择合适的字段。

4. 输出模式

2025 年 6 月 18 日发布的 MCP 规范更新引入了输出模式，这是一种描述工具预期输出格式的新方式。当提供了输出模式时，工具的函数必须返回符合该模式的结构化输出，否则会引发错误。

当你为函数添加返回类型注解时，FastMCP 会自动生成描述预期输出格式的 JSON 模式，这部分已经在上一小节《结构化返回值》中介绍。这些模式帮助 MCP 客户端理解和验证所接收到的结构化数据。

对于原始返回类型（如 int、str、bool），FastMCP 会自动将结果包装在“result”键下，以生成有效的结构化输出，相关内容与案例已在 § 3.5.7.3.2《结构化返回值》中的“基本类型与集合”部分说明。

对于手动的自定义输出模式，可以通过在@mcp.tool 装饰器的参数表中提供自定义的 output_schema 来覆盖自动生成的模式，但需要注意自定义的 output_schema 必须符合 JSON schema 的格式要求，并与函数返回值格式匹配。详见 § 3.5.2 小节的《@mcp.tool 装饰器》中“自定义输出模式”的部分。

5. ToolResult 对象

要完全控制工具响应，可以返回一个 ToolResult 对象。这使我们可以明确控制工具输出的所有字段：传统内容、结构化数据和元数据。返回 ToolResult 时，我们拥有完全控制权——**FastMCP 不会自动包装或转换您的数据**。无论是否定义了输出模式，都可以返回 ToolResult。需要注意的是，返回 ToolResult 对象的工具函数在函数定义要么添加类型注解 ToolResult（即 -> ToolResult），要么不添加类型注解，否则会报错。

要想使用 ToolResult，需要先从 fastmcp.tools.tool 库导入该类，代码如下：

```
from fastmcp.tools.tool import ToolResult
```

ToolResult 对象的实例化构造函数有三个参数 content、structured_content 和 meta，分别对应 MCP 服务器返回信息的三个字段。三个参数中必须至少提供 content 或 structured_content 中的一个，具体介绍如下：

• **content 参数**：对应传统的 MCP 内容块，客户端会将其展示给用户。可以是一个字符串（自动转换为 TextContent）、一个 MCP 内容块列表，或任何可序列化的值（转换为 JSON 字符串）。与传统 str 型返回值对比的示例代码如下：

```
# Simple string
ToolResult(content="Hello, world!")

# List of content blocks
ToolResult(content=[
    TextContent(type="text", text="Result: 42"),
    ImageContent(type="image", data="base64...", mimeType="image/png")
])
```

• **structured_content 参数**：一个包含结构化数据的字典，需与您工具的输出模式匹配。这使得客户端能够以编程方式处理结果。如果提供 structured_content 参数，它必须是一个字典（dict）或 None。如果只提供了 structured_content 参数，它也会被用作 content 字段（转换为 JSON 字符串）。示例代码如下：

```
ToolResult(
    content="Found 3 users",
    structured_content={"users": [{"name": "Alice"}, {"name": "Bob"}]}
)
```

• **meta 参数**：用于控制元数据字段，此字段用于性能指标、调试信息，或任何不属于内容或结构化输出、但需传递给 MCP 客户端的特定数据，例如：

```
ToolResult(
    content="Analysis complete",
    structured  content={"result": "positive"},
    meta={
        "execution_time_ms": 145,
        "model_version": "2.1",
        "confidence": 0.95
    }
)
```

需要注意的是，ToolResult 中的 meta 字段用于工具执行的运行时元数据（例如，执行时间、性能指标），这与@mcp.tool(meta={...})中的 meta 参数是有区别的，后者提供的是关于工具定义本身的静态元数据。

3.5.8 工具组件的错误处理

当工具组件在运行过程中遇到错误时，可以抛出标准的 Python 异常（例如 ValueError、TypeError、FileNotFoundError 以及自定义的异常等），也可以抛出 FastMCP 类的 ToolError 异常。

默认情况下，所有的异常（包括异常详细信息）都会被记录，并转换为 MCP

错误响应返回给客户端的大语言模型（LLM）或智能体等宿主应用。这有助于宿主 AI 应用理解失败原因并作出适当的应对。

若出于安全考虑需要隐藏内部错误细节，可以通过以下方式实现：

方法一：在创建 FastMCP 实例时设置 mask_error_details=True 参数：

```
mcp = FastMCP(name="SecureServer", mask_error_details=True)
```

方法二：通过 ToolError 显式控制发送给客户端的错误信息。要使用 ToolError，则需要先从 fastmcp.exceptions 库中导入 TooleError 类。代码 3.10 分别使用了 ToolError 和标准的 TypeError 抛出异常。

代码 3.10　使用 ToolError 和标准的 TypeError 抛出异常的工具组件代码

```
1. from fastmcp import FastMCP
2. from fastmcp.exceptions import ToolError
3.
4. @mcp.tool
5. def divide(a: float | str, b: float) -> float:
6.     """Divide a by b."""
7.
8.     if b == 0:
9.         # Error messages from ToolError are always sent to clients,
10.        # regardless of mask_error_details setting
11.        raise ToolError("Division by zero is not allowed.")
12.
13.    # If mask_error_details=True, this message would be masked
14.    if not isinstance(a, (int, float)) or not isinstance(b, (int,
float)):
15.        raise TypeError("Both arguments must be numbers.")
16.
17.    return a / b
```

当 FastMCP 构造函数的参数 mask_error_details=True 时，只有通过 ToolError 抛出的错误信息会包含具体详情，其他异常均会被转换为通用错误消息。代码 3.10 的工具组件在 mask_error_details 参数不同设置下，以对应报错触发条件调用工具后 MCP 服务器返回的信息如表 3.12 所示。

表 3.12　代码 3.10 在 mask_error_details 参数不同设置下 MCP 服务器返回信息对比表

工具参数		mask_error_details=False	mask_error_details=True
a	b		
“1”	1	{ "content": [{ "type": "text", "text": "Error calling tool 'divide': Both arguments must be numbers." }], "isError": true }	{ "content": [{ "type": "text", "text": "Error calling tool 'divide'" }], "isError": true }
1	0	{ "content": [{	{ "content": [{

		"type": "text", "text": "Division by zero is not allowed." }], "isError": true }	"type": "text", "text": "Division by zero is not allowed." }], "isError": true }

从表 3.12 可以看出，抛出标准 Python 错误时，若不隐藏错误细节，则报错信息为“Error calling tool ‘<工具名称>’: <程序中给定的错误信息>”，若开启隐藏错误细节，则仅报错“Error calling tool ‘<工具名称>’”。而抛出 ToolError 时，无论是否隐藏细节，都直接以程序给出的错误信息作为 MCP 服务器向客户端发送的信息。

3.5.9 工具组件变更通知

当工具组件被添加、移除、启用或禁用时，FastMCP 服务器会自动向连接的客户端发送 notifications/tools/list_changed 通知。这使得客户端能够及时了解当前的工具集，无需手动轮询变更。这一功能在使用动态启用/禁用工具组件时非常重要。假设发生变更相关的工具为 example_tool，则 mcp.add_tool(example_tool)、example_tool.disable()、example_tool.enable 和 mcp.remove_tool(“example_tool”)方法会导致 MCP 服务器自动向客户端发送 tools/list_changed 通知。

仅当这些操作发生在活动的 MCP 请求上下文内时（例如，从工具或其他 MCP 操作中调用时），才会发送通知。在服务器初始化期间执行的操作不会触发通知。

客户端可以使用消息处理程序（Message Handler）处理这些通知，以自动刷新其工具列表或更新界面。

3.5.10 从工具组件中访问 MCP 上下文（Context）

工具可通过上下文对象（Context）访问 MCP 功能，如日志记录、资源读取或进度报告。使用 Context 类需要从 fastmcp 库导入 Context 类，代码为“from fastmcp import Context”。具体操作时，只需在工具函数定义语句中添加带 Context 类型提示的参数即可，该参数对 MCP 客户端不可见。添加上下文参数的工具函数定义语句例如“async def add(a:int,b:int,ctx:Context) -> int:”。

上下文（Context）对象提供以下功能：

- **日志记录：**ctx.debug()、ctx.info()、ctx.warning()、ctx.error()
- **进度报告：**ctx.report_progress(progress, total)
- **资源访问：**ctx.read_resource(uri)
- **采样调用：**ctx.sample(...)
- **请求信息：**ctx.request_id、ctx.client_id

关于 Contex 对象的完整功能及其应用，详见进阶功能中的§ 5.1 小节 MCP 上下文部分。

3.5.11 工具转换

工具转换（Tool Transformation）允许你从现有工具创建新的增强工具。这一强大功能使你能够根据不同的上下文调整工具、简化复杂接口，或在无需重复代码的情况下添加自定义逻辑。详细可以查阅 FastMCP 官方文档中 *Patterns—Tool Transformation* 部分（https://gofastmcp.com/v2/patterns/tool-transformation）。

在现有工具参数或描述对 LLM 不够直观、希望向 LLM 隐藏部分参数、增加输入验证、修改现有工具的输出格式等应用场景，我们希望在现有工具基础上修改以使工具更加符合当前新的场景，则需要使用工具转换功能。

该功能有些类似类的继承，因此参考类继承当中的说法，原始工具也可被称为“父工具”。

1. 基础的工具转换

创建转换工具的主要方法是使用 Tool.from_tool()类方法。简单来说，我们可以用它来更改工具的顶层元数据，例如名称、描述或标签等。

使用基础的工具转换功能的样例请参考代码 3.11。

代码 3.11　使用 Tool.from_tool()方法创建新工具组件代码

```
1. from fastmcp import FastMCP
2. from fastmcp.tools import Tool
3.
4. mcp = FastMCP()
5.
6. # 原始的通用工具
7. @mcp.tool
8. def search(query: str, category: str = "all") -> list[dict]:
9.     """Searches for items in the database."""
10.     return database.search(query, category)
11.
12. # 通过更改元数据创建更详细描述的版本
13. product_search_tool = Tool.from_tool(
14.     search,
15.     name="find_products",
16.     description="""
17.         Search for products in the e-commerce catalog.
18.         Use this when customers ask about finding specific items,
19.         checking availability, or browsing product categories.
20.         """,
21. )
22.
23. mcp.add_tool(product_search_tool)
```

需要注意的是，使用 Tool.from_tool 方法需要先从 fastmcp.tools 库导入 Tool 类，并且转换获得新工具后需要用 FastMCP 实例的 mcp.add_tool(example_tool) 方法注册，否则无法调用新工具。

当转换工具创建新工具时，原始工具仍然会注册在服务器上，为避免混淆，建议使用 disable()方法等禁用原始工具。

Tool.from_tool()方法的参数如表 3.13 所示。

表 3.13　Tool.from_tool()方法参数表

参数	描述	说明
tool	要转换的工具	**这是唯一必需的参数**
name	新工具的可选名称	建议给出，此参数缺省时新旧工具重名
description	新工具的可选描述	
transform_args	一个包含 ArgTransform 对象的字典，每个对象对应你想要修改的一个参数	
transform_fn	一个可选函数，调用时将替代父工具的逻辑	
output_schema	控制输出架构和结构化输出。传入 None 以移除原始工具的自定义输出模式	
tags	新工具的可选标签集合	
annotations	新工具的可选 ToolAnnotations 集合	
serializer	一个可选函数，用于序列化新工具的结果	
meta	控制工具的元信息。传入 None 以移除元信息，传入任意字典以设置元信息，或不传入以继承父工具的元信息	

注：此表中与@tool 装饰器参数表中同名参数用法相同，但缺省时继承原始工具同名参数值。

下面将对表 3.13 中的部分参数作补充说明。

• **[transform_args] 修改参数**：提供一个字典（dict），其中每个键值对的键为 str 格式的原始工具参数名称，值为 ArgTransform 对象（ArgTransform 对象的参数为需要修改的工具参数属性，如表 3.14 所示）。使用 ArgTransform 前需要先从 fastmcp.tools.tool_transform 库中导入 ArgTransform 类。例如以下代码将原始工具 search 的参数 q 重命名为 search_query，将原始工具参数 api_key 隐藏并设置默认值为“test”，将原始工具参数 timestamp 隐藏并设置为每次调用时通过指定的 lambda 表达式生成：

```
from fastmcp.tools.tool_transform import ArgTransform
new_tool = Tool.from_tool(
    search,
    transform_args={
        "q": ArgTransform(name="search_query"),
        "api_key": ArgTransform(
            hide=True,
            default="test"),
        'timestamp': ArgTransform(
                hide=True,
                default_factory=lambda: datetime.now())
    }
)
```

表 3.14 ArgTransform 类参数表

参数	描述
name	参数的新名称
description	参数的新描述
default	参数的新默认值
default_factory	一个函数，调用该函数将为参数生成默认值。这对于需要在每次工具调用时生成的参数（如时间戳或唯一 ID）非常有用
hide	是否向 LLM 隐藏该参数

required	参数是否为必需，通常用于将可选参数变为必需参数
type	参数的新类型

注：①只能在 hide=True 时使用 default_factory，因为动态默认值无法在客户端的 JSON schema 中表示。②只能对未声明默认值的参数设置 required=True。

• **[transform_fn] 父工具逻辑替换**：此参数是一个**异步**函数，它完全替换父工具的逻辑。此函数的名称和文档字符串（docstring）均会被忽略，只有它的参数用于确定最终模式。任何在父工具模式或 transform_args 中不存在的参数都将被添加到新工具的模式中。请注意，当 transform_args 与 transform_fn 函数具有相同参数名时，如果提供了 transform_args 的元数据，该元数据将优先使用。使用 tranform_fn 完全替代父工具逻辑的代码例如：

```
async def my_custom_logic(user_input: str, max_length: int = 100) -> str:
    # 在此编写您的自定义逻辑——这完全替换父工具
    return f"自定义结果: {user_input[:max_length]}"

Tool.from_tool(transform_fn=my_custom_logic)
```

大多数情况下，我们并不希望完全替换父工具的行为，而是希望**在保留父工具核心功能的同时，添加验证、修改输入或对输出进行后处理**。为此，FastMCP 提供了特殊的 forward()和 forward_raw()函数。它们都是异步函数，允许我们在 transform_fn 内部调用父工具。forward()函数（**推荐使用**）根据 ArgTransform 对象配置自动处理参数映射，调用时需传入转换后的参数名。forward_raw()函数则绕过所有转换，直接使用原始参数名调用父工具。除非你需要进行复杂的参数操作（可能没有使用 arg_transforms），否则很少需要此函数。

最常见的转换模式是在调用父工具之前对（可能已重命名的）参数进行验证。例如代码 3.12，add_new 工具由 add 工具转换得到，add_new 工具的参数从 add 工具的 a 和 b 改名为 x 和 y，在父工具逻辑前添加输入必须为正数验证。

代码 3.12　使用 transform_fn 和 forward()函数新增处理逻辑代码

```
1. from fastmcp import FastMCP
2. from fastmcp.tools import Tool
3. from fastmcp.tools.tool_transform import forward
4. from fastmcp.tools.tool_transform import ArgTransform
5.
6. mcp=FastMCP("Demo_add", host="0.0.0.0", port=8000)
7.
8. @mcp.tool()
9. async def add(a:int,b:int) -> int:
10.     """
11.     将两个整数相加
12.     Args:
13.         a:第一个加数
14.         b:第二个加数
15.     Return:
16.         两个数 a 和 b 的和
17.     """
18.     return a+b
19.
```

```
20. async def ensure_positive(x: int, y: int) -> int:
21.     if x<=0 or y<=0:
22.         raise ValueError("a and b must be positive")
23.     return await forward(x=x, y=y)
24.
25. add_new=Tool.from_tool(add,name="add_new",
26.                        transform_args={
27.                            "a": ArgTransform(name="x"),
28.                            "b": ArgTransform(name="y")
29.                        },
30.                        transform_fn=ensure_positive)
31. mcp.add_tool(add_new)
32.
33. if __name__=="__main__":
34.     mcp.run(transport="streamable-http")
```

注意代码 3.12 中，使用 forward()函数时第 20 行和第 23 行中参数的名称应该与新工具的保持一致，如果使用父工具的参数名称则会报错，本例中会有错误“ ValueError: Function missing parameters required after transformation: x, y. Function declares: a, b”。使用 forward_raw()则有所区别，具体的案例详见 FastMCP 官方文档中 *Tool Transformation—Modifying Tool Behavior—Calling the Parent Tool* 部分（https://gofastmcp.com/v2/patterns/tool-transformation#calling-the-parent-tool）。

transform_fn 也可使用**kwargs 传递参数，在 ArgTransform 配置应用之后，它会接收来自父工具的所有参数。这对于创建灵活的验证函数非常有用，无需将所有参数都显式添加到函数签名中。采用此写法的 transform_fn 函数样例如下：

```
async def ensure_a_positive(x: int, **kwargs) -> int:
    if x <= 0:
        raise ValueError("x 必须为正数")
    return await forward(x=x, **kwargs)
```

在上述示例中，**kwargs 接收到的是重命名后的参数 y，而不是原始参数 b。因此，建议与 forward()一起使用，而不是 forward_raw()。

transform_fn 也可用于输出逻辑替换，例如输出 ToolResult 对象实现修改输出格式，例如：

```
async def custom_output(**kwargs) -> ToolResult:
    result = await forward(**kwargs)
    return ToolResult(content=[...], structured_content={...})

new_tool=Tool.from_tool(parent_tool, transform_fn=custom_output)
```

• **[serializer] 序列化器**：默认的序列化器将 Python 对象转为 MCP 协议消息格式（JSON-RPC）。如果想要采用自定义格式可用此参数指定自定义序列化器，通常使用默认的即可。

• **[meta] 元数据**：若传入 None 则移除元信息；若传入任意字典则设置元信息，覆盖原始工具的元数据信息；若此参数缺省则继承父工具的元信息。

2. 将现有 Python 代码库方法暴露为工具

除了本文一直介绍的自己编写工具代码之外，还可以使用 Tool.from_function

方法将现有的用于与外部服务或系统进行交互的 Python 代码库（例如 GitHub 客户端、API 客户端、数据库客户端等）直接暴露为 MCP 工具。这种模式消除了样板化的包装函数，并将工具视为对客户端方法的注解。代码 3.13 为一个将 GitHub 客户端转为 MCP 工具样例。

代码 3.13　将现有 Python 代码库暴露为 MCP 工具组件代码

```
1. from fastmcp import FastMCP
2. from fastmcp.tools import Tool
3. from fastmcp.tools.tool_transform import ArgTransform
4.
5. mcp = FastMCP("GitHub Tools")
6.
7. # Wrap a client method directly as a tool
8. get_repo_tool = Tool.from_tool(
9.     tool=Tool.from_function(fn=github_client.get_repository),
10.     description="Get basic information about a GitHub repository.",
11.     transform_args={
12.         "owner": ArgTransform(description="The owner of the
repository."),
13.         "repo": ArgTransform(description="The name of the
repository."),
14.     }
15. )
16.
17. mcp.add_tool(get_repo_tool)
```

如果包含一些不应暴露给大语言模型的内部参数（如调试标志、认证令牌、速率限制设置等）。通过调整 Tool.from_tool()方法的参数，例如使用 hide=True 并配合默认值，可以自动处理这些参数，仅暴露合适的部分参数给大模型。

3. 链式工具转换

就像类的多级继承那样，通过工具转换得到的新工具，可以作为父工具参与新的工具转换产生更新的工具。这使得我们能够分层构建复杂的行为，例如先重命名参数，然后为已重命名的工具添加验证逻辑。

3.5.12 工具组件重复管理与移除

1. 重复工具管理

当在代码中尝试注册多个同名工具时，我们可以手动控制 FastMCP 服务器的行为。在创建 FastMCP 实例时，通过 on_duplicate_tools 参数进行配置。

重复工具的处理选项包括：

- **"warn"（默认）**：记录警告，并使用新工具替换旧工具。
- **"error"**：抛出一个 ValueError，阻止重复注册。
- **"replace"**：静默替换，不记录并用新工具覆盖现有工具。
- **"ignore"**：保留原工具，忽略新工具的注册尝试。

2. 移除工具

除了 § 3.5.3 中所述的工具禁用方法，我们还可以使用 remove_tool()方法动

态移除服务器中的工具。假设 FastMCP 实例名为 mcp，则使用该方法代码如下：

```
mcp.remove_tool("example_tool")
```

需要注意的是，这种移除与 disable()方法不同，remove_tool 之后无法恢复，除非重新运行 MCP 服务器代码以重新注册工具组件。对于生产环境而言，因为不能随意地重新启动服务器，所以不推荐使用 remove_tool 方法。

3.5.13 自定义工具序列化

默认情况下，当需要将工具返回值转换为文本时，FastMCP 将其序列化为 JSON。但可以通过在创建服务器时向参数 tool_serializer 提供自定义序列化器函数来自定义此行为，例如：

```
import yaml
from fastmcp import FastMCP

# 定义一个将字典格式化为 YAML 的自定义序列化器
def yaml_serializer(data):
    return yaml.dump(data, sort_keys=False)

# 使用自定义序列化器创建服务器
mcp = FastMCP(name="MyServer", tool_serializer=yaml_serializer)

@mcp.tool
def get_config():
    """以 YAML 格式返回配置。"""
    return {"api_key": "abc123", "debug": True, "rate_limit": 100}
```

序列化器函数适用于工具的**所有非字符串返回值，已经返回字符串的工具会绕过序列化器**。该函数接受任何非字符串的数据对象并返回字符串表示。

自定义序列化在以下的情况下很有用：需要以特定方式格式化数据（如 YAML 或自定义格式等）时；需要控制特定的序列化选项（如缩进或排序等）时；需要在将数据发送给客户端之前添加元数据或转换数据时。

注意，如果序列化器函数引发异常，工具将回退到默认的 JSO 序列化以避免破坏服务器。

3.6 MCP 资源组件

资源（Resource）组件表示 MCP 客户端可以读取的数据或文件，而资源模板则扩展了这一概念，允许客户端根据 URI[5]中传递的参数请求动态生成的资源。资源可被分为两类——静态资源、资源模板（动态资源）。

资源为 LLM 或客户端应用程序提供只读数据访问。当客户端请求资源 URI 时：FastMCP 会查找相应的资源定义；如果该资源是动态的（由函数定义），则执行该函数；内容（文本、JSON、二进制数据）将返回给客户端。这使得 LLM

[5] URI 是统一资源标识符，用以唯一标识一个资源，不包括地址和访问方式等信息。下文中有些地方会要求提供 URL 以供定位和访问资源。需要注意区分 URI 和 URL。

能够访问与对话相关的文件、数据库内容、配置或动态生成的信息。

3.6.1 @resource 装饰器

和工具组件类似，定义资源最常见的方式是装饰一个 Python 函数。该装饰器需要资源的唯一 URI。与装饰器相关的有三个核心概念：

• **URI**：@resource 的第一个参数是唯一的 URI（例如"resource://greeting"），客户端通过此 URI 请求数据。

• **延迟加载（Lazy Loading）**：被@resource 装饰的函数仅在 MCP 客户端通过 resources/read 明确请求该资源 URI 时才会执行。

• **元数据推断**：默认情况下，以函数名称作为资源名称，以函数的注释（文档字符串 docstring）作为资源的描述。

与工具组件相似，我们可以通过@mcp.resource 装饰器中的参数自定义资源属性，该装饰器的参数表如表 3.15 所示。可能有部分参数未在此列出，可以查阅 FastMCP 官方文档中 *Resources—The @resource Decorator—Decorator Arguments* 部分（https://gofastmcp.com/v2/servers/resources#decorator-arguments）。

表 3.15　@mcp.resource 装饰器常见参数表

参数	描述	数据类型	默认值	备注
uri	资源的唯一标识	str		**必须提供此参数**
name	通过 MCP 服务器公开显示的资源名称	str | None	None	参数缺省则使用 uri 作为资源名称
mime_type	指定内容类型。FastMCP 通常会推断默认类型（如 text/plain 或 application/json），但对于非文本类型，明确指定更佳	str | None	None	
title	资源的可读标题	str | None	None	
description	通过 MCP 服务器公开的资源描述	str | None	None	此参数覆盖函数注释（文档字符串），建议此参数缺省
tags	标签，用于对资源分类	set[str] | None	None	标签管理详见 § 3.4
enabled	启用或禁用资源	bool	True	
icons	资源的图标列表	list[Icon] | None	None	v2.13.0 版本新增 详见 § 5.3.2
meta	关于资源的可选元信息，会被传递给 MCP 客户端（**主要面向人类**）	dict[str, Any] | None	None	v2.11.0 版本新增
annotations	可选的 Annotations 对象或字典，用于添加有关资源的额外注释（**主要面向程序**）	Annotations | dict | None	None	v2.2.7 版本新增

注：①灰色标注的参数是与工具组件参数有区别的，未标注的与工具组件参数同名的参数用法相同；②annotations 参数支持的属性未在表中列出，在下面的补充说明部分详细介绍

在下面的章节中，将对一些参数进行额外的补充说明。

1. [uri] 资源唯一标识符

uri 参数指定了该资源组件的唯一标识符，不同资源的唯一标识符不可相同，该参数不可缺省。通常 uri 以形如“resource://greeting”、“data://config”的形式给出。FastMCP 可自动地根据 uri 参数字符串中是否包含{param}形式的占位符（例如"resource://{city}/weather"）判断该资源是静态资源还是资源模板（动态资源），另一种判断方式是根据函数有无定义参数来判断资源是否为静态的。

2. [mime_type] MIME 类型

多用途互联网邮件扩展类型（Multipurpose Internet Mail Extensions）又称 MIME 类型或内容类型，是用于标识互联网传输内容格式的标准分类体系，由主类型、子类型及可选参数构成，通过文件扩展名关联应用程序。一个 MIME 类型包括一个类型（type），一个子类型（subtype），其格式为“<类型>/<子类型>”，例如“text/plain”。

通常情况下，FastMCP 可以根据资源文件的后缀名自动推断 MIME 类型。为了准确起见，可以通过 mmie_type 参数手动表明资源的 MMIE 类型。常见的资源文件后缀名对应的 MIME 类型如表 3.16 所示，完整的后缀名与 MIME 类型对应表详见附录 A。

表 3.16 常见资源文件后缀名与 MIME 类型对应表

后缀名	MIME 类型
.txt, .log	text/plain
.html, .htm	text/html
.csv	text/csv
.md, .markdown	text/markdown
.xml	text/xml 或 application/xml
.jpg, .jpeg	image/jpeg
.png	image/png
.gif	image/gif
.webp	image/webp
.svg	image/svg+xml
.bmp	image/bmp
.json	application/json
.pdf	application/pdf
.zip	application/zip
.rar	application/vnd.rar
.mp3	audio/mpeg
.wav	audio/wav
.aac	audio/aac
.mp4	video/mp4
.mepg	video/mepg

注：.xml 对应两种 MIME 类型，更推荐使用 application/xml

3. [enabled] 启用/禁用资源

我们可以通过启用或禁用来控制静态资源和资源模板的可见性与可用性。被禁用的资源不会出现在可用资源或模板列表中，尝试读取已禁用的资源将导致“未知资源”（“Unknown resource”）错误。

与工具组件相同，默认情况下所有资源均为启用状态。在创建资源时可以设置@mcp.resource 装饰器中的 enabled 参数为 False 进行禁用。

也可以在资源创建后通过编程方式动态地切换其状态，例如：

```
@mcp.resource("data://config")
def get_config(): return {"version": 1}

# Disable and re-enable the resource
get_config.disable()
get_config.enable()
```

通过设置@mcp.resource 装饰器 enabled 参数、通过 disable()和 enable()方法、使用标签管理，这三种方法启用/禁用资源组件的优先级与工具组件的相同。

除此之外，还可以通过设计中间件的方式，实现更加精细的资源启用/禁用，例如针对特定的用户禁止请求特定的资源等，详见§5.2 小节。

4. [annotations] 资源组件注释

与@tool 装饰器类似，@resource 装饰器的 annotations 参数是面向程序的注释，仅有几个可选的注释项，包括注明资源是否只读等。这些注释项可被视为 annotations 参数的参数，与@tool 装饰器不同的是，@resource 装饰器的 annotations 参数仅有两个注释项，如表 3.17 所示。

表 3.17 @mcp.resource 装饰器 annotations 参数的参数表

参数	描述	数据类型	默认值	备注
readOnlyHint	指示资源是否为只读的。如果为 true，资源是只读的，不会修改其环境	bool	True	
idempotentHint	指示资源是否是幂等[6]的，即重复读取资源是否会产生额外影响。如果为 true，重复读取资源对其环境不会产生额外影响	bool	True	

需要注意的是，理论上 annotations 参数内容应当被包含在 MCP 服务器对 resources/list 的响应内容中的“annotations”字段内。但实际测试发现并未被包含，与@mcp.tool 装饰器不同。

3.6.2 异步资源与同步资源

与工具组件类似，使用装饰器注册组件时，函数有同步和异步两种写法，即组件函数不带或带有 async 关键字修饰。在大多数的应用场景下，例如资源以文

[6] 幂等性（Idempotent）：同一个操作无论执行一次还是多次，产生的结果（或系统的状态）都是相同的，并且不会因为重复执行而产生额外的副作用。在 MCP 资源组件中，在资源本身不变的前提下，重复请求该资源若干次，产生的后果与仅请求该资源一次相同，则该资源具有幂等性。通常只读的资源天然地具有幂等性。MCP 客户端可以放心地请求幂等的资源，而无需担心重复请求导致预期外后果。

件形式存储在服务器上，或者以函数返回值形式提供给客户端等，可以直接采用同步资源的写法，例如：

```
@mcp.resource("resource://greeting")
def get_greeting() -> str:
    """
    提供一句简单的问候语
    Return:
        问候语
    """
    return "Hello world!"
```

但是对于执行 I/O 操作（例如，从数据库或网络读取）的资源函数，若采用同步资源形式则会在被请求时占用服务器资源，不利于生产环境高并发场景，必须使用异步资源形式（即使用 async 关键字修饰 def）以避免阻塞服务器。代码 3.14 为一个异步读取特定日志文件的异步资源函数案例。

代码 3.14　MCP 异步资源组件案例

```
 1. import aiofiles
 2. from fastmcp import FastMCP
 3.
 4. mcp = FastMCP(name="DataServer")
 5.
 6. @mcp.resource("file:///app/data/important_log.txt",
mime_type="text/plain")
 7. async def read_important_log() -> str:
 8.     """异步读取特定日志文件的内容。"""
 9.     try:
10.         async with aiofiles.open("/app/data/important_log.txt",
mode="r") as f:
11.             content = await f.read()
12.         return content
13.     except FileNotFoundError:
14.         return "未找到日志文件。"
```

与异步的工具组件类似，在函数中的各行代码需要注意完全按照异步写法编写，即在需要的地方添加 async 和 await 关键字。如果缺少了必要的关键字修饰，理论上 IDE 会提示“未等待协程”等。

3.6.3 资源组件的返回值

FastMCP 自动将组件函数的返回值转换为适当的 MCP 资源内容，函数返回值数据类型和转换得到的资源类型对应关系如表 3.18 所示。

表 3.18　函数返回值类型和转换得到的资源类型对应表

函数返回值类型	转换得到的资源类型
str	作为 TextResourceContents 发送（默认 mime_type="text/plain"）。
dict、list、pydantic.BaseModel	自动序列化为 JSON 字符串并作为 TextResourceContents 发送（默认 mime_type="application/json"）。
bytes	Base64 编码并作为 BlobResourceContents 发送。**必须在@resource 装饰器参数中指定适当的 mime_type**（参见§3.6.1 中 mime_type 参数部分，例如，"image/png"、"application/octet-stream"）。

None	返回空的资源内容列表。

3.6.4 资源模板

与普通的静态资源不同，资源模板的 URI 中包含可变的嵌入参数。资源模板的 URI 被称为“URI 模板”。资源模板的注册使用相同的@mcp.resource 装饰器定义模板，但在 URI 字符串中包含形如{parameter_name}的占位符，并在函数定义语句中添加相应的参数，除了上下文对象参数以外，**函数的参数必须与占位符中的参数同名**。

在不隐藏查询参数的情况下（隐藏查询参数的情况详见本节中“3. URI 查询参数匹配”部分）（均不计算上下文对象参数）：假设某资源模板的 URI 模板中有 2 个占位符，但对应的组件函数定义中仅有 1 个参数，则在服务器启动时会报错。假设某资源模板的 URI 模板中有 1 个占位符，但对应的组件函数定义中有 2 个参数，则在服务器启动时也会报错。

当 MCP 客户端请求资源时，服务器自动将客户端提供的 URI 与各个资源的 URI 和资源模板的 URI 模板匹配，匹配成功则从中提取对应的参数值以调用组件函数。**当客户端提供的 URI 可以同时与某个资源的 URI、某个资源模板的 URI 模板匹配时，优先请求该资源，而不是请求资源模板。当客户端提供的 URI 可以同时与多个资源模板的 URI 模板匹配时，优先请求先注册的资源模板。**例如：

• 资源 A 的 URI 为“resource://greeting”，资源模板 B 的 URI 模板为“resource://{name}”，无论 A 和 B 中的哪个先注册。当客户端提供 URI 为“resource://greeting”时，请求资源 A。

• 资源模板 B 的 URI 模板为“resource://{name}”，资源模板 C 的 URI 模板为“resource://{test}”，B 比 C 先注册。当客户端提供 URI 为“resource://temp”时，请求资源模板 B。

总之，当客户端提供的 URI 能与多个资源/资源模板的 URI 匹配时，优先级为 资源>先注册的资源模板>后注册的资源模板。若有多个资源组件的 URI 完全相同，则需参考§ 3.6.10的《资源组件重复管理》部分。由于可维护性较低，强烈不建议多个资源/资源模板的 URI 匹配范围存在重叠。

资源模板与常规资源共享大多数配置选项（名称、描述、mime_type、标签、注解），@mcp.resource 装饰器的参数设置与静态资源相同，但添加了定义映射到函数参数的 URI 参数的能力。

1. URI 标准参数匹配

最常见的占位符形如“{parameter_name}”，组件函数必须包含同名参数，否则无法在客户端请求资源时从 URI 自动获取参数值传入函数。

同一个资源模板的 URI 中可以同时包含多个参数占位符，例如“repos://{owner}/{repo}/info”，以便更加灵活地按需生成资源。

资源模板为每个参数值生成一个新资源，这意味着资源可以按需动态创建。例如，如果注册了资源模板 "user://profile/{name}"，MCP 客户端可以请求 "user://profile/ford" 或 "user://profile/marvin" 来检索这两个用户配置文件中的任意一个作为资源，而无需单独注册每个资源。

代码 3.15 为定义了两个资源模板的样例代码。

代码 3.15　MCP 资源模板案例

```
1. from fastmcp import FastMCP
2.
3. mcp = FastMCP(name="DataServer")
4.
5. # 模板 URI 包含 {city} 占位符
6. @mcp.resource("weather://{city}/current")
7. def get_weather(city: str) -> dict:
8.     """为特定城市提供天气信息。"""
9.     # 在实际实现中，这将调用天气 API
10.    # 这里我们使用简化的逻辑作为示例
11.    return {
12.        "city": city.capitalize(),
13.        "temperature": 22,
14.        "condition": "Sunny",
15.        "unit": "celsius"
16.    }
17.
18. # 具有多个参数和注解的模板
19. @mcp.resource(
20.    "repos://{owner}/{repo}/info",
21.    annotations={
22.        "readOnlyHint": True,
23.        "idempotentHint": True
24.    }
25. )
26. def get_repo_info(owner: str, repo: str) -> dict:
27.    """检索有关 GitHub 仓库的信息。"""
28.    # 在实际实现中，这将调用 GitHub API
29.    return {
30.        "owner": owner,
31.        "name": repo,
32.        "full_name": f"{owner}/{repo}",
33.        "stars": 120,
34.        "forks": 48
35.    }
```

代码 3.15 定义了这两种资源模板后，MCP 客户端可以使用例如以下的 URI 请求相应的各种资源：

- weather://london/current → 返回伦敦的天气
- weather://paris/current → 返回巴黎的天气
- repos://jlowin/fastmcp/info → 返回有关 jlowin/fastmcp 仓库的信息
- repos://prefecthq/prefect/info → 返回有关 prefecthq/prefect 仓库的信息

代码 3.15 中的资源模板实现的功能也可以写成工具组件来实现，但 MCP 客

户端可能会预先获取常用资源以丰富上下文，工具则仅在模型认为需要时才调用。当数据是静态或变化不频繁时，或者希望客户端能够像浏览文件一样浏览这些数据等情况下，建议采用资源组件写法。而当参数复杂以至于不适合放在路径中，或者信息不需要长期使用等情况下，建议采用工具组件写法。

需要注意的是，具有*args 的函数不支持作为资源模板。然而，与工具组件和提示词组件不同，资源模板支持**kwargs，因为 URI 模板定义了将被收集并作为关键字参数传递的特定参数名称。[7]使用**kwargs 传递参数的资源模板例如：

```
@mcp.resource("user://profile/{name}")
def get_user_profile(**kwargs):
    name = kwargs.get("name")
    # 其余代码省略
```

2. URI 通配符参数匹配

形如“{parameter_name}”的标准参数的值不能包含“/”，当参数值需要包括文件路径等含有斜杠时，必须使用形如“{param*}”的通配符参数（Wildcard Parameters）。通配符会捕获从当前位置到 URI 模板中定义的下一个元素（无论是静态文本还是其他参数）之间的所有后续路径段。例如某资源模板组件有 URI 模板“path://{filepath*}”，则可以匹配 URI“path://docs/server/resources.mdx”，此时 filepath 参数值为“docs/server/resources.mdx”。

URI 通配符参数可以与标准参数同时用于单个 URI 模板中，例如“repo://{owner}/{path*}/template.py”。但通常单个 URI 模板不能包含连续的通配符参数，原因是会产生歧义，例如有 URI 模板“data://files/{folder*}/{filename*}”，当传入 URI“data://files/aaa/bbb/ccc.txt”时，参数可以有两种匹配方案——①folder=”aaa”，filename=”bbb/ccc.txt”；②folder=”aaa/bbb”，filename=”ccc.txt”。这样就会产生歧义，此种 URI 模板是不允许的。在写 URI 模板时，可以思考一下会不会产生歧义，以免产生错误。

如果无法写出不会产生歧义的 URI 模板，则应写成工具组件，直接为组件函数提供参数，而不依赖于自动从 URI 中提取参数。

代码 3.16 为使用 URI 通配符参数匹配的资源模板代码示例。

代码 3.16　使用 URI 通配符参数匹配的 MCP 资源模板案例

```
1. from fastmcp import FastMCP
2.
3. mcp = FastMCP(name="DataServer")
4.
5.
6. # 标准参数仅匹配一个段
```

[7] **为什么不支持*args：***args 用于按照位置收集参数，调用时必须顺序传参。而实际上客户端请求时采用类似于“name=”test”” 的方式传参而非顺序传参，*args 仅能收到空元组，无法正确接收参数。

为什么支持kwargs：****kwargs 用于收集任意数量的关键字参数，传入的参数以字典形式储存。但在函数体内需要手动为参数赋值，例如 name = kwargs.get("name")。

```
7. @mcp.resource("files://{filename}")
8. def get_file(filename: str) -> str:
9.     """按名称检索文件。"""
10.     # 将仅匹配 files://<single-segment>
11.     return f"File content for: {filename}"
12.
13.
14. # 通配符参数可以匹配多个段
15. @mcp.resource("path://{filepath*}")
16. def get_path_content(filepath: str) -> str:
17.     """检索特定路径的内容。"""
18.     # 可以匹配 path://docs/server/resources.mdx
19.     return f"Content at path: {filepath}"
20.
21.
22. # 混合标准和通配符参数
23. @mcp.resource("repo://{owner}/{path*}/template.py")
24. def get_template_file(owner: str, path: str) -> dict:
25.     """从特定仓库和路径检索文件，但
26.     仅当资源以 `template.py` 结尾时"""
27.     # 可以匹配 repo://jlowin/fastmcp/src/resources/template.py
28.     return {
29.         "owner": owner,
30.         "path": path + "/template.py",
31.         "content": f"File at {path}/template.py in {owner}'s
repository"
32.     }
```

通配符参数在这些情况下很有用：处理文件路径或分层数据；创建需要捕获可变长度路径段的 API；构建类似于 REST API 的 URL 模式。

3. URI 查询参数匹配

FastMCP 支持使用{?param1,param2}语法的 RFC 6570 表单样式查询参数。查询参数提供了一种干净的方式来向资源传递可选配置，而不会使路径变得杂乱。**查询参数必须是可选的函数参数（即具有默认值）**，而路径参数映射到必需的函数参数。这强制实行了清晰的分离：必需数据在路径中，可选配置在查询参数中。

对于带有查询参数的 URI 模板的资源模板，MCP 客户端在请求资源时，提供的 URI 格式类似于 HTTP GET 请求的 URL 格式，即在 URI 末尾采用形如“?param1=12&param2=aaa”的形式提供一个或多个查询参数值。

代码 3.17 是一个注册多种带有查询参数的 URI 模板的资源模板组件的示例。

代码 3.17　使用 URI 查询参数匹配的 MCP 资源模板案例

```
1. from fastmcp import FastMCP
2.
3. mcp = FastMCP(name="DataServer")
4.
5. # 基本查询参数
6. @mcp.resource("data://{id}{?format}")
7. def get_data(id: str, format: str = "json") -> str:
8.     """Retrieve data in specified format."""
9.     if format == "xml":
```

```
10.         return f"<data id='{id}' />"
11.     return f'{{"id": "{id}"}}'
12.
13. # 带有类型强制转换的多个查询参数
14. @mcp.resource("api://{endpoint}{?version,limit,offset}")
15. def call_api(endpoint: str, version: int = 1, limit: int = 10,
offset: int = 0) -> dict:
16.     """调用带有分页的 API 端点。"""
17.     return {
18.         "endpoint": endpoint,
19.         "version": version,
20.         "limit": limit,
21.         "offset": offset,
22.         "results": fetch_results(endpoint, version, limit, offset)
23.     }
24.
25. # 带有通配符的查询参数
26. @mcp.resource("files://{path*}{?encoding,lines}")
27. def read_file(path: str, encoding: str = "utf-8", lines: int = 100)
-> str:
28.     """读取文件，带有可选编码和行数限制。"""
29.     return read_file_content(path, encoding, lines)
```

对于注册了代码 3.17 中的资源模板的 MCP 服务器，客户端如果提供以下 URI 以请求资源，从服务器中请求资源的结果分别为：

- data://123 → 使用默认格式 "json"
- data://123?format=xml → 使用格式 "xml"
- api://users?version=2&limit=50 → version=2, limit=50, offset=0
- files://src/main.py?encoding=ascii&lines=50 → 按照查询参数设置 encoding=”ascii”, lines=50 自定义编码和行数限制

FastMCP 会根据函数定义中的参数类型提示（例如 int、float、bool、str 等）自动将查询参数字符串值强制转换为正确的类型。如果无法转换（例如为 int 型参数提供“aaa”之类的 str 型值），则会产生报错，但这类报错不会影响服务器运行，服务器对来自此客户端和其它客户端的其它请求仍然能够正常响应。

若要完全对客户端隐藏一个或多个可选参数（即始终使用默认值），只需在 URI 模板中省略它们，例如：

```
# 客户端可以通过查询字符串覆盖 max_results
@mcp.resource("search://{query}{?max_results}")
def search_configurable(query: str, max_results: int = 10) -> dict:
    return {"query": query, "limit": max_results}

# 客户端无法覆盖 max_results（不在 URI 模板中）
@mcp.resource("search://{query}")
def search_fixed(query: str, max_results: int = 10) -> dict:
    return {"query": query, "limit": max_results}
```

上面的代码是不隐藏 max_results 参数的，而下面的代码则对 MCP 客户端隐藏了 max_results 参数，始终采用默认值 10 作为参数值。

隐藏查询参数通常有如下应用场景：在生产环境中预先规划若干参数，但目前尚未实现部分参数的功能，之后再开发；编写一个资源函数，但需要对应多个 URI 模板（详见下文），部分 URI 模板无需提供某些参数值，使用默认值即可。

4. 单个资源函数对应多个 URI 模板

FastMCP 允许一个资源函数对应多个 URI 模板，注册多个资源模板。要实现这样的目标，则不能采用常规方式使用@mcp.resource 装饰器注册资源模板，而需要使用手动应用装饰器注册，如代码 3.18 所示。

代码 3.18　手动注册多个资源模板暴露同一函数案例

```
1. from fastmcp import FastMCP
2.
3. mcp = FastMCP(name="DataServer")
4.
5. # 定义可以通过不同标识符访问的用户查找函数
6. def lookup_user(name: str | None = None, email: str | None = None) ->
dict:
7.     """通过名称或电子邮件查找用户。"""
8.     if email:
9.         return find_user_by_email(email)  # 伪代码
10.    elif name:
11.        return find_user_by_name(name)  # 伪代码
12.    else:
13.        return {"error": "No lookup parameters provided"}
14.
15. # 手动将多个装饰器应用于同一函数
16. mcp.resource("users://email/{email}")(lookup_user)
17. mcp.resource("users://name/{name}")(lookup_user)
```

代码 3.18 所创建的两个资源模板可以和两种格式的 URI 匹配，例如：

- users://email/alice@example.com → 通过电子邮件查找用户（使用 name=None）
- users://name/Bob → 通过名称查找用户（使用 email=None）

3.6.5 资源类

虽然使用上文中使用的@mcp.resource 装饰器注册资源非常适合动态内容，但需要将静态文件或简单的文本等直接注册为资源时，还可以使用 FastMCP 类的 add_resource()方法和具体的 Resource 子类。

代码 3.19 展示了使用 add_resource()方法注册静态文件和几种 Resource 子类为资源的写法，包括使用自定义资源键（详见§3.6.6）的案例。

代码 3.19　使用 add_resource()方法和 Resource 子类注册资源

```
1. from pathlib import Path
2. from fastmcp import FastMCP
3. from fastmcp.resources import FileResource, TextResource,
DirectoryResource
4.
5. mcp = FastMCP(name="DataServer")
6.
```

```
 7. # 1. 直接暴露静态文件
 8. readme_path = Path("./README.md").resolve()
 9. if readme_path.exists():
10.     # 使用 file:// URI 方案
11.     readme_resource = FileResource(
12.         uri=f"file://{readme_path.as_posix()}",
13.         path=readme_path, # 实际文件的路径
14.         name="README File",
15.         description="项目的 README。",
16.         mime_type="text/markdown",
17.         tags={"documentation"}
18.     )
19.     mcp.add_resource(readme_resource)
20.
21. # 2. 暴露简单的预定义文本
22. notice_resource = TextResource(
23.     uri="resource://notice",
24.     name="Important Notice",
25.     text="System maintenance scheduled for Sunday.",
26.     tags={"notification"}
27. )
28. mcp.add_resource(notice_resource)
29.
30. # 3. 使用与 URI 不同的自定义键
31. special_resource = TextResource(
32.     uri="resource://common-notice",
33.     name="Special Notice",
34.     text="This is a special notice with a custom storage key.",
35. )
36. mcp.add_resource(special_resource, key="resource://custom-key")
37.
38. # 4. 暴露目录列表
39. data_dir_path = Path("./app_data").resolve()
40. if data_dir_path.is_dir():
41.     data_listing_resource = DirectoryResource(
42.         uri="resource://data-files",
43.         path=data_dir_path, # 目录的路径
44.         name="Data Directory Listing",
45.         description="列出数据目录中可用的文件。",
46.         recursive=False # 设置为 True 以列出子目录
47.     )
48.     mcp.add_resource(data_listing_resource) # 返回文件的 JSON 列表
```

当使用 Resource 子类时，需要先从 fastmcp.resources 库中导入相应的类，如代码 3.19 的第 3 行所示。**注意不要从 mcp.server.fastmcp.resources 库[8]中导入。**

常见的资源类和对应的用途如表 3.19 所示。

表 3.19　常见资源类和用途对应表

资源类	用途
TextResource	用于简单的字符串内容。
BinaryResource	用于原始 bytes 内容。
FileResource	从本地文件路径读取内容。处理文本/二进制模式和延迟读取。

[8] § 3.1.2 中已说明，该库是 MCP 协议组织收录的 FastMCP 库，其版本为 v1.0，已过时，与高版本 FastMCP 库同时使用时容易出现问题。

HttpResource	从 HTTP(S) URL 获取内容（需要 httpx 库[9]）。
DirectoryResource	列出本地目录中的文件（返回 JSON）。
FunctionResource	@mcp.resource 使用的内部类。

注：FunctionResource 类用于将函数注册为资源，供@mcp.resource 装饰器使用，我们不用。

这些资源类都是 Resource 类的子类，Resource 类的构造函数参数与@mcp.resource 装饰器的参数相同[10]，这些参数应如何设置详见§ 3.6.1。资源类各自有其它专有的参数，将在下面详细介绍。

1. TestResource 类

该类将简单的文本字符串内容注册为资源，当需要暴露一段固定的文本内容（例如说明、简短的信息等）时使用，默认的 mime_type 为 text/plain。该子类专有的参数有：

• 参数 text：str 型，其值作为文本资源的内容。

2. BinaryResource 类

该类将原始字节数据注册为资源，常在需要返回非文本内容（例如生成二维码图片、返回图标文件）时使用。当 MCP 客户端请求由该子类注册的资源时，原始字节数据作为键“blob”的值包含在响应中。例如，将字符串“hello”以字节流形式生成 BinaryResource 类并注册为资源，客户端请求该资源时收到以 BASE64 编码的“hello”（显示为“aGVsbG8=”）：

MCP 服务器代码	客户端收到的响应

b=BinaryResource(uri="resource://aaa",
data=b'hello')
mcp.add_resource(b)
	{
	"contents": [
	{
	"uri": "resource://aaa",
	"mimeType": "text/plain",
	"blob": "aGVsbG8="
	}
	]
	}

该子类专有的参数[11]有：

• 参数 data：bytes 型，其值为字节形式的资源内容。

此外，出于便于客户端解析数据的考虑，**强烈建议提供 mime_type 参数以表明原始数据类型**（例如图片 image/png 等）。

3. FileResource 类

该类从服务器文件系统读取文件内容，支持文本或二进制模式，并具有延迟加载（即访问时才真正读取文件）的特性。常在需要直接暴露本地文件内容，但

[9] “需要 httpx 库”是指在 HttpResource 类的实现中调用了 httpx 库中的方法，在 FastMCP 库的源代码中已经 import httpx 了，无需我们写 import httpx 来手动导入 httpx 库。

[10] title、description、tags 等部分参数未在 Resource 类的构造函数中列出，在该类及其子类的构造函数中提供这些参数时，IDE 会提示“意外实参”并建议移除，实际上可以忽略提示，提供这些参数仍能够像在@resource 装饰器中提供的那样正确地设置资源组件的属性。经过实际测试，效果符合预期。

[11] 在 BinaryResource、FileResource 等子类构造函数中，参数 uri 以形如“uri="resource://aaa"”的格式用 str 型常量给出时，IDE 可能提示“应为类型 'AnyUrl'，但实际为 'str'”，可以忽略提示。

不想自己写文件读取逻辑时使用。该子类专有的参数有：

• 参数 path：Path 类，其值为该文件在文件系统中的路径。建议使用形如“Path("./README.md").resolve()”的方法提供，以实现兼容不同操作系统的路径格式，提升跨平台兼容能力。使用 Path 类时，需要先从 pathlib 库中导入 Path 类，代码为“from pathlib import Path”。

• 参数 is_binary：bool 型，默认值为 False，表示是（True）否（False）以二进制形式读取文件。

此外，出于便于客户端解析数据的考虑，**强烈建议提供 mime_type 参数以表明原始数据类型**，mime_type 参数默认值为 text/plain。

需要特别注意原始文件和客户端的编码/解码方式是否相同，否则可能解析出乱码，例如：

MCP 服务器代码	**test.txt 内容** （以 UTF-8 编码）	**客户端收到的响应** （以 GBK 解码）

file_path=
Path("./test.txt").resolve()
c=FileResource(uri="file://myFile",
path=file_path)
mcp.add_resource(c)
	This is a test file.
	Hello world!
	我在使用 FastMCP v2
		{
		"contents": [
		{
		"uri": "file://myFile",
		"mimeType": "text/plain",
		"text": "This is a test file.\nHello world!\n鎳戝湼浣跨敷 FastMCP v2"
		}
		]
		}

4. HttpResouce 类

该类从 HTTP/HTTPS URL 获取内容作为资源，需要 httpx 库。常在需要代理外部 API 或网页内容给大模型、智能体时使用。该子类专有的参数有：

• 参数 url：str 型，其值为作为资源来源的 HTTP/HTTPS URL。

• 参数 mime_type：str 型，默认值为 application/json，表示资源类型。出于便于客户端解析数据的考虑，**强烈建议提供该参数以表明原始数据类型**。

当 MCP 客户端请求 HttpResource 类的资源时，从 url 获取的内容会作为键“text”的值包含在响应中。对于普通的网页，读取并作为资源的是原始的网页内容；对于通过 HTTP/HTTPS 获取的文件，则与 FileResource 类相似，只是文件位置不同（一个是在 MCP 服务器上，一个是在外部的服务器上）。例如：

MCP 服务器代码	**客户端收到的响应**
d=HttpResource(uri="resource://test_website", url="https://gofastmcp.com/v2/getting-started/welcome", mime_type="application/json") mcp.add_resource(d)	{ "contents": [{ "uri": "resource://test_website", "mimeType": "application/json", "text": "<!DOCTYPE html><html lang=\"en\" class=\"__variable_47c970

	variable 3bbdad dark\" data-banner-state=\"visible\" data-page-mode=\"none\"><head><meta charSet=\"utf-8\"/><meta name..." }] }

d=HttpResource(uri="resource://test_website",
url="https://gofastmcp.com/v2/getting-started/welcome.md",
mime_type="text/markdown")
mcp.add_resource(d)
	{
	"contents": [
	{
	"uri": "resource://test_website",
	"mimeType": "text/markdown",
	"text": "> ## Documentation Index\n> Fetch the complete documentation index at: https://gofastmcp.com/llms.txt..."
	}
	]
	}

上面的代码以网页内容作为资源，客户端请求得到的是该网页的 html 源代码。下面的代码以外部服务器上的.md 文件作为资源，客户端请求得到的是该外部文件的内容。由于网页和文件内容过多，为便于展示已省略部分内容。

5. DirectoryResource 类

该类列出服务器上指定目录下的文件列表，返回 JSON 格式的元数据（如文件名、文件大小、修改时间）。常用于向大模型/智能体提供目录结构信息，便于大模型选择需要进一步读取的文件。该子类专有的参数有：

• 参数 path：Path 类，其值为该目录在服务器文件系统中的路径。建议使用形如“Path("./app_data").resolve()”的方法提供，以实现兼容不同操作系统的路径格式，提升跨平台兼容能力。使用 Path 类时，需要先从 pathlib 库中导入 Path 类，代码为“from pathlib import Path”。

• 参数 recursive：bool 型，默认值为 False，表示是（True）否（False）递归地列出文件。若不递归地列出文件，则仅列出指定目录下的文件；若递归地列出文件，则指定目录的子目录、子目录的子目录等包含的文件也被列出。例如：

目录结构	DirectoryResource_test/ ├── a.txt ├── b.txt └── c/ ├── d.txt └── e/ └── f.txt
MCP 服务器代码	**客户端收到的响应**

e=DirectoryResource(uri="directory://test",
path=Path("./DirectoryResource_test").resolve(),
recursive=True)
mcp.add_resource(e)
	{
	"contents": [
	{
	"uri": "directory://test",
	"mimeType": "application/json",
	"text": "{
	"files": [
	"a.txt",

	"b.txt", "c\\d.txt", "c\\e\\f.txt"] }" }] }

e=DirectoryResource(uri="directory://test",
path=Path("./DirectoryResource_test").resolve(),
recursive=False)
mcp.add_resource(e)
	{
	"contents": [
	{
	"uri": "directory://test",
	"mimeType": "application/json",
	"text": "{
	"files": [
	"a.txt",
	"b.txt",
	]
	}"
	}
	]
	}

• 参数 pattern：str | None 型，默认值为 None，其值为通配符匹配模式（写法与 glob()方法中相同），用于筛选文件。常用的通配符有：

通配符	用途	示例
*	匹配零个或多个字符	“*.txt”匹配所有以“.txt”结尾的文件
?	匹配单个字符	“file?.txt”匹配 file1.txt、fileA.txt 等文件
[]	匹配指定字符集合中的一个字符	“file[123].txt”匹配 file1.txt、file2.txt、file3.txt 文件，fileA.txt 文件则不匹配

例如，若想筛选出目录下以“.in”结尾的文件，则可以设置参数 pattern="*.in"，这样仅有与 pattern 中指定的模式匹配的文件出现在列表中。

此外，mime_type 参数可表示该资源类型，默认值为 application/json。

6. FunctionResource 类

该类用于将函数注册为资源，即供@mcp.resource 装饰器使用。我们如果想要将函数注册为资源，建议采用装饰器的写法，更加方便和易于维护。

3.6.6 自定义资源键

（2026/02/27 更新）最新版本的官方文档中已移除“自定义资源键”功能，建议不要使用此功能。

当使用资源类和 mcp.add_resource()方法注册资源时，可以提供 key 参数自定义资源键，而通过@mcp.resource 注册资源则不具有自定义资源键的功能。自定义资源键的情况下，MCP 客户端通过 key 参数指定的 URI 而不是通过 uri 参数指定的 URI 请求资源。

需要注意的是，目前（截至 2026/2/25）官方文档中自定义资源键（Custom Resource Keys）部分的描述与实际 FastMCP 库不符，按照官方文档中的设置方

式（如下灰色部分描述）会产生错误“TypeError: FastMCP.add_resource() got an unexpected keyword argument 'key'”。这里将官方描述列出以供参考，**如需使用自定义资源键的功能，请跳过下面的灰色部分继续阅读。**

在使用 mcp.add_resource() 直接添加资源时，您可以可选地提供自定义存储键：

```
# 使用标准 URI 作为键创建资源
resource = TextResource(uri="resource://data")
mcp.add_resource(resource)  # 将使用 "resource://data" 存储和访问

# 使用自定义键创建资源
special_resource = TextResource(uri="resource://special-data")
mcp.add_resource(special_resource, key="internal://data-v2")  # 将使用 "internal://data-v2" 存储和访问
```

请注意，此参数仅在直接使用 add_resource()时可用，而不通过@resource 装饰器，因为在使用装饰器时会显式提供 URI。

经过实际测试，发现如果按照官方文档中的写法，会产生错误导致无法启动 MCP 服务器。原因在于，参数 key 并非 add_resource()方法的参数，而是资源类的父类 Resoruce 类的父类 FastMCPComponent 类的初始化函数参数，如果将 key 参数的赋值语句移动到资源类的构造函数中，即可实现官方文档所述的功能。参数 uri、name、key 都可用于控制资源组件的属性 uri 和 name。当提供不同的参数 uri、name、key 时，注册的资源属性如表 3.20 所示。

表 3.20　资源类构造函数参数 uri、name、key 与实际注册资源属性 uri 和 name 对应表

资源类构造函数参数			实际注册的资源属性	
uri	**name**	**key**	**uri**	**name**
提供	提供	提供	参数 key	参数 name
提供	缺省	提供	参数 key	参数 uri
提供	提供	缺省	参数 uri	参数 name
提供	缺省	缺省	参数 uri	参数 uri

注意此参数仅在直接使用 add_resource()时可用，通过@resource 装饰器注册资源无法设置 key 参数。

3.6.7 资源组件的错误处理

与工具组件类似，如果资源函数遇到错误，可以引发标准 Python 异常（ValueError、TypeError、FileNotFoundError、自定义异常等）或 FastMCP 库专有的 ResourceError。

默认情况下，所有的异常（包括异常详细信息）都会被记录，并转换为 MCP 错误响应返回给客户端的大语言模型（LLM）或智能体等宿主应用。这有助于宿主 AI 应用理解失败原因并作出适当的应对。

若出于安全考虑需要隐藏内部错误细节，则有两种方式：①在创建 FastMCP

实例时设置 mask_error_details=True 参数；②通过 ResourceError 显式控制发送给客户端的错误信息，这与 § 3.5.8 中所述的 ToolError 异常类似，请自行参考。

3.6.8 资源组件变更通知

当资源或资源模板被添加、启用或禁用时，FastMCP 会自动向连接的客户端发送 notifications/resources/list_changed 通知。这允许客户端保持最新的资源集，而无需手动轮询更改。这一功能在使用动态启用/禁用资源组件时非常重要。

假设发生变更相关的资源/资源模板为example_resource，则mcp.add_resource (example_resource) 、 example_resource.disable() 、 example_resource.enable 和 mcp.remove_resource("example_resource")方法会导致 MCP 服务器自动向客户端发送 resources/list_changed 通知。

仅当这些操作发生在活动的MCP请求上下文内时（例如，从工具或其他MCP操作中调用时），才会发送通知。在服务器初始化期间执行的操作不会触发通知。

3.6.9 从资源组件中访问 MCP 上下文（Context）

资源和资源模板可以通过 Context 对象访问额外的 MCP 信息和功能。和工具组件一样，要访问它，需要在资源函数中添加一个类型注解为 Context 的参数。

代码 3.20 展示了资源/资源模板访问 MCP 上下文的案例。

代码 3.20　资源/资源模板访问 MCP 上下文

```
1. from fastmcp import FastMCP, Context
2.
3. mcp = FastMCP(name="DataServer")
4.
5. @mcp.resource("resource://system-status")
6. async def get_system_status(ctx: Context) -> dict:
7.     """提供系统状态信息。"""
8.     return {
9.         "status": "operational",
10.        "request_id": ctx.request_id
11.    }
12.
13. @mcp.resource("resource://{name}/details")
14. async def get_details(name: str, ctx: Context) -> dict:
15.    """获取特定名称的详细信息。"""
16.    return {
17.        "name": name,
18.        "accessed_at": ctx.request_id
19.    }
```

关于 Contex 对象的完整功能，详见进阶功能中的 § 5.1 小节。

3.6.10 资源组件重复管理

与工具组件相同，我们可以配置 FastMCP 服务器在处理尝试注册具有相同 URI 的多个资源或资源模板时的行为模式。在 FastMCP 类构造函数中使用 参数 on_duplicate_resources 设置。

重复资源/资源模板的处理选项包括：

- **"warn"（默认）**：记录警告，新资源/模板替换旧的。
- **"error"**：引发 ValueError，阻止重复注册。
- **"replace"**：静默地用新的替换现有资源/模板。
- **"ignore"**：保留原始资源/模板并忽略新的注册尝试。

3.7 MCP 提示组件

提示[12]（Prompt）组件是 FastMCP v2.x 三大核心组件之一，它是可重用的消息模板，帮助 LLM 生成结构化、有目的的响应，FastMCP 简化了这些模板的定义。当客户端请求提示时，FastMCP 找到相应的提示定义。如果它有参数，则根据函数签名验证这些参数，然后函数使用验证过的输入执行，生成的消息返回给 LLM 以指导其响应。

这允许我们定义一致的、可重用的模板，并且 LLM 可以在不同的客户端和上下文中使用这些提示模板。

提示组件与前面的工具组件和资源组件不同，工具组件由大模型决定是否调用和如何调用，资源组件可以由大模型和宿主应用请求获取。而提示组件由用户（人类）调用，用户提供参数后 FastMCP 服务器将参数填入提示词模板并发送给客户端，指导大模型的工作。如果用户自己编写完整的提示词可能不够规范，而采用服务器预设的提示词模板既减小用户工作量，又提高了提示词的规范性。

3.7.1 @prompt 装饰器

和前面的工具组件和资源组件类似，定义提示最常见的方法是装饰 Python 函数。装饰器默认使用函数名称作为提示的标识符。

需要注意的是，FastMCP 不支持具有*args 或**kwargs 的函数作为提示。此限制存在是因为 FastMCP 需要为 MCP 协议生成完整的参数架构，这对于可变参数列表是不可能的。

与其它核心组件相似，我们可以通过@mcp.prompt 装饰器中的参数自定义提示属性，该装饰器的参数表如表 3.21 所示。可能有部分参数未在此列出，可以查阅 FastMCP 官方文档中 *Prompts—The @prompt Decorator—Decorator Arguments* 部分（https://gofastmcp.com/v2/servers/prompts#decorator-arguments）。

表 3.21　@mcp.prompt 装饰器常见参数表

参数	描述	数据类型	默认值	备注
name	通过 MCP 服务器公开显示的提示名称	str | None	None	参数缺省则以函数名称作为提示名称
title	提示的可读标题	str | None	None	
description	通过 MCP 服务器公开的提示组件的描述	str | None	None	此参数覆盖函数注释（文档字符串），建议此参数缺省

[12] Prompt 更常见的翻译为“提示词”，但为了将 MCP 中的 Prompt 组件与大模型中常用的 prompt 区分，这里将它翻译为“提示”组件。而用户给大模型提供的 prompt 则翻译为“提示词”。

tags	标签，用于对资源分类	set[str] | None	None	标签管理详见 § 3.4
enabled	启用或禁用提示	bool	True	
icons	资源的图标列表	list[Icon] | None	None	v2.13.0 版本新增 详见 § 5.3.2
meta	关于提示的可选元信息，会被传递给 MCP 客户端（**主要面向人类**）	dict[str, Any] | None	None	v2.11.0 版本新增

与资源组件类似，我们可以通过启用或禁用来控制提示的可见性与可用性。被禁用的提示不会出现在可用提示列表中，尝试读取已禁用的提示将导致“未知提示”（“Unknown prompt”）错误。

默认情况下所有提示均为启用状态。在创建提示时可以设置@mcp.prompt 装饰器中的 enabled 参数为 False 进行禁用。也可以在提示注册后通过编程方式动态地切换其状态，例如：

```
@mcp.prompt
def seasonal_prompt(): return “Happy Holidays!”

# 禁用和重新启用提示
seasonal_prompt.disable()
seasonal_prompt.enable()
```

通过设置@mcp.prompt 装饰器 enabled 参数、通过 disable()和 enable()方法、使用标签管理，这三种方法启用/禁用提示组件的优先级与工具组件的相同。

除此之外，还可以通过设计中间件的方式，实现更加精细的提示启用/禁用，例如针对特定的用户禁止请求特定的提示等，详见 § 5.2 小节。

3.7.2 异步提示和同步提示

FastMCP 无缝支持标准（def）和异步（async def）函数作为提示。

与资源组件同步/异步的选择一样，当提示函数执行网络请求、数据库查询、文件 I/O 或外部服务调用等 I/O 操作时，需要使用 异步函数 async def，以避免阻塞服务器。同步提示和异步提示的代码例如：

```
# 同步提示
@mcp.prompt
def simple_question(question: str) -> str:
    “”””生成向 LLM 提问的简单问题。”””
    return f”Question: {question}”

# 异步提示
@mcp.prompt
async def data_based_prompt(data_id: str) -> str:
    “”””基于需要获取的数据生成提示。”””
    # 在实际场景中，您可能从数据库或 API 获取数据
    async with aiohttp.ClientSession() as session:
        async with session.get(f”https://api.example.com/data/{data_id}”) as response:
            data = await response.json()
            return f”Analyze this data: {data[‘content’]}”
```

3.7.3 提示组件函数的参数

1. 使用复杂的参数类型

MCP 规范要求所有提示参数都以字符串传递，但 FastMCP 允许使用函数参数类型注解以获得更好的开发体验。

当函数定义中的参数使用复杂类型（例如 ist[int]或 dict[str, str]）时，FastMCP：

- 自动转换来自 MCP 客户端的字符串参数为预期类型
- 生成有用的描述以显示所需的确切 JSON 字符串格式
- 保留直接使用——仍可以使用正确类型的参数调用提示

由于 MCP 规范仅允许字符串参数，MCP 客户端只能提供 str 型的参数，因此它需要知道复杂类型使用什么字符串格式。FastMCP 通过自动增强参数描述（在描述中带有 JSON 架构信息）来解决这个问题，使人类和 LLM 都清楚如何格式化它们的参数。例如：

MCP 服务器代码	**生成的提示组件** （“description”字段为自动生成）

@mcp.prompt
def analyze_data(
numbers: list[int],
metadata: dict[str, str],
threshold: float
) -> str:
"""分析数值数据。"""
avg = sum(numbers) / len(numbers)
return f"Average: {avg}, above threshold: {avg > threshold}"
	{
	"name": "analyze_data",
	"description": "分析数值数据。",
	"arguments": [
	{
	"name": "numbers",
	"description": "Provide as a JSON string matching the following schema: {\"items\":{\"type\":\"integer\"},\"type\":\"array\"}",
	"required": true
	},
	{
	"name": "metadata",
	"description": "Provide as a JSON string matching the following schema: {\"additionalProperties\":{\"type\":\"string\"},\"type\":\"object\"}",
	"required": true
	},
	{
	"name": "threshold",
	"description": "Provide as a JSON string matching the following schema: {\"type\":\"number\"}",
	"required": true
	}
	]
	}

对于上面示例的提示组件，MCP 客户端将使用字符串参数调用此提示：

```
{
  "numbers": "[1, 2, 3, 4, 5]",
  "metadata": "{\"source\": \"api\", \"version\": \"1.0\"}",
  "threshold": "2.5"
}
```

通常我们基于现有的智能体框架平台开发，MCP 客户端代码无需我们手动编写。但在 MCP 客户端中，若采用手写代码的方式控制调用提示组件，也可以使用提示组件函数定义中的参数类型：

```
# 这也适用于直接调用
result = await prompt.render({
    "numbers": [1, 2, 3, 4, 5],
    "metadata": {"source": "api", "version": "1.0"},
    "threshold": 2.5
})
```

需要注意的是，使用复杂的参数类型时，需要尽量保持参数类型注解简单。复杂的嵌套类型或自定义类可能无法从 JSON 字符串可靠地转换。自动生成的字段结构描述是用户获得的关于预期格式的唯一指导。比较好的参数类型例如 list[int]、dict[str, str]、float、bool；需要避免的参数类型例如复杂的 Pydantic 模型、深度嵌套结构、自定义类等。

2. 使用必须参数与可选参数

与普通的 Python 函数一样，函数签名中未给出默认值的参数被视为“必须参数”，而给出了默认值的参数则为“可选参数”，可选参数是可以缺省的。

在下面的样例中，客户端必须提供 data_uri 参数。如果省略 analysis_type 或 include_charts 参数，则将使用它们的默认值：

```
@mcp.prompt
def data_analysis_prompt(
    data_uri: str,                          # 必需 – 没有默认值
    analysis_type: str = “summary”,         # 可选 – 有默认值
    include_charts: bool = False            # 可选 – 有默认值
) -> str:
    “””创建使用特定参数分析数据的请求。”””
    prompt = f”Please perform a ‘{analysis_type}’ analysis on the data found at {data_uri}.”
    if include_charts:
        prompt += “ Include relevant charts and visualizations.”
    return prompt
```

3.7.4 提示组件的返回值

FastMCP 可以智能地处理提示组件函数的不同返回类型。对于不同类型的返回值，FastMCP 的处理方式如下：

- **str**：自动转换为单个 PromptMessage。
- **PromptMessage**：按提供的方式直接使用。（注意可以使用更加用户友好的 Message 构造函数，它可以接受原始字符串而不是 TextContent 对象，而 PromptMessage 类的构造函数只接受 TextContent 对象）
- **list[PromptMessage | str]**：用作消息序列（对话）。可用 Message 对象代替其中的 PromptMessage 对象作为 list 的元素。
- **Any**：如果返回类型不是上述之一，则尝试将返回值转换为字符串并用作 PromptMessage。

如果要使用 Message、PromptMessage、TextContent 类，需要先从 fastmcp.prompts.prompt 库中导入它们（from fastmcp.prompts.prompt import Message, PromptMessage, TextContent），注意不要从 mcp.types 库导入。

PromptMessage 类构造函数必须要提供两个参数：①参数 role：str 型，其值必须为“user”或“assistant”，表示该消息在会话中是“用户”说的还是“助手”（大模型）说的，此功能可用于构建一个多轮对话的开场以引导大模型后续的行为。②参数 content：TextContent 对象，其值为消息内容。

TextContent 类构造函数必须提供两个参数：①参数 type：str 型，其值必须为“text”。②参数 text：str 型，其值为消息内容。

从中可以看出，PromptMessage 的实例化参数较多、比较复杂，因此简单的消息可以采用 Message 类，其构造函数可以直接提供一个字符串作为内容，默认 role=”user”，也可以由我们给出 role 参数值以实现 role=”assistant”的设置。**因此推荐使用 Message 类，它比 PromptMessage 更方便且功能相同。**

以下是一个生成 role=”user”的提示组件代码，分别使用 PromptMessage 对象和 Message 对象作为提示组件返回值：

```
@mcp.prompt()
def greeting_a(name: str):
    """生成问候语提示词"""
    return PromptMessage(role="user",content=TextContent(type="text",text=f"Hello, {name}!"))

@mcp.prompt()
def greeting_b(name: str):
    """生成问候语提示词"""
        return Message(f"Hello, {name}!")
```

以下为一个 list[Message]型返回值的提示组件，它返回了一个对话，可作为大模型某轮会话的开始以引导后续生成：

```
from fastmcp.prompts.prompt import Message, PromptResult

@mcp.prompt
def roleplay_scenario(character: str, situation: str) -> PromptResult:
    """设置带有初始消息的角色扮演场景。"""
    return [
        Message(f"Let's roleplay. You are {character}. The situation is: {situation}"),
        Message("Okay, I understand. I am ready. What happens next?", role="assistant")
        ]
```

3.7.5 提示组件变更通知

当提示被添加、启用或禁用时，FastMCP 会自动向连接的客户端发送 notifications/prompts/list_changed 通知。这允许客户端保持最新的提示集，而无需手动轮询更改。这一功能在使用动态启用/禁用提示组件时非常重要。

假设发生变更相关的提示组件为 example_prompt，则 mcp.add_prompt (example_prompt)、example_prompt.disable()、example_prompt.enable 和

mcp.remove_prompt (“example_prompt”)方法会导致 MCP 服务器自动向客户端发送 prompts/list_changed 通知。

仅当这些操作发生在活动的MCP请求上下文内时（例如，从工具或其他MCP操作中调用时），才会发送通知。在服务器初始化期间执行的操作不会触发通知。

3.7.6 从提示组件中访问 MCP 上下文（Context）

提示组件可以通过 Context 对象访问额外的 MCP 信息和功能。和工具组件一样，要访问它，需要在提示函数中添加一个类型注解为 Context 的参数。

代码 3.21 展示了提示组件访问 MCP 上下文的案例。

代码 3.21 提示组件访问 MCP 上下文

```
1. from fastmcp import FastMCP, Context
2.
3. mcp = FastMCP(name="PromptServer")
4.
5. @mcp.prompt
6. async def generate_report_request(report_type: str, ctx: Context) ->
str:
7.     """生成报告请求。"""
8.     return f"Please create a {report_type} report. Request ID:
{ctx.request_id}"
```

关于 Contex 对象的完整功能，详见进阶功能中的§ 5.1 小节。

3.7.7 提示组件重复管理

与工具组件相同，我们可以配置 FastMCP 服务器在处理尝试注册具有相同名称的提示组件时的行为模式。在 FastMCP 类构造函数中使用 参数 on_duplicate_prompts 设置。

重复提示组件的处理选项包括：

- **"warn"（默认）**：记录警告，新提示组件替换旧的。
- **"error"**：引发 ValueError，阻止重复注册。
- **"replace"**：静默地用新的替换现有提示组件。
- **"ignore"**：保留原始提示组件并忽略新的注册尝试。

3.8 部署与运行 MCP 服务器

3.8.1 传输协议

MCP 支持的三种传输协议——STDIO、SSE、Streamable HTTP 的介绍已在§ 2.1 章节中说明，采用不同的协议时，默认的通信端点[13]是不同的。假设监听主机地址为 127.0.0.1，监听端口为 8000，则采用不同传输协议时默认的通信端点如表 3.22 所示。

[13] 通信端点 URL 形如“http://127.0.0.1:8000/mcp”，由协议+地址+端口+路径组成，客户端通过地址+端口定位到服务器上正在运行的 FastMCP 服务器程序，再通过路径告知服务器需要何种功能。除了基础的 MCP 相关功能外，服务器的开发者还可以添加自定义端点以提供额外服务，详见§ 3.8.4 小节。

表 3.22　不同的传输协议和默认通信端点对应表

传输协议	默认通信端点	
STDIO	无	
SSE	服务器发送响应/消息	http://127.0.0.1:8000/sse
	客户端发送请求	http://127.0.0.1:8000/message
Streamable HTTP	http://127.0.0.1:8000/mcp	

当然，可以通过设置 FastMCP 构造函数参数 sse_path、message_path、streamable_http_path 分别设置自定义的通信端点，详见§3.3.6 小节。

3.8.2 常规启动方法

完成服务器组件的开发后，可以运行 MCP 服务器，启动它最简单的方法是通过 FastMCP 类的 run()方法。此方法启动服务器后直到停止（手动停止或因严重错误停止）可以处理所有连接管理。

run()方法有 3 个最为重要的参数，分别是参数 transport、host 和 port。

后两者分别为监听的主机地址和端口，已在§3.3.2 和§3.3.3 小节中说明。在 FastMCP v2 的旧版本（例如 v2.2.0 等）中，这两个参数仅能在 FastMCP 构造函数中提供。而在较新的版本（例如 v2.14.5 等）中，这两个参数在 run()方法和 FastMCP 构造函数中均可提供，但在构造函数中提供会出现 warning。在 v3.0.0 及以上新版本中，这两个参数仅能在 run()方法中提供，否则会报错。

参数 transport 为 str 型，用于指定服务器使用的传输协议，FastMCP 支持的三种传输协议——STDIO、SSE、Streamable HTTP 的介绍已在§2.1 章节中说明。transport 参数仅能为“stdio”、“sse”、“streamable-http”和“http”四种值中的一个，默认为“stdio”。“streamable-http”和“http”均对应 Streamable HTTP 协议。用于本地调试与部署时可以使用 STDIO 协议，而生产环境建议使用 Streamable HTTP 协议。

为了获得最大兼容性，最佳实践是将 run() 调用放在 if __name__ == "__main__": 块中。这确保服务器仅在脚本直接执行时启动，而不是在作为模块导入时启动。例如：

```
from fastmcp import FastMCP

mcp = FastMCP(name="MyServer")

@mcp.tool
def hello(name: str) -> str:
    return f"Hello, {name}!"

if __name__ == "__main__":
        mcp.run(transport="streamable-http", host=”0.0.0.0”, port=8000)
```

3.8.3 异步启动方法

FastMCP 服务器基于异步 Python 构建，但该框架提供同步和异步 API 来

适应您应用程序的需求。最常使用的 run() 方法实际上是异步服务器实现的同步包装器。

但如果需要在现有的异步函数中启动 MCP 服务器，则不可以使用 run()方法，因为它在内部创建自己的异步事件循环。如果尝试在异步函数内调用 run()，将引发关于事件循环已经运行的错误。对于这一应用场景，FastMCP 提供了 run_async()方法，它的参数与 run()方法完全相同。在异步函数中使用 run_async()方法的样例代码如下：

```
from fastmcp import FastMCP
import asyncio

mcp = FastMCP(name="MyServer")

@mcp.tool
def hello(name: str) -> str:
    return f"Hello, {name}!"

async def main():
    # 在异步上下文中使用 run_async()
    await mcp.run_async(transport="http", port=8000)

if __name__ == "__main__":
    asyncio.run(main())
```

总之，始终在异步函数内使用 run_async()，在同步上下文中使用 run()。

3.8.4 自定义路由

在使用 HTTP 传输时，我们可能希望在 MCP 服务器旁边添加自定义 Web 端点。这对于健康检查、状态页面或简单 API 很有用。

自定义路由通过与 MCP 端点相同的 Web 服务器提供服务。它们在服务器域名的根目录下可用，而 MCP 服务端点位于/mcp。例如，MCP 服务的通信端点为 http://127.0.0.1:8000/mcp，则自定义路由的端点位于 http://127.0.0.1:8000/下。

我们可以使用@custom_route 装饰器添加自定义路由。例如代码 3.22 所示。

代码 3.22 使用@custom_route 装饰器添加自定义路由示例

```
1. from fastmcp import FastMCP
2. from starlette.requests import Request
3. from starlette.responses import PlainTextResponse
4.
5. mcp = FastMCP("MyServer")
6.
7. @mcp.custom_route("/health", methods=["GET"])
8. async def health_check(request: Request) -> PlainTextResponse:
9.     return PlainTextResponse("OK")
10.
11. @mcp.tool
12. def process(data: str) -> str:
13.     return f"已处理: {data}"
14.
15. if __name__ == "__main__":
```

```
mcp.run(transport="http")  # 健康检查 http://localhost:8000/health
```

代码 3.22 中在 http://localhost:8000/health 创建了自定义路由，MCP 客户端可以通过向该通信端点发送 GET 请求进行“健康检查”并收到响应“OK”。此时，由于采用 Streamable HTTP 传输协议，此服务器上常规的 MCP 服务端点为 http://localhost:8000/mcp。

通过自定义路由的功能，可以在同一个端口实现多个服务，而不仅限于提供常规的 MCP 服务。

3.8.5 常规运行 MCP 服务器（直接 HTTP）

采用本章中上述的各方法配置启动参数后，若采用常规的 mcp.run()方法或异步的 mcp.run_async()方法，则可以通过直接运行 python 代码启动服务器。如果没有报错的话，会自动创建进程并使用内置的 uvicorn 运行服务器，服务器启动后的运行窗口和启动阶段日志如图 3.2 和图 3.3 所示。

图 3.2　常规运行 MCP 服务器的运行窗口

图 3.3　常规运行 MCP 服务器启动阶段日志

从图 3.3 的启动阶段日志中可以看出，已经写明了服务器的名称、传输协议

和 URL 等参数，以便于客户端建立连接。图 3.2 和图 3.3 所示的服务器名称为“MCP_Redis_Demo”，传输协议为 Streamable HTTP，监听 http://0.0.0.0:8000/mcp，本机上可通过 http://localhost:8000/mcp 访问，同局域网内设备则可以通过服务器 IP 地址访问。

3.8.6 高级部署 MCP 服务器（ASGI 应用）

在生产部署中，通常需要对服务器运行方式拥有更高的控制力。FastMCP 可以创建标准的 ASGI 应用，并与任意 ASGI 服务器（如 Uvicorn、Gunicorn 或 Hypercorn）协同工作。当需要配置高级服务器选项、运行多个工作进程或集成现有基础设施时，此方案尤其实用。

将 MCP 服务器创建为 ASGI 应用时，无需使用 if __name__==”__main__”:指明程序主入口，也不需要 mcp.run()和 mcp.run_async()方法运行服务器，而是使用 app = mcp.http_app()语句创建 ASGI 应用。

这样，创建的 ASGI 应用可以使用任意的 ASGI 服务器运行，例如使用 Uvicorn：

```
uvicorn app:app --host 0.0.0.0 --port 8000
```

与直接 HTTP 运行的方法相同，使用此方法运行的服务器可通过的地址访问：http://localhost:8000/mcp/（在远程访问时使用服务器的实际 IP）。

ASGI 方案在需要高可靠性和高性能的生产环境中表现突出。比如可以运行多个工作进程来处理并发请求、添加自定义中间件用于日志或监控、与现有部署流水线集成，或将 MCP 服务器挂载为大型应用的一部分。

MCP 服务器的身份验证

对于生产环境中公开部署的 MCP 服务器，开启身份验证功能对于服务器的安全性而言至关重要。在 FastMCP v2.11.0 版本首次引入了身份验证的功能。

MCP 服务器的身份验证有多种实现方案，有比较简单但安全性较低的访问控制列表（Access Control List, ACL），也有比较成熟安全的 OAuth。选择何种身份验证管理方式取决于服务器安全等级的要求。

4.1 简单的访问控制列表

如果对身份验证的安全性要求不是特别高的话，可以采用简单的访问控制列表（ACL）对连接的客户端进行权限管理。

这种方法在客户端发起连接之前，通过人工或非 MCP 协议的方式，为用户提供一个或多个用于身份验证的参数。MCP 客户端连接服务器时，采用的是 HTTP 请求，通过在 URL 末尾添加查询参数的方式或在 HTTP 请求头（Header）中添加参数的方式，将事先获得的参数传递给服务器。服务器则使用自定义中间件（详见 § 5.2 小节）拦截所有的请求，对请求中传递的参数加以验证，例如与预先存储的列表进行对比，若在白名单上则继续正常提供 MCP 服务，否则返回 403 Forbidden 拒绝提供服务。

示例如代码 4.1，展示了两种客户端提供参数的方式对应的服务器代码。

代码 4.1 使用查询参数和 HTTP 请求头提供参数实现白名单身份验证示例

```
1. from fastmcp import FastMCP
2. from fastmcp.server.middleware import Middleware, MiddlewareContext
3. from fastmcp.server.dependencies import get_http_request
4.
5. mcp = FastMCP("Demo")
6.
7. class ACLMiddleware(Middleware):
8.     __key_set = {"mytestkey"}   # 白名单集合
9.
10.     async def __call__(self, ctx: MiddlewareContext, call_next):
11.         request = get_http_request()    # 获取 HTTP 请求
12.
13.         # 从查询参数中读取 key 参数
14.         public_token_query = request.query_params.get('key')
15.
16.         # 从 http header 读取 authorization 参数
17.         public_token_header = request.headers.get('authorization')
18.
19.         # 此处使用查询参数，仅作示例
20.         public_token = public_token_query
21.         if (public_token is None) or len(public_token)<=0:
22.             # 若未提供参数则返回 401
23.             raise Exception("401 Key Required")
24.         elif public_token in self.__key_set:
25.             # 提供的 key 集合内则正常提供服务
26.             pass
27.         else:
```

```
            # 身份验证未通过，返回 403，拒绝服务
            raise Exception("403 Forbidden")
        return await call_next(ctx) # 继续后续程序
mcp.add_middleware(ACLMiddleware())

if __name__=="__main__":
    mcp.run(transport="streamable-http", host="127.0.0.1", port=8000)
```

对于代码 4.1 的案例，若采用查询参数的方式，则客户端连接的 URL 为 http://127.0.0.1/mcp?key=mytestkey；若采用 HTTP Header 的方式，则客户端连接的 URL 仍为常规的 http://127.0.0.1/mcp，但需要在 HTTP Header 中添加参数 authorization=mytestkey。若不提供要求的参数，客户端则会收到“401 Key Required”错误；若提供的参数值不在白名单上，客户端则会收到“403 Forbidden”错误。**需要注意的是，这种参数在不在白名单上的判断是区分大小写的**，如果想要不区分大小写请自行添加大小写转换相关代码。

注意，可能有部分宿主应用（大模型、智能体平台等）不支持自定义客户端请求的 HTTP Header，为了提高服务器兼容性，建议使用查询参数的方式提供身份验证参数，因为它可以附在填写的服务器 URL 中。

现在很多 MCP 服务器采用这种方式身份验证，例如高德地图的 MCP 服务等。但这种方式有个缺陷，无论是谁获取了白名单上的 key，都可以访问我们的服务器提供的服务，可以采用例如人工分发 key、定期更新 key 等措施保护，或者采用 OAuth 等安全性更高的身份验证措施。

4.2 Bearer Token

Bearer Token 身份验证仅适用于基于 HTTP 的传输方式，该方式与上节中的访问控制列表法相似，适用于预先分发好访问令牌，客户端访问服务器时将令牌作为参数提供给服务器。此处的访问令牌被称为 Bearer Token。

Bearer token 是一个用于验证请求的 JSON Web Token (JWT)。它最常用于 HTTP 请求的 Authorization 标头中，使用 Bearer 方案：

```
Authorization: Bearer <token>
```

Authorization 标头可以被视作 HTTP Header 中的参数。服务器代码中对该参数的获取、验证等，与 § 4.1 小节中“在 HTTP 请求头（Header）中添加参数的方式”相同，但访问令牌不直接作为参数 Authorization 的值提供，而必须添加“Bearer ”的前缀，即以“Bearer <访问令牌>”的格式（注意“Bearer”与令牌之间以单个空格分隔）提供，服务器代码中需要对此处理一下。

Bearer Token 身份验证本质上和简单的访问控制列表相同，其优势和缺陷自然与之相同，无论是谁获取了白名单上的 key，都可以访问我们的服务器提供的服务。如果需要更高安全性，则可以采用 OAuth 等身份验证方式。

4.3 OAuth

MCP 中的身份验证提出了与传统 Web 应用程序不同的独特挑战。MCP 客户

端需要自动发现身份验证要求，在无用户干预的情况下协商 OAuth 流，并在不同的身份提供商之间无缝工作。FastMCP 通过提供与 MCP 协议集成的身份验证模式来应对这些挑战，同时保持实现和部署的简单性。

身份验证仅适用于 FastMCP 的基于 HTTP 的传输协议（即 Streamable HTTP 和 SSE）。STDIO 传输从其本地执行环境继承安全性。

实现 MCP 服务器的身份验证功能有两种模式：①我们的 MCP 服务器仅验证令牌，而将其创建委托给受信任的内外部系统，例如企业内现有的微服务平台或 SSO 系统、外部身份验证提供商（Github、Google、JWT、WorkOS 等）。②我们的服务器实现完整的 OAuth 2.0 授权服务器，处理从用户凭据验证到令牌生命周期管理的一切。

MCP 服务器进阶开发

5.1 MCP 上下文（Context）

在定义 FastMCP 工具、资源/资源模板或提示时，您的函数可能需要与底层 MCP 会话（Session）交互或访问高级服务器功能。FastMCP 为实现此目的提供了上下文（Context）对象。服务器代码中，在定义函数时声明一个类型为 Context 的参数（通常命名为 ctx），框架会自动将当前会话的上下文注入进去。

Context 对象提供了一个干净的接口，用于在函数中访问 MCP 功能，包括：

- **日志记录**：向客户端发送调试、信息、警告和错误消息
- **进度报告**：向客户端更新长时间运行操作的进度
- **资源访问**：列出并从服务器注册的资源中读取数据
- **提示访问**：列出并检索服务器注册的提示
- **LLM 采样**：请求客户端的 LLM 基于提供的消息生成文本
- **用户征询**：在工具执行期间从用户请求结构化输入
- **状态管理**：在单个请求中的中间件和处理程序之间存储和共享数据
- **请求信息**：访问当前请求的元数据
- **服务器访问**：需要时，访问底层 FastMCP 服务器实例

5.1.1 访问上下文

1. 通过函数参数注入上下文

要在任何函数中使用上下文对象，只需在函数签名中添加一个参数并将其类型提示为 Context，它不会作为有效参数暴露给 MCP 客户端，FastMCP 将在调用函数时自动注入上下文实例。这是最常见的注入上下文的方法。

只需要在函数签名中添加的参数类型是 Context 即可，参数名称没有限定。但通常以 ctx 作为参数名称以便于代码维护，本手册中也基本上采用 ctx 命名。

需要特别注意以下几点：

- 上下文方法是异步的，因此使用上下文的函数通常也需要是异步的。
- 类型提示除了最标准的 Context 以外，还可以是联合类型（Context | None）或使用 Annotated[]，这样写仍然可以正常工作。
- 每个 MCP 请求都会收到一个新的上下文对象。上下文范围限定为单个请求；在一个请求中设置的状态或数据在后续请求中将不可用。在请求之外尝试使用上下文方法将引发错误。如果确实需要在请求之外调试或调用上下文方法，可以将变量类型设置为 Context | None=None 以避免缺少参数的错误。

2. 通过 get_context()函数注入上下文

对于函数调用嵌套较深、通过参数传递上下文不方便的情况，可以使用 get_context()在请求执行流程中的任何位置检索活动上下文。如代码 5.1 所示。

代码 5.1 通过 get_context()函数在工具中注入上下文示例

```
1. from fastmcp import FastMCP
2. from fastmcp.server.dependencies import get_context
3.
4. mcp = FastMCP(name="Dependency Demo")
5.
6. # 需要访问上下文但未通过参数接收上下文的工具函数
7. async def process_data(data: list[float]) -> dict:
8.     # 获取活动上下文—仅在请求内部调用时有效
9.     ctx = get_context()
10.     await ctx.info(f"正在处理 {len(data)} 个数据点")
11.
12. @mcp.tool
13. async def analyze_dataset(dataset_name: str) -> dict:
14.     # 调用内部使用上下文的工具函数
15.     data = load_data(dataset_name)
16.     await process_data(data)
```

使用该方法需要注意以下几点：

• 需要先从 fastmcp.server.dependencies 库中导入 get_context 函数，如代码 5.1 中的第 2 行所示。

• 与通过函数参数注入上下文一样，get_context() 函数只能在服务器请求的上下文中使用，在请求外部调用它将引发 RuntimeError。

• get_context() 函数仅适用于服务器端，不应在客户端代码中使用。

5.1.2 上下文对象的常用功能

大多数常用功能都在后文中以单独小节详细讲解，这里仅以案例作简略介绍。

1. 日志记录

可通过 Context 对象将调试、信息、警告和错误消息发送回 MCP 客户端，以便了解函数执行情况，例如：

```
await ctx.debug("开始分析")
await ctx.info(f"处理 {len(data)} 项内容")
await ctx.warning("使用了弃用的参数")
await ctx.error("处理失败")
```

2. 客户端征询

在工具执行期间从 MCP 客户端请求结构化输入，实现交互式工作流程和渐进式公开，例如：

```
result = await ctx.elicit("Enter your name:", response_type=str)
if result.action == "accept":
    name = result.data
```

3. 大模型采样

请求 MCP 客户端的 LLM 基于提供的消息生成文本，用于在工具中利用 AI 功能，例如：

```
response = await ctx.sample("Analyze this data", temperature=0.7)
```

4. 进度报告

向 MCP 客户端更新长时间运行操作的进度，以便于启用进度指示器并提供更好的用户体验，例如：

```
await ctx.report_progress(progress=50, total=100)   # 50% 完成
```

5. 资源访问

列出在 FastMCP 服务器中注册的资源或从这些资源读取数据，允许访问文件、配置或动态内容，例如：

```
# 列出可用资源
resources = await ctx.list_resources()

# 读取特定资源
content_list = await ctx.read_resource("resource://config")
content = content_list[0].content
```

与之相关的两个方法的签名与用途分别为：

• ctx.list_resources() -> list[MCPResource]：返回所有可用资源的列表。

• ctx.read_resource(uri: str | AnyUrl) -> list[ReadResourceContents]：返回指定资源内容的列表。

6. 提示访问

与资源访问类似，列出并检索在 FastMCP 服务器中注册的提示，允许工具和中间件以编程方式发现和使用可用的提示，例如：

```
# 列出可用提示
prompts = await ctx.list_prompts()

# 使用参数获取特定提示
result = await ctx.get_prompt("analyze_data", {"dataset": "users"})
messages = result.messages
```

与之相关的两个方法的签名与用途分别为：

• ctx.list_prompts() -> list[MCPPrompt]：返回所有可用提示的列表。

• ctx.get_prompt(name: str, arguments: dict[str, Any] | None = None) -> GetPromptResult：使用可选参数获取特定提示。

7. 状态管理

上下文对象维护一个状态字典，特别适用于从中间件（Middleware）[14]向工具传递信息。类似于一个限制在单个 MCP 请求内的键值型数据库。

要在上下文状态中存储值，使用 ctx.set_state(key, value)。要在上下文状态中检索值，使用 ctx.get_state(key)，若未找到则返回 None。**注意，上下文状态的作用域限定在单个 MCP 请求内。每个操作（工具调用、资源读取、列表操作等）**

[14] 中间件允许服务器开发者在 MCP 请求和响应流经您的服务器时拦截和修改它们可以将它想象为一个管道，其中每个中间件都可以检查正在发生的事情、进行更改，然后将控制权传递给链中的下一个中间件。**中间件是 FastMCP 特有的概念，不是官方 MCP 协议规范的一部分。这个中间件系统专为 FastMCP 服务器设计，可能与其他 MCP 实现不兼容。**

都会收到一个新的上下文对象。在一个请求中设置的状态在后续请求中将不可用。对于跨请求的持久数据存储，必须使用外部存储机制，如数据库、文件或内存缓存。

以下案例展示了如何使用中间件在上下文状态中存储用户信息，以及如何在工具中访问该状态：

```
from fastmcp.server.middleware import Middleware, MiddlewareContext

class UserAuthMiddleware(Middleware):
    async def on_call_tool(self, context: MiddlewareContext, call_next):

        # 中间件在上下文状态中存储用户信息
        context.fastmcp_context.set_state("user_id", "user_123")
        context.fastmcp_context.set_state("permissions", ["read", "write"])

        return await call_next(context)

@mcp.tool
async def secure_operation(data: str, ctx: Context) -> str:
    """工具可以访问中间件设置的状态。"""

    user_id = ctx.get_state("user_id")   # "user_123"
    permissions = ctx.get_state("permissions")   # ["read", "write"]

    if "write" not in permissions:
        return "Access denied"

    return f"Processing {data} for user {user_id}"
```

当创建新上下文（嵌套上下文）时，它会继承其父级状态的副本。这确保了：在子上下文上设置的状态永远不会影响父上下文；在子上下文初始化后在父上下文上设置的状态不会传播到子上下文。这使状态管理变得可预测，并防止嵌套操作之间的意外副作用。

8. 变更通知

FastMCP 在添加、删除、启用或禁用组件（如工具、资源或提示）时会自动发送列表变更通知，在 § 3.5.9、§ 3.6.8 和 § 3.7.5 小节已详细说明。但在需要手动触发这些通知的罕见情况下，可以使用上下文方法，例如：

```
@mcp.tool
async def custom_tool_management(ctx: Context) -> str:
    """自定义工具更改后手动通知的示例。"""
    # 对工具进行自定义更改后
    await ctx.send_tool_list_changed()
    await ctx.send_resource_list_changed()
    await ctx.send_prompt_list_changed()
    return "Notifications sent"
```

这些方法主要由 FastMCP 的自动通知系统内部使用，大多数开发者不需要直接调用它们。

9. 访问底层 FastMCP 服务器实例

可以使用上下文对象的 ctx.fastmcp 属性访问底层 FastMCP 服务器实例，例如 ctx.fastmcp.name 即为 FastMCP 实例的 name 属性。

10. 获取 MCP 请求和客户端的元数据

MCP 请求是 MCP SDK 的底层功能，用于高级用例。大多数开发者不需要直接使用它。

可用的属性主要有三个：

• **ctx.request_id -> str**：获取当前 MCP 请求的唯一 ID。

• **ctx.client_id -> str | None**：获取发起请求的客户端 ID（如果在初始化期间客户端提供了 ID，才能成功获取到客户端 ID）。

• **ctx.session_id -> str | None**：获取 MCP 会话 ID 以用于基于会话的数据共享（仅限采用 HTTP 传输协议时）。

在工具中读取这些属性的方法例如：

```
@mcp.tool
async def request_info(ctx: Context) -> dict:
    """返回当前请求的信息。"""
    return {
        "request_id": ctx.request_id,
        "client_id": ctx.client_id or "Unknown client"
    }
```

除了上述三个属性以外，还有一个属性——ctx.request_context 属性用于请求上下文可用性，其链式属性 ctx.request_context.meta 可用于读取客户端元数据。

•**ctx.request_context 属性**：提供对底层 MCP 请求上下文的访问，但在 MCP 会话尚未建立时（在 MCP 握手完成前的 on_request 钩子中执行中间件期间、客户端连接的初始化阶段）返回 None。MCP 请求上下文与 HTTP 请求不同，对于 HTTP 传输，即使 MCP 会话尚未建立，HTTP 请求数据也可能可用。为了在可能无法获取请求上下文的情况下安全地访问它，需要先检查 MCP 会话是否可用：

```
from fastmcp import FastMCP, Context
from fastmcp.server.dependencies import get_http_request

mcp = FastMCP(name="Session Aware Demo")

@mcp.tool
async def session_info(ctx: Context) -> dict:
    """当可用时返回会话信息。"""

    # 检查 MCP 会话是否可用
    if ctx.request_context:
        # MCP 会话可用 - 可以访问 MCP 特定的属性
        return {
            "session_id": ctx.session_id,
            "request_id": ctx.request_id,
            "has_meta": ctx.request_context.meta is not None
        }
    else:
```

```
        #MCP 会话不可用 - 使用 HTTP 辅助函数获取请求数据（如果使用 HTTP 传输）
        request = get_http_request()
        return {
            "message": "MCP 会话不可用",
            "user_agent": request.headers.get("user-agent", "未知")
        }
```

当使用 HTTP 传输时，对于无论 MCP 会话是否可用都能工作的 HTTP 请求访问，需要使用 HTTP 请求辅助函数，如 get_http_request()和 get_http_headers()，如上面代码中的 else 分支所示。

ctx.request_context 的属性 ctx.request_context.meta 可在所有 MCP 操作（工具调用、资源访问、提示词交互）中用于读取客户端元数据。当客户端未提供元数据时，该字段值为 None；若提供了元数据，则可通过属性访问方式（例如 meta.user_id）获取，而非字典索引方式。元数据结构由发起请求的客户端定义。以下是通过 ctx.request_context.meta 读取客户端元数据的案例：

```
@mcp.tool
def send_email(to: str, subject: str, body: str, ctx: Context) -> str:
    """Send an email, logging metadata about the request."""

    # 访问客户端提供的元数据
    meta = ctx.request_context.meta

    if meta:
        # 通过属性访问方式获取元数据
        user_id = meta.user_id if hasattr(meta, 'user_id') else None
        trace_id = meta.trace_id if hasattr(meta, 'trace_id') else None

        # 将元数据用于日志记录、可观测性等场景
        if trace_id:
            log_with_trace(f"Sending email for user {user_id}", trace_id)

    # 执行邮件发送逻辑...
    return f"Email sent to {to}"
```

5.1.3 运行时依赖项

1. HTTP 请求（Requests）

访问当前 HTTP 请求的推荐方法是通过 get_http_request()依赖函数：

```
from fastmcp import FastMCP
from fastmcp.server.dependencies import get_http_request
from starlette.requests import Request

mcp = FastMCP(name="HTTP Request Demo")

@mcp.tool
async def user_agent_info() -> dict:
    """返回用户代理信息。"""
    # 获取 HTTP 请求
    request: Request = get_http_request()
```

```
    # 访问请求数据
    user_agent = request.headers.get("user-agent", "Unknown")
    client_ip = request.client.host if request.client else "Unknown"

    return {
        "user_agent": user_agent,
        "client_ip": client_ip,
        "path": request.url.path,
    }
```

此方法适用于请求执行流程中的任何位置，而不仅仅是在 MCP 函数内部。这在以下情况下很有用：需要在辅助函数中访问 HTTP 信息时；调用需要 HTTP 请求数据的嵌套函数时；在使用中间件或其他请求处理代码时。

2. HTTP 头（Headers）

如果只需要请求头并希望避免潜在错误，可以使用 get_http_headers()辅助函数，例如：

```
from fastmcp import FastMCP
from fastmcp.server.dependencies import get_http_headers

mcp = FastMCP(name="Headers Demo")

@mcp.tool
async def safe_header_info() -> dict:
    """安全地获取标头信息而不引发错误。"""
    # 获取标头（如果没有请求上下文则返回空字典）
    headers = get_http_headers()

    # 获取授权标头
    auth_header = headers.get("authorization", "")
    is_bearer = auth_header.startswith("Bearer ")

    return {
        "user_agent": headers.get("user-agent", "Unknown"),
        "content_type": headers.get("content-type", "Unknown"),
        "has_auth": bool(auth_header),
        "auth_type": "Bearer" if is_bearer else "Other" if auth_header else "None",
        "headers_count": len(headers)
    }
```

默认情况下，get_http_headers()函数排除诸如 host 和 content-length 等可能引发问题的标头。如果要包含所有标头，使用 get_http_headers(include_all=True)。

3. 访问令牌（Access Tokens）

在 FastMCP 服务器中使用了身份验证功能时，可以使用 get_access_token()依赖函数访问已认证用户的访问令牌信息：

```
from fastmcp import FastMCP
from fastmcp.server.dependencies import get_access_token, AccessToken

mcp = FastMCP(name="Auth Token Demo")
```

```
@mcp.tool
async def get_user_info() -> dict:
    """获取已认证用户的信息。"""
    # 获取访问令牌（如果未认证则为 None）
    token: AccessToken | None = get_access_token()

    if token is None:
        return {"authenticated": False}

    return {
        "authenticated": True,
        "client_id": token.client_id,
        "scopes": token.scopes,
        "expires_at": token.expires_at,
        "token_claims": token.claims,   # JWT 声明或自定义令牌数据
    }
```

这一功能在以下情况下特别有用：①**访问用户标识**：从令牌声明中获取 client_id 或请求主体信息；②**检查权限**：在执行操作之前验证范围或自定义声明；③**多租户应用程序**：从令牌声明中提取租户信息；④**审计日志**：跟踪哪个用户执行了哪些操作。

get_access_token()函数的返回值的 claims 字段包含来自原始令牌的所有数据（JWT 令牌的 JWT 声明，或其他令牌类型的自定义数据），例如：

```
from fastmcp import FastMCP
from fastmcp.server.dependencies import get_access_token

mcp = FastMCP(name="Multi-tenant Demo")

@mcp.tool
async def get_tenant_data(resource_id: str) -> dict:
    """使用令牌声明获取特定租户的数据。"""
    token: AccessToken | None = get_access_token()

    # 从令牌声明中提取租户 ID
    tenant_id = token.claims.get("tenant_id") if token else None

    # 从标准 JWT 主题声明中提取用户 ID
    user_id = token.claims.get("sub") if token else None

    # 使用租户和用户信息授权并过滤数据
    if not tenant_id:
        raise ValueError("No tenant information in token")

    return {
        "resource_id": resource_id,
        "tenant_id": tenant_id,
        "user_id": user_id,
        "data": f"Tenant-specific data for {tenant_id}",
    }
```

如上面的例子中所示，可通过 get_access_token().claims.get()函数获取原始令牌数据中指定的键值。

5.1.4 自定义依赖项

FastMCP 的依赖注入功能由 Docket 驱动，它提供了一个灵活的系统，用于将值注入到我们的函数中。除了像 CurrentContext()这样的内置依赖项外，还可以创建自己的依赖项。

1. 使用 Depends()函数

创建自定义依赖项最简单的方法是使用 Depends()。传入任何可调用对象（同步或异步函数，或异步上下文管理器），其返回值将被注入，例如：

```
from contextlib import asynccontextmanager
from fastmcp import FastMCP
from fastmcp.dependencies import Depends

mcp = FastMCP(name="Custom Deps Demo")

# Simple function dependency
def get_config() -> dict:
    return {"api_url": "https://api.example.com", "timeout": 30}

# Async function dependency
async def get_user_id() -> int:
    return 42

@mcp.tool
async def fetch_data(
    query: str,
    config: dict = Depends(get_config),
    user_id: int = Depends(get_user_id),
) -> str:
    return f"User {user_id} fetching '{query}' from {config['api_url']}"
```

使用 Depends() 创建的依赖项会自动从 MCP 模式中排除——MCP 客户端永远不会将它们视为参数。

2. 使用上下文管理器（Context Managers）进行资源管理

对于需要清理的依赖项（数据库连接、文件句柄等），需要使用**异步**上下文管理器，例如：

```
from contextlib import asynccontextmanager
from fastmcp import FastMCP
from fastmcp.dependencies import Depends

mcp = FastMCP(name="Resource Demo")

@asynccontextmanager
async def get_database():
    db = await connect_to_database()
    try:
        yield db
    finally:
        await db.close()

@mcp.tool
async def query_users(sql: str, db = Depends(get_database)) -> list:
```

```
    return await db.execute(sql)
```

由于异步上下文管理器中 finally:子句的存在，即使在发生错误的情况下，上下文管理器的清理代码也会在函数完成后运行。

3. 嵌套依赖（Nested Dependencies）

依赖项也可以依赖于其他依赖项，被称为“嵌套依赖”，例如：

```
from fastmcp import FastMCP
from fastmcp.dependencies import Depends

mcp = FastMCP(name="Nested Demo")

def get_base_url() -> str:
    return "https://api.example.com"

def get_api_client(base_url: str = Depends(get_base_url)) -> dict:
    return {"base_url": base_url, "version": "v1"}

@mcp.tool
async def call_api(endpoint: str, client: dict = Depends(get_api_client)) -> str:
    return f"Calling {client['base_url']}/{client['version']}/{endpoint}"
```

4. [高级功能]继承 Dependency 类

对于更复杂的依赖模式（例如需要访问 Docket 执行上下文或需要自定义生命周期管理的依赖），可以继承 Docket 的 Dependency 类创建子类。详情请参阅 Docket 文档中关于依赖的部分（https://chrisguidry.github.io/docket/dependencies）。

5.2 中间件（Middleware）

MCP 中间件允许我们在 MCP 请求和响应流经您的服务器时拦截和修改它们。可以将它想象为一个管道，其中每个中间件都可以检查正在发生的事情、进行更改，然后将控制权传递给链中的下一个中间件。使用中间件时，需要从 fastmcp.server.middleware 库中导入 Middleware 和 MiddlewareContext 类。

FastMCP 中间件适用于所有传输类型，包括本地 stdio 传输和 HTTP 传输，尽管不是所有中间件实现都在所有传输中兼容（例如，检查 HTTP 头的中间件不适用于 stdio 传输）。

FastMCP 中间件采用管道模型运行。当请求进入时，它会按照添加到服务器的顺序流经中间件。其顺序如图 5.1 所示，图中从左到右的箭头表示请求，从右到左的箭头表示响应。

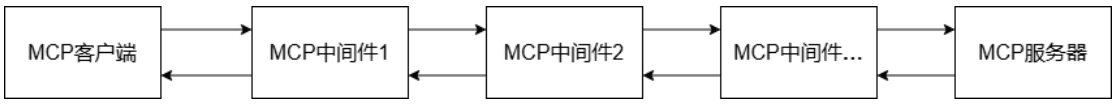

图 5.1　消息流经中间件的顺序示意图

中间件的顺序按照注册到 MCP 服务器的顺序决定，可以使用 FastMCP 类的 add_middleware()方法将以类的形式定义的中间件类注册到服务器，也可以使用@mcp.add_middleware 装饰器将函数注册为中间件，如代码 5.2 所示。

代码 5.2 定义和注册中间件示例

```
1. from fastmcp import FastMCP
2. from fastmcp.server.middleware import Middleware, MiddlewareContext
3.
4. mcp = FastMCP("Demo")
5.
6. # 定义 Middleware 子类定义中间件
7. class FirstMiddleware(Middleware):
8.     async def __call__(self, context: MiddlewareContext, call_next):
9.         # 此方法接收所有消息，无论类型如何
10.        print(f"FirstMiddleware 正在处理：{context.method}")
11.        result = await call_next(context)
12.        print(f"FirstMiddleware 完成：{context.method}")
13.        return result
14. mcp.add_middleware(FirstMiddleware) # 注册中间件
15.
16. # 使用装饰器注册中间件，注意装饰器不可写为@mcp.middleware()
17. @mcp.add_middleware
18. async def SecondMiddleware(context: MiddlewareContext, call_next):
19.     print(f"SecondMiddleware 正在处理：{context.method}")
20.     result = await call_next(context)
21.     print(f"SecondMiddleware 完成：{context.method}")
22.     return result
23.
24. if __name__=="__main__":
25.     mcp.run(transport="streamable-http")
```

由于使用继承 Middleware 类的方式定义中间件可以自定义中间件初始化操作、设置成员变量、重写特定的中间件钩子方法等，更加灵活，建议使用类定义中间件，而不是使用装饰器将函数注册为中间件。

每个 MCP 中间件都可以实现以下功能：

- 检查传入的请求及其上下文；
- 修改请求，然后将其传递给下一个中间件或处理程序；
- 执行链中的下一个中间件/处理程序，通过调用 call_next()；
- 检查和修改响应，然后返回它；
- 处理在处理过程中发生的错误。

中间件形成了一个链，其中每个部分都决定是继续处理还是完全停止链。

实现中间件的最基本方式是通过重写 Middleware 基类上的__call__方法：

```
from fastmcp.server.middleware import Middleware, MiddlewareContext

class RawMiddleware(Middleware):
    async def __call__(self, context: MiddlewareContext, call_next):
        # 此方法接收所有消息，无论类型如何
        print(f"RawMiddleware 正在处理：{context.method}")
        result = await call_next(context)
        print(f"RawMiddleware 完成：{context.method}")
        return result
```

在这个例子中，await call_next(context)语句即调用后续中间件/处理程序，

await call_next(context)语句之前可以对客户端发送的请求（可能已经经过前面的中间件处理）进行处理，await call_next(context)语句之后可以对从服务器返回的响应（可能已经经过后面的中间件处理）进行处理。

对于重写了__call__方法的中间件，流经该中间件的每条消息都会调用__call__方法。

5.2.1 中间件钩子

如上文所述，流经中间件的每一条消息，无论是请求还是通知，都会调用__call__方法。如果我们想要针对特定的消息，例如仅想要针对特定的请求（比如 tools/call）进行处理，则可以重写仅在特定类型的操作中调用的特定钩子（Hook）[15]方法，而不是重写__call__方法。FastMCP 提供了多个在不同特定性级别下调用的钩子。

当请求进入中间件时，同一个请求可能会调用多个钩子，其顺序为从一般到特定：①on_message：所有 MCP 消息（请求和通知）都调用该钩子；②on_request 或 on_notification：根据消息类型（请求/通知）调用；③操作特定钩子：为特定的 MCP 操作调用，例如 on_call_tool 钩子仅在请求为 tools/call 时调用。中间件的钩子调用顺序用图表示的话，如图 5.2 所示。

图 5.2　中间件钩子调用顺序示意图

但需要注意，MCP SDK 可能会执行额外的操作，比如为缓存目的列出工具，这将在直接工具执行之外触发额外的中间件调用。

这个层次结构使得我们能够以正确的特定性级别针对中间件逻辑。例如使用 on_message 处理广泛的关注点（如日志记录），使用 on_request 处理身份验证，使用 on_call_tool 处理工具特定的逻辑（如性能监控）。如果难以判断中间件钩子的调用顺序，则可以使用代码 5.3 所示的程序运行 MCP 服务器，使用 MCP Inspector 等工具（使用方法请自行查阅相关资料）发送指定请求，然后在服务器上查看日志记录。

代码 5.3 MCP 中间件钩子调用顺序测试示例	
MCP 服务器代码（Python 文件名为 Demo_middleware.py 并且行号已注明）	1. import logging 2. from fastmcp import FastMCP 3. from fastmcp.server.middleware import Middleware, MiddlewareContext 4. from fastmcp.utilities.logging import get_logger

[15] 钩子（Hook）是一些预定义的方法，能在特定的事件、函数调用或特定消息发生时拦截并执行自定义逻辑。它无需修改其它部分原始代码，而是插入自定义的代码，像在程序执行路径上设置“挂钩“，运行到该位置时自动触发挂载的自定义函数。

```
 5.
 6. mcp = FastMCP("Demo")
 7. logger = get_logger(name="logger")
 8. logger.setLevel(level=logging.DEBUG)
 9.
10. class MyMiddleware(Middleware):
11.     async def on_message(self, context:
MiddlewareContext, call_next):
12.         logger.info("on_message_1")
13.         result = await call_next(context)
14.         logger.info("on_message_2")
15.         return result
16.
17.     async def on_request(self, context:
MiddlewareContext, call_next):
18.         logger.info("on_request_1")
19.         result = await call_next(context)
20.         logger.info("on_request_2")
21.         return result
22.
23.     async def on_list_tools(self, context:
MiddlewareContext, call_next):
24.         logger.info("on_list_tools_1")
25.         result = await call_next(context)
26.         logger.info("on_list_tools_2")
27.         return result
28. mcp.add_middleware(MyMiddleware())
29.
30. if __name__=="__main__":
31.     mcp.run(transport="streamable-http",
32.             host="0.0.0.0",
33.             port=8000)
```

MCP 服务器日志节选（MCP Inspector 发送 tools/list 请求后的部分）

```
INFO:     127.0.0.1:50172 - "POST /mcp HTTP/1.1" 200 OK
[03/02/26 09:57:33] INFO     on_message_1                    Demo_middleware.py:13
                    INFO     on_request_1                    Demo_middleware.py:19
                    INFO     on_list_tools_1                 Demo_middleware.py:25
                    INFO     on_list_tools_2                 Demo_middleware.py:27
                    INFO     on_request_2                    Demo_middleware.py:21
                    INFO     on_message_2                    Demo_middleware.py:15
```

通过这种测试，我们可以了解准确的中间件钩子调用顺序。

FastMCP 中可用的中间件钩子及其调用条件如表 5.1 所示。

表 5.1　中间件钩子与调用条件对应表

中间件钩子	调用条件
on_message	为所有 MCP 消息（请求和通知）调用
on_request	专门为 MCP 请求（期望响应）调用
on_notification	专门为 MCP 通知（发送后忘记）调用
on_call_tool	在执行工具时调用
on_read_resource	在读取资源时调用
on_get_prompt	在检索提示时调用
on_list_tools	在列出可用工具时调用
on_list_resources	在列出可用资源时调用
on_list_resource_templates	在列出资源模板时调用
on_list_prompts	在列出可用提示时调用

on_initialize	在客户端连接并初始化会话时调用（**必须返回 None**）

注：**on_initialize 钩子接收客户端的初始化请求，但必须返回 None，与其它钩子不同。**初始化响应由 MCP 协议内部处理，中间件无法修改。此钩子对于客户端检测、记录连接或初始化会话状态很有用，但不能用于修改初始化握手本身。

5.2.2 中间件钩子结构

每个中间件钩子的结构都遵循相同的模式，以 on_message 钩子为例：

```
async def on_message(self, context: MiddlewareContext, call_next):
    # 1. 预处理：检查并可选修改请求
    print(f"正在处理 {context.method}")

    # 2. 链继续：调用下一个中间件/处理程序
    result = await call_next(context)

    # 3. 后处理：检查并可选修改响应
    print(f"已完成 {context.method}")

    # 4. 返回结果（可能已修改）
    return result
```

每个钩子都接收两个参数：

- **参数 context**：MiddlewareContext 型，包含当前请求的信息，其中
  context.method - MCP 方法名称（例如，“tools/call”）
  context.source - 请求来源（“client”或“server”）
  context.type - 消息类型（“request”或“notification”）
  context.message - MCP 消息数据
  context.timestamp - 接收请求的时间
  context.fastmcp_context - FastMCP 上下文对象（如果可用）
- **参数 call_next**：继续中间件链的函数。**必须**调用它才能继续后续的中间件或处理程序，否则请求在此中间件中断，无法提供常规的 MCP 服务。

基于如上所述的钩子结构，我们对请求流拥有完全的控制，例如：

- 继续处理：调用 await call_next(context)来继续中间件链/处理程序
- 修改请求：在调用 call_next 之前更改上下文
- 修改响应：在调用 call_next 之后更改结果
- 停止链：不调用 call_next（很少需要，通常用于身份验证、拒绝特定用户对特定组件的请求等）
- 处理错误：在 try/catch 块中包装 call_next

除了修改请求和响应外，还可以通过中间件存储状态数据，使得工具可以（可选地）在后面访问。为此，请使用 FastMCP 上下文适当地调用 set_state 或 get_state 方法，详见§ 5.1 章节。

5.2.3 中间件中的组件访问

中间件钩子分为两种类型，分别是列表操作钩子和执行操作钩子。

列表操作钩子（on_list_tools、on_list_resources、on_list_prompts 等）：这些钩子仅在列表操作时调用，中间件接收带有完整元数据的 FastMCP 组件对象，这些对象包括 FastMCP 特有的属性（如 tags）并可直接从组件访问。其返回的结果包含了在被转换为 MCP 格式之前完整的组件信息。在返回给 MCP 客户端的列表响应中，标签包含在组件的元数据字段内。

执行操作钩子（on_call_tool、on_read_resource、on_get_prompt）：中间件在组件执行之前运行，中间件返回的结果是执行结果或在找不到组件时的错误。组件元数据在这些钩子参数中不直接可用。

如果需要在执行操作期间检查组件属性（如标签），请通过中间件上下文访问组件属性，例如：

```
from fastmcp.server.middleware import Middleware, MiddlewareContext
from fastmcp.exceptions import ToolError

class TagBasedMiddleware(Middleware):
    async def on_call_tool(self, context: MiddlewareContext, call_next):
        # 访问工具对象以检查其元数据
        if context.fastmcp_context:
            try:
                tool = await context.fastmcp_context.fastmcp.get_tool(context.message.name)
                # 检查此工具是否具有 "private" 标签
                if "private" in tool.tags:
                    raise ToolError("访问被拒绝：私有工具")
                # 检查工具是否已启用
                if not tool.enabled:
                    raise ToolError("工具当前已禁用")
            except Exception:
                # 找不到工具或其他错误 - 让执行继续
                # 并自然地处理错误
                pass
        return await call_next(context)

    async def on_read_resource(self, context: MiddlewareContext, call_next):
        if context.fastmcp_context:
            try:
            resource = await context.fastmcp_context.fastmcp.get_resource(context.messag
e.uri)
                if "restricted" in resource.tags:
                    raise ResourceError("访问被拒绝：受限资源")
            except Exception:
                pass
        return await call_next(context)

    async def on_get_prompt(self, context: MiddlewareContext, call_next):
        if context.fastmcp_context:
            try:
                prompt = await context.fastmcp_context.fastmcp.get_prompt(context.mes
sage.name)
                if not prompt.enabled:
                    raise PromptError("提示当前已禁用")
```

```
            except Exception:
                pass
        return await call_next(context)
```

5.2.4 中间件的常见应用场景

1. 处理列表结果

对于列表操作，中间件的 call_next 函数在转换为 MCP 格式之前返回 FastMCP 组件列表。可以过滤或修改此列表并将其返回给客户端，例如：

```
from fastmcp.server.middleware import Middleware, MiddlewareContext

class ListingFilterMiddleware(Middleware):
    async def on_list_tools(self, context: MiddlewareContext, call_next):
        result = await call_next(context)

        # 过滤出带有 "private" 标签的工具
        filtered_tools = [
            tool for tool in result
            if "private" not in tool.tags
        ]

        # 返回修改后的列表
        return filtered_tools
```

这种过滤在组件转换为 MCP 格式并返回给客户端之前发生。标签属性在过滤期间可访问，并包含在最终列表响应中组件的 meta 字段中。

注意，在列表操作中过滤组件时，请确保在相应的执行钩子（on_call_tool、on_read_resource、on_get_prompt）中也阻止已过滤组件的执行。虽然不这样做的话不会报错，但客户端可以调用未被包含在列表中的组件，逻辑上不合理。

2. 工具调用拒绝

我们可以通过在中间件中抛出 ToolError 来拒绝对客户端特定工具的访问。这是阻止工具执行的正确方式，因为它与 FastMCP 错误处理系统正确集成。

```
from fastmcp.server.middleware import Middleware, MiddlewareContext
from fastmcp.exceptions import ToolError

class AuthMiddleware(Middleware):
    async def on_call_tool(self, context: MiddlewareContext, call_next):
        tool_name = context.message.name

        # 拒绝对受限工具的访问
        if tool_name.lower() in ["delete", "admin_config"]:
            raise ToolError("访问被拒绝：工具需要管理员权限")

        # 允许其他工具继续
        return await call_next(context)
```

注意，在拒绝工具调用时，请始终抛出 ToolError 而不是返回 ToolResult 对象或其他值。ToolError 确保通过中间件链正确地传播错误信息，并转换为正确的 MCP 错误响应格式。

3. 工具调用修改

对于工具调用等执行操作，可以在执行之前修改参数或在执行之后转换结果：

```
from fastmcp.server.middleware import Middleware, MiddlewareContext

class ToolCallMiddleware(Middleware):
    async def on_call_tool(self, context: MiddlewareContext, call_next):
        # 在执行之前修改参数
        if context.message.name == "calculate":
            # 确保输入为正数
            if context.message.arguments.get("value", 0) < 0:
                context.message.arguments["value"] = abs(context.message.arguments["value"])

        result = await call_next(context)

        # 在执行后转换结果
        if context.message.name == "get_data":
            # 向结果添加元数据
            if result.structured_content:
                result.structured_content["processed_at"] = "2024-01-01T00:00:00Z"

        return result
```

对于更复杂的工具重写场景，请考虑使用工具转换模式，详见 § 3.5.11 小节，它为创建修改的工具变体提供了更结构化的方法。

5.2.5 服务器组合中的中间件

使用 mount 或 import_server 进行服务器组合时，中间件行为遵循以下规则：

- 父服务器中间件为所有请求运行，包括路由到挂载服务器的请求；
- 挂载服务器中间件仅为该特定服务器处理的请求运行；
- 中间件在每个服务器内部保持预设的执行顺序。

这些规则便于我们创建分层的中间件架构，其中父服务器处理横切的关注点（如身份验证），而子服务器则专注于特定领域的中间件。

例如以下代码创建了各自带有中间件的父子服务器：

```
# 带有中间件的父服务器
parent = FastMCP("Parent")
parent.add_middleware(AuthenticationMiddleware("token"))

# 带有自己中间件的子服务器
child = FastMCP("Child")
child.add_middleware(LoggingMiddleware())

@child.tool
def child_tool() -> str:
    return "from child"

# 挂载子服务器
parent.mount(child, prefix="child")
```

在上面的这个案例中，当客户端调用“child_tool”时，请求将首先流经父服

务器的身份验证中间件，然后路由到子服务器，在那里它将经过子服务器的日志记录中间件。

5.2.6 内置的中间件

FastMCP 内置包含几个中间件实现，它们展示了最佳实践并提供立即可用的功能。我们既可以参考它们设计自定义中间件，也可以之间使用它们。

1. 计时中间件

性能监控对于理解服务器的行为和识别瓶颈至关重要。FastMCP 在 fastmcp.server.middleware.timing 库中包含了计时中间件，它的工作原理示例：

```
import time
from fastmcp.server.middleware import Middleware, MiddlewareContext

class SimpleTimingMiddleware(Middleware):
    async def on_request(self, context: MiddlewareContext, call_next):
        start_time = time.perf_counter()

        try:
            result = await call_next(context)
            duration_ms = (time.perf_counter() - start_time) * 1000
            print(f"Request {context.method} completed in {duration_ms:.2f}ms")
            return result
        except Exception as e:
            duration_ms = (time.perf_counter() - start_time) * 1000
            print(f"Request {context.method} failed after {duration_ms:.2f}ms: {e}")
            raise
```

注意，这并不是计时中间件的定义，仅作为示例展示其实现逻辑。要想使用该中间件，请参考以下代码：

```
from fastmcp.server.middleware.timing import (
    TimingMiddleware,
    DetailedTimingMiddleware
)

# 所有请求的基本计时
mcp.add_middleware(TimingMiddleware())

# 详细的按操作计时（工具、资源、提示）
mcp.add_middleware(DetailedTimingMiddleware())
```

TimingMiddeleware 中间件提供了针对所有请求的基本计时功能，DetailedTimingMiddleware 中间件则提供了特定于操作的钩子，如 on_call_tool 和 on_read_resource，用于细粒度计时。

2. 工具注入中间件

工具注入中间件是在请求生命周期中向服务器注入工具的中间件，用法例如：

```
from fastmcp.server.middleware.tool_injection import ToolInjectionMiddleware

def my_tool_fn(a: int, b: int) -> int:
    return a + b
```

```
my_tool = Tool.from_function(fn=my_tool_fn, name="my_tool")
mcp.add_middleware(ToolInjectionMiddleware(tools=[my_tool]))
```

3. 提示工具中间件

提示工具中间件是针对无法列出或获取提示组件的客户端的兼容性中间件。它提供两个工具：list_prompts 和 get_prompt，允许客户端仅使用工具调用来分别列出和获取提示，导入该中间件的代码：

```
from fastmcp.server.middleware.tool_injection import PromptToolMiddleware

mcp.add_middleware(PromptToolMiddleware())
```

4. 资源工具中间件

资源工具中间件是针对无法列出或读取资源组件的客户端的兼容性中间件。它提供两个工具：list_resources 和 read_resource，允许客户端仅使用工具调用来分别列出和读取资源，导入该中间件的代码：

```
from fastmcp.server.middleware.tool_injection import ResourceToolMiddleware

mcp.add_middleware(ResourceToolMiddleware())
```

5. 缓存中间件

缓存中间件对于提高性能和减少服务器负载至关重要。FastMCP 在 fastmcp.server.middleware.caching 库中提供缓存中间件，使用该中间件：

```
from fastmcp.server.middleware.caching import ResponseCachingMiddleware

mcp.add_middleware(ResponseCachingMiddleware())
```

它将调用/列出工具、资源和提示缓存到具有基于 TTL 过期的内存缓存中。缓存条目根据 TTL 过期；没有基于事件的缓存失效。列表调用存储在全局键下，因此在多个服务器之间共享存储后端时，需要考虑命名空间集合以防止冲突。

缓存相关的每个方法都可以单独配置。例如，缓存列出工具 30 秒，将缓存限制在特定工具，并禁用资源读取的缓存：

```
from fastmcp.server.middleware.caching import ResponseCachingMiddleware, CallToolSettings,
ListToolsSettings, ReadResourceSettings

mcp.add_middleware(ResponseCachingMiddleware(
    list_tools_settings=ListToolsSettings(
        ttl=30,
    ),
    call_tool_settings=CallToolSettings(
        included_tools=["tool1"],
    ),
    read_resource_settings=ReadResourceSettings(
        enabled=False
    )
))
```

有关更多高级配置的选项和使用方法，可以自行查阅 FastMCP 官方文档中 *Servers—MCP Middleware—Built-in Middleware Examples—Caching Middleware*

部分（https://gofastmcp.com/v2/servers/middleware?search=bearer#caching-middleware）和 FastMCP 官方文档中 *Servers—Storage Backends* 部分（https://gofastmcp.com/v2/servers/storage-backends）。

缓存中间件通过底层存储层收集操作统计信息（比如命中、未命中等）：

```
from fastmcp.server.middleware.caching import ResponseCachingMiddleware

middleware = ResponseCachingMiddleware()
mcp.add_middleware(middleware)

# 稍后检索统计信息
stats = middleware.statistics()
print(f"总缓存操作数：{stats}")
```

6. 日志记录中间件

请求和响应日志记录对于调试、监控和理解 MCP 服务器中的使用模式至关重要。FastMCP 在 fastmcp.server.middleware.logging 库中提供了全面的日志记录中间件，包括 LoggingMiddleware 和 StructuredLogginMiddleware，使用案例：

```
from fastmcp.server.middleware.logging import (
    LoggingMiddleware,
    StructuredLoggingMiddleware
)

# 具有载荷支持的人类可读日志记录
mcp.add_middleware(LoggingMiddleware(
    include_payloads=True,
    max_payload_length=1000
))

# 用于日志聚合工具的 JSON 结构化日志记录
mcp.add_middleware(StructuredLoggingMiddleware(include_payloads=True))
```

这两个内置的中间件包括载荷日志记录、结构化 JSON 输出、自定义日志记录器支持、载荷大小限制以及用于精细控制的操作特定钩子。

7. 速率限制中间件

速率限制对于保护您的服务器免受滥用、确保公平的资源使用以及在负载下保持性能至关重要。FastMCP 在 fastmcp.server.middleware.rate_limiting 库 中包含复杂的速率限制中间件。其工作原理示例：

```
import time
from collections import defaultdict
from fastmcp.server.middleware import Middleware, MiddlewareContext
from mcp import McpError
from mcp.types import ErrorData

class SimpleRateLimitMiddleware(Middleware):
    def __init__(self, requests_per_minute: int = 60):
        self.requests_per_minute = requests_per_minute
        self.client_requests = defaultdict(list)
```

```
    async def on_request(self, context: MiddlewareContext, call_next):
        current_time = time.time()
        client_id = "default"   # 实际使用中，从标头或上下文提取

        # 清理旧请求并检查限制
        cutoff_time = current_time - 60
        self.client_requests[client_id] = [
            req_time for req_time in self.client_requests[client_id]
            if req_time > cutoff_time
        ]

        if len(self.client_requests[client_id]) >= self.requests_per_minute:
            raise McpError(ErrorData(code=-32000, message="速率限制超出"))

        self.client_requests[client_id].append(current_time)
        return await call_next(context)
```

注意，这并不是速率限制中间件的定义，仅作为示例展示其实现逻辑。要想使用该中间件，请参考以下代码：

```
from fastmcp.server.middleware.rate_limiting import (
    RateLimitingMiddleware,
    SlidingWindowRateLimitingMiddleware
)

# 令牌桶速率限制（允许受控突发）
mcp.add_middleware(RateLimitingMiddleware(
    max_requests_per_second=10.0,
    burst_capacity=20
))

# 滑动窗口速率限制（精确的基于时间的控制）
mcp.add_middleware(SlidingWindowRateLimitingMiddleware(
    max_requests=100,
    window_minutes=1
))
```

内置的速率限制中间件包括令牌桶算法、按客户端识别、全局速率限制，以及具有可配置客户端识别函数的异步安全实现功能。

8. 错误处理中间件

一致的错误处理和恢复对于健壮的 MCP 服务器至关重要。FastMCP 在 fastmcp.server.middleware.error_handlin 库中提供全面的错误处理中间件，使用错误处理中间件的案例代码：

```
from fastmcp.server.middleware.error_handling import (
    ErrorHandlingMiddleware,
    RetryMiddleware
)

# 全面的错误日志记录和转换
mcp.add_middleware(ErrorHandlingMiddleware(
    include_traceback=True,
    transform_errors=True,
    error_callback=my_error_callback
```

```
))

# 具有指数退避的自动重试
mcp.add_middleware(RetryMiddleware(
    max_retries=3,
    retry_exceptions=(ConnectionError, TimeoutError)
))
```

内置的错误处理中间件包括错误转换、自定义回调、可配置的重试逻辑以及正确的 MCP 错误格式化功能。

这些内置的中间件可以被有选择地添加到我们的服务器中，可以组合起来使用，也可以和自定义的中间件配合使用。

5.3 图标

图标为 MCP 服务器和组件提供视觉表示，帮助客户端应用程序呈现更好的用户界面。当在 MCP 客户端中显示时，图标可以帮助用户快速识别和导航您服务器的功能。虽然图标对于功能实现没有显著的重要性，但用于商业用途，为服务器设置标志性的图标比较重要。

规范地，图标以 Icon 类的数据类型提供，使用 Icon 类需要先从 mcp.types 库中导入。对于每个图标（即一个 Icon 对象），可以提供 3 个参数：

- **参数 src**：str 型，必须提供，指向图标的 URL 或数据 URI。
- **参数 mimeType**：str 型，可缺省，表示图像的 MIME 类型。常见的例如“image/png”、“image/svg+xml”等。
- **参数 sizes**：list[str]型，可缺省，描述图像大小。以 str 列表形式提供，每个字符串表示一种大小。例如“["48x48"]”、“["any"]”、“["48x48", "96x96"]”等

5.3.1 服务器图标

可以为服务器添加图标以在客户端用户程序中显示。在 FastMCP 构造函数的参数 icons 中以 list[Icon]形式提供，例如：

```
from fastmcp import FastMCP
from mcp.types import Icon

mcp = FastMCP(
    name="WeatherService",
    website_url="https://weather.example.com",
    icons=[
        Icon(
            src="https://weather.example.com/icon-48.png",
            mimeType="image/png",
            sizes=["48x48"]
        ),
        Icon(
            src="https://weather.example.com/icon-96.png",
            mimeType="image/png",
            sizes=["96x96"]
        ),
    ]
```

```
)
```

上面的例子中，提供了两种尺寸的服务器图标，这有利于客户端应用程序根据显示的需要选择合适尺寸的图标。

5.3.2 组件图标

除了服务器以外，还可以为单个工具、资源/资源模板和提示添加图标。

在这些组件的装饰器参数 icons 中以 list[Icon]形式提供，例如：

```
from mcp.types import Icon

@mcp.tool(
    icons=[Icon(src="https://example.com/calculator-icon.png")]
)
def calculate_sum(a: int, b: int) -> int:
    """将两个数字相加。"""
    return a + b

@mcp.resource(
    "config://settings",
    icons=[Icon(src="https://example.com/config-icon.png")]
)
def get_settings() -> dict:
    """检索应用程序设置。"""
    return {"theme": "dark", "language": "en"}

@mcp.prompt(
    icons=[Icon(src="https://example.com/prompt-icon.png")]
)
def analyze_code(code: str):
    """创建用于代码分析的提示。"""
    return f"请分析此代码：\n\n{code}"
```

5.3.3 使用数据 URI 嵌入图标

对于小图标或当想要直接嵌入图标时，可以使用数据 URI，例如：

```
from mcp.types import Icon

# SVG 图标作为数据 URI
svg_icon = Icon(

src="
MC9zdmciIHdpZHRoPSIyNCIgaGVpZ2h0PSIyNCI+PHBhdGggZD0iTTEyIDJDNi40OCAyID
IgNi40OCAyIDEyczQuNDggMTAgMTAgMTAgMTAtNC40OCAxMC0xMFMxNy41MiAyID
EyIDJ6Ii8+PC9zdmc+",
    mimeType="image/svg+xml"
)

@mcp.tool(icons=[svg_icon])
def my_tool() -> str:
    """带有嵌入 SVG 图标的工具。"""
    return "result"
```

附录 A　　MIME 类型

后缀名	MIME 类型
123	application/vnd.lotus-1-2-3
3dml	text/vnd.in3d.3dml
3ds	image/x-3ds
3g2	video/3gpp2
3gp	video/3gpp
7z	application/x-7z-compressed
aab	application/x-authorware-bin
aac	audio/x-aac
aam	application/x-authorware-map
aas	application/x-authorware-seg
abs	audio/x-mpeg
abw	application/x-abiword
ac	application/pkix-attr-cert
acc	application/vnd.americandynamics.acc
ace	application/x-ace-compressed
acu	application/vnd.acucobol
acutc	application/vnd.acucorp
adp	audio/adpcm
aep	application/vnd.audiograph
afm	application/x-font-type1
afp	application/vnd.ibm.modcap
ahead	application/vnd.ahead.space
ai	application/postscript
aif	audio/x-aiff
aifc	audio/x-aiff
aiff	audio/x-aiff
aim	application/x-aim
air	application/vnd.adobe.air-application-installer-package+zip
ait	application/vnd.dvb.ait
ami	application/vnd.amiga.ami
anx	application/annodex
apk	application/vnd.android.package-archive
appcache	text/cache-manifest
application	application/x-ms-application
apr	application/vnd.lotus-approach
arc	application/x-freearc
art	image/x-jg
asc	application/pgp-signature
asf	video/x-ms-asf
asm	text/x-asm
aso	application/vnd.accpac.simply.aso
asx	video/x-ms-asf
atc	application/vnd.acucorp
atom	application/atom+xml
atomcat	application/atomcat+xml
atomsvc	application/atomsvc+xml
atx	application/vnd.antix.game-component
au	audio/basic
avi	video/x-msvideo
avx	video/x-rad-screenplay
aw	application/applixware

axa	audio/annodex
axv	video/annodex
azf	application/vnd.airzip.filesecure.azf
azs	application/vnd.airzip.filesecure.azs
azw	application/vnd.amazon.ebook
bat	application/x-msdownload
bcpio	application/x-bcpio
bdf	application/x-font-bdf
bdm	application/vnd.syncml.dm+wbxml
bed	application/vnd.realvnc.bed
bh2	application/vnd.fujitsu.oasysprs
bin	application/octet-stream
blb	application/x-blorb
blorb	application/x-blorb
bmi	application/vnd.bmi
bmp	image/bmp
body	text/html
book	application/vnd.framemaker
box	application/vnd.previewsystems.box
boz	application/x-bzip2
bpk	application/octet-stream
btif	image/prs.btif
bz	application/x-bzip
bz2	application/x-bzip2
c	text/x-c
c11amc	application/vnd.cluetrust.cartomobile-config
c11amz	application/vnd.cluetrust.cartomobile-config-pkg
c4d	application/vnd.clonk.c4group
c4f	application/vnd.clonk.c4group
c4g	application/vnd.clonk.c4group
c4p	application/vnd.clonk.c4group
c4u	application/vnd.clonk.c4group
cab	application/vnd.ms-cab-compressed
caf	audio/x-caf
cap	application/vnd.tcpdump.pcap
car	application/vnd.curl.car
cat	application/vnd.ms-pki.seccat
cb7	application/x-cbr
cba	application/x-cbr
cbr	application/x-cbr
cbt	application/x-cbr
cbz	application/x-cbr
cc	text/x-c
cct	application/x-director
ccxml	application/ccxml+xml
cdbcmsg	application/vnd.contact.cmsg
cdf	application/x-cdf
cdkey	application/vnd.mediastation.cdkey
cdmia	application/cdmi-capability
cdmic	application/cdmi-container
cdmid	application/cdmi-domain
cdmio	application/cdmi-object
cdmiq	application/cdmi-queue
cdx	chemical/x-cdx
cdxml	application/vnd.chemdraw+xml

cdy	application/vnd.cinderella
cer	application/pkix-cert
cfs	application/x-cfs-compressed
cgm	image/cgm
chat	application/x-chat
chm	application/vnd.ms-htmlhelp
chrt	application/vnd.kde.kchart
cif	chemical/x-cif
cii	application/vnd.anser-web-certificate-issue-initiation
cil	application/vnd.ms-artgalry
cla	application/vnd.claymore
class	application/java
clkk	application/vnd.crick.clicker.keyboard
clkp	application/vnd.crick.clicker.palette
clkt	application/vnd.crick.clicker.template
clkw	application/vnd.crick.clicker.wordbank
clkx	application/vnd.crick.clicker
clp	application/x-msclip
cmc	application/vnd.cosmocaller
cmdf	chemical/x-cmdf
cml	chemical/x-cml
cmp	application/vnd.yellowriver-custom-menu
cmx	image/x-cmx
cod	application/vnd.rim.cod
com	application/x-msdownload
conf	text/plain
cpio	application/x-cpio
cpp	text/x-c
cpt	application/mac-compactpro
crd	application/x-mscardfile
crl	application/pkix-crl
crt	application/x-x509-ca-cert
cryptonote	application/vnd.rig.cryptonote
csh	application/x-csh
csml	chemical/x-csml
csp	application/vnd.commonspace
css	text/css
cst	application/x-director
csv	text/csv
cu	application/cu-seeme
curl	text/vnd.curl
cww	application/prs.cww
cxt	application/x-director
cxx	text/x-c
dae	model/vnd.collada+xml
daf	application/vnd.mobius.daf
dart	application/vnd.dart
dataless	application/vnd.fdsn.seed
davmount	application/davmount+xml
dbk	application/docbook+xml
dcr	application/x-director
dcurl	text/vnd.curl.dcurl
dd2	application/vnd.oma.dd2+xml
ddd	application/vnd.fujixerox.ddd
deb	application/x-debian-package

def	text/plain
deploy	application/octet-stream
der	application/x-x509-ca-cert
dfac	application/vnd.dreamfactory
dgc	application/x-dgc-compressed
dib	image/bmp
dic	text/x-c
dir	application/x-director
dis	application/vnd.mobius.dis
dist	application/octet-stream
distz	application/octet-stream
djv	image/vnd.djvu
djvu	image/vnd.djvu
dll	application/x-msdownload
dmg	application/x-apple-diskimage
dmp	application/vnd.tcpdump.pcap
dms	application/octet-stream
dna	application/vnd.dna
doc	application/msword
docm	application/vnd.ms-word.document.macroenabled.12
docx	application/vnd.openxmlformats-officedocument.wordprocessingml.document
dot	application/msword
dotm	application/vnd.ms-word.template.macroenabled.12
dotx	application/vnd.openxmlformats-officedocument.wordprocessingml.template
dp	application/vnd.osgi.dp
dpg	application/vnd.dpgraph
dra	audio/vnd.dra
dsc	text/prs.lines.tag
dssc	application/dssc+der
dtb	application/x-dtbook+xml
dtd	application/xml-dtd
dts	audio/vnd.dts
dtshd	audio/vnd.dts.hd
dump	application/octet-stream
dv	video/x-dv
dvb	video/vnd.dvb.file
dvi	application/x-dvi
dwf	model/vnd.dwf
dwg	image/vnd.dwg
dxf	image/vnd.dxf
dxp	application/vnd.spotfire.dxp
dxr	application/x-director
ecelp4800	audio/vnd.nuera.ecelp4800
ecelp7470	audio/vnd.nuera.ecelp7470
ecelp9600	audio/vnd.nuera.ecelp9600
ecma	application/ecmascript
edm	application/vnd.novadigm.edm
edx	application/vnd.novadigm.edx
efif	application/vnd.picsel
ei6	application/vnd.pg.osasli
elc	application/octet-stream
emf	application/x-msmetafile
eml	message/rfc822

emma	application/emma+xml
emz	application/x-msmetafile
eol	audio/vnd.digital-winds
eot	application/vnd.ms-fontobject
eps	application/postscript
epub	application/epub+zip
es3	application/vnd.eszigno3+xml
esa	application/vnd.osgi.subsystem
esf	application/vnd.epson.esf
et3	application/vnd.eszigno3+xml
etx	text/x-setext
eva	application/x-eva
evy	application/x-envoy
exe	application/octet-stream
exi	application/exi
ext	application/vnd.novadigm.ext
ez	application/andrew-inset
ez2	application/vnd.ezpix-album
ez3	application/vnd.ezpix-package
f	text/x-fortran
f4v	video/x-f4v
f77	text/x-fortran
f90	text/x-fortran
fbs	image/vnd.fastbidsheet
fcdt	application/vnd.adobe.formscentral.fcdt
fcs	application/vnd.isac.fcs
fdf	application/vnd.fdf
fe_launch	application/vnd.denovo.fcselayout-link
fg5	application/vnd.fujitsu.oasysgp
fgd	application/x-director
fh	image/x-freehand
fh4	image/x-freehand
fh5	image/x-freehand
fh7	image/x-freehand
fhc	image/x-freehand
fig	application/x-xfig
flac	audio/flac
fli	video/x-fli
flo	application/vnd.micrografx.flo
flv	video/x-flv
flw	application/vnd.kde.kivio
flx	text/vnd.fmi.flexstor
fly	text/vnd.fly
fm	application/vnd.framemaker
fnc	application/vnd.frogans.fnc
for	text/x-fortran
fpx	image/vnd.fpx
frame	application/vnd.framemaker
fsc	application/vnd.fsc.weblaunch
fst	image/vnd.fst
ftc	application/vnd.fluxtime.clip
fti	application/vnd.anser-web-funds-transfer-initiation
fvt	video/vnd.fvt
fxp	application/vnd.adobe.fxp
fxpl	application/vnd.adobe.fxp

fzs	application/vnd.fuzzysheet
g2w	application/vnd.geoplan
g3	image/g3fax
g3w	application/vnd.geospace
gac	application/vnd.groove-account
gam	application/x-tads
gbr	application/rpki-ghostbusters
gca	application/x-gca-compressed
gdl	model/vnd.gdl
geo	application/vnd.dynageo
gex	application/vnd.geometry-explorer
ggb	application/vnd.geogebra.file
ggt	application/vnd.geogebra.tool
ghf	application/vnd.groove-help
gif	image/gif
gim	application/vnd.groove-identity-message
gml	application/gml+xml
gmx	application/vnd.gmx
gnumeric	application/x-gnumeric
gph	application/vnd.flographit
gpx	application/gpx+xml
gqf	application/vnd.grafeq
gqs	application/vnd.grafeq
gram	application/srgs
gramps	application/x-gramps-xml
gre	application/vnd.geometry-explorer
grv	application/vnd.groove-injector
grxml	application/srgs+xml
gsf	application/x-font-ghostscript
gtar	application/x-gtar
gtm	application/vnd.groove-tool-message
gtw	model/vnd.gtw
gv	text/vnd.graphviz
gxf	application/gxf
gxt	application/vnd.geonext
gz	application/x-gzip
h	text/x-c
h261	video/h261
h263	video/h263
h264	video/h264
hal	application/vnd.hal+xml
hbci	application/vnd.hbci
hdf	application/x-hdf
hh	text/x-c
hlp	application/winhlp
hpgl	application/vnd.hp-hpgl
hpid	application/vnd.hp-hpid
hps	application/vnd.hp-hps
hqx	application/mac-binhex40
htc	text/x-component
htke	application/vnd.kenameaapp
htm	text/html
html	text/html
hvd	application/vnd.yamaha.hv-dic
hvp	application/vnd.yamaha.hv-voice

hvs	application/vnd.yamaha.hv-script
i2g	application/vnd.intergeo
icc	application/vnd.iccprofile
ice	x-conference/x-cooltalk
icm	application/vnd.iccprofile
ico	image/x-icon
ics	text/calendar
ief	image/ief
ifb	text/calendar
ifm	application/vnd.shana.informed.formdata
iges	model/iges
igl	application/vnd.igloader
igm	application/vnd.insors.igm
igs	model/iges
igx	application/vnd.micrografx.igx
iif	application/vnd.shana.informed.interchange
imp	application/vnd.accpac.simply.imp
ims	application/vnd.ms-ims
in	text/plain
ink	application/inkml+xml
inkml	application/inkml+xml
install	application/x-install-instructions
iota	application/vnd.astraea-software.iota
ipfix	application/ipfix
ipk	application/vnd.shana.informed.package
irm	application/vnd.ibm.rights-management
irp	application/vnd.irepository.package+xml
iso	application/x-iso9660-image
itp	application/vnd.shana.informed.formtemplate
ivp	application/vnd.immervision-ivp
ivu	application/vnd.immervision-ivu
jad	text/vnd.sun.j2me.app-descriptor
jam	application/vnd.jam
jar	application/java-archive
java	text/x-java-source
jisp	application/vnd.jisp
jlt	application/vnd.hp-jlyt
jnlp	application/x-java-jnlp-file
joda	application/vnd.joost.joda-archive
jpe	image/jpeg
jpeg	image/jpeg
jpg	image/jpeg
jpgm	video/jpm
jpgv	video/jpeg
jpm	video/jpm
js	application/javascript
jsf	text/plain
json	application/json
jsonml	application/jsonml+json
jspf	text/plain
kar	audio/midi
karbon	application/vnd.kde.karbon
kfo	application/vnd.kde.kformula
kia	application/vnd.kidspiration
kml	application/vnd.google-earth.kml+xml

kmz	application/vnd.google-earth.kmz
kne	application/vnd.kinar
knp	application/vnd.kinar
kon	application/vnd.kde.kontour
kpr	application/vnd.kde.kpresenter
kpt	application/vnd.kde.kpresenter
kpxx	application/vnd.ds-keypoint
ksp	application/vnd.kde.kspread
ktr	application/vnd.kahootz
ktx	image/ktx
ktz	application/vnd.kahootz
kwd	application/vnd.kde.kword
kwt	application/vnd.kde.kword
lasxml	application/vnd.las.las+xml
latex	application/x-latex
lbd	application/vnd.llamagraphics.life-balance.desktop
lbe	application/vnd.llamagraphics.life-balance.exchange+xml
les	application/vnd.hhe.lesson-player
lha	application/x-lzh-compressed
link66	application/vnd.route66.link66+xml
list	text/plain
list3820	application/vnd.ibm.modcap
listafp	application/vnd.ibm.modcap
lnk	application/x-ms-shortcut
log	text/plain
lostxml	application/lost+xml
lrf	application/octet-stream
lrm	application/vnd.ms-lrm
ltf	application/vnd.frogans.ltf
lvp	audio/vnd.lucent.voice
lwp	application/vnd.lotus-wordpro
lzh	application/x-lzh-compressed
m13	application/x-msmediaview
m14	application/x-msmediaview
m1v	video/mpeg
m21	application/mp21
m2a	audio/mpeg
m2v	video/mpeg
m3a	audio/mpeg
m3u	audio/x-mpegurl
m3u8	application/vnd.apple.mpegurl
m4a	audio/mp4
m4b	audio/mp4
m4r	audio/mp4
m4u	video/vnd.mpegurl
m4v	video/mp4
ma	application/mathematica
mac	image/x-macpaint
mads	application/mads+xml
mag	application/vnd.ecowin.chart
maker	application/vnd.framemaker
man	text/troff
mar	application/octet-stream
mathml	application/mathml+xml
mb	application/mathematica

mbk	application/vnd.mobius.mbk
mbox	application/mbox
mc1	application/vnd.medcalcdata
mcd	application/vnd.mcd
mcurl	text/vnd.curl.mcurl
mdb	application/x-msaccess
mdi	image/vnd.ms-modi
me	text/troff
mesh	model/mesh
meta4	application/metalink4+xml
metalink	application/metalink+xml
mets	application/mets+xml
mfm	application/vnd.mfmp
mft	application/rpki-manifest
mgp	application/vnd.osgeo.mapguide.package
mgz	application/vnd.proteus.magazine
mid	audio/midi
midi	audio/midi
mie	application/x-mie
mif	application/x-mif
mime	message/rfc822
mj2	video/mj2
mjp2	video/mj2
mk3d	video/x-matroska
mka	audio/x-matroska
mks	video/x-matroska
mkv	video/x-matroska
mlp	application/vnd.dolby.mlp
mmd	application/vnd.chipnuts.karaoke-mmd
mmf	application/vnd.smaf
mmr	image/vnd.fujixerox.edmics-mmr
mng	video/x-mng
mny	application/x-msmoney
mobi	application/x-mobipocket-ebook
mods	application/mods+xml
mov	video/quicktime
movie	video/x-sgi-movie
mp1	audio/mpeg
mp2	audio/mpeg
mp21	application/mp21
mp2a	audio/mpeg
mp3	audio/mpeg
mp4	video/mp4
mp4a	audio/mp4
mp4s	application/mp4
mp4v	video/mp4
mpa	audio/mpeg
mpc	application/vnd.mophun.certificate
mpe	video/mpeg
mpeg	video/mpeg
mpega	audio/x-mpeg
mpg	video/mpeg
mpg4	video/mp4
mpga	audio/mpeg
mpkg	application/vnd.apple.installer+xml

mpm	application/vnd.blueice.multipass
mpn	application/vnd.mophun.application
mpp	application/vnd.ms-project
mpt	application/vnd.ms-project
mpv2	video/mpeg2
mpy	application/vnd.ibm.minipay
mqy	application/vnd.mobius.mqy
mrc	application/marc
mrcx	application/marcxml+xml
ms	text/troff
mscml	application/mediaservercontrol+xml
mseed	application/vnd.fdsn.mseed
mseq	application/vnd.mseq
msf	application/vnd.epson.msf
msh	model/mesh
msi	application/x-msdownload
msl	application/vnd.mobius.msl
msty	application/vnd.muvee.style
mts	model/vnd.mts
mus	application/vnd.musician
musicxml	application/vnd.recordare.musicxml+xml
mvb	application/x-msmediaview
mwf	application/vnd.mfer
mxf	application/mxf
mxl	application/vnd.recordare.musicxml
mxml	application/xv+xml
mxs	application/vnd.triscape.mxs
mxu	video/vnd.mpegurl
n-gage	application/vnd.nokia.n-gage.symbian.install
n3	text/n3
nb	application/mathematica
nbp	application/vnd.wolfram.player
nc	application/x-netcdf
ncx	application/x-dtbncx+xml
nfo	text/x-nfo
ngdat	application/vnd.nokia.n-gage.data
nitf	application/vnd.nitf
nlu	application/vnd.neurolanguage.nlu
nml	application/vnd.enliven
nnd	application/vnd.noblenet-directory
nns	application/vnd.noblenet-sealer
nnw	application/vnd.noblenet-web
npx	image/vnd.net-fpx
nsc	application/x-conference
nsf	application/vnd.lotus-notes
ntf	application/vnd.nitf
nzb	application/x-nzb
oa2	application/vnd.fujitsu.oasys2
oa3	application/vnd.fujitsu.oasys3
oas	application/vnd.fujitsu.oasys
obd	application/x-msbinder
obj	application/x-tgif
oda	application/oda
odb	application/vnd.oasis.opendocument.database
odc	application/vnd.oasis.opendocument.chart

odf	application/vnd.oasis.opendocument.formula
odft	application/vnd.oasis.opendocument.formula-template
odg	application/vnd.oasis.opendocument.graphics
odi	application/vnd.oasis.opendocument.image
odm	application/vnd.oasis.opendocument.text-master
odp	application/vnd.oasis.opendocument.presentation
ods	application/vnd.oasis.opendocument.spreadsheet
odt	application/vnd.oasis.opendocument.text
oga	audio/ogg
ogg	audio/ogg
ogv	video/ogg
ogx	application/ogg
omdoc	application/omdoc+xml
onepkg	application/onenote
onetmp	application/onenote
onetoc	application/onenote
onetoc2	application/onenote
opf	application/oebps-package+xml
opml	text/x-opml
oprc	application/vnd.palm
org	application/vnd.lotus-organizer
osf	application/vnd.yamaha.openscoreformat
osfpvg	application/vnd.yamaha.openscoreformat.osfpvg+xml
otc	application/vnd.oasis.opendocument.chart-template
otf	application/x-font-otf
otg	application/vnd.oasis.opendocument.graphics-template
oth	application/vnd.oasis.opendocument.text-web
oti	application/vnd.oasis.opendocument.image-template
otp	application/vnd.oasis.opendocument.presentation-template
ots	application/vnd.oasis.opendocument.spreadsheet-template
ott	application/vnd.oasis.opendocument.text-template
oxps	application/oxps
oxt	application/vnd.openofficeorg.extension
p	text/x-pascal
p10	application/pkcs10
p12	application/x-pkcs12
p7b	application/x-pkcs7-certificates
p7c	application/pkcs7-mime
p7m	application/pkcs7-mime
p7r	application/x-pkcs7-certreqresp
p7s	application/pkcs7-signature
p8	application/pkcs8
pas	text/x-pascal
paw	application/vnd.pawaafile
pbd	application/vnd.powerbuilder6
pbm	image/x-portable-bitmap
pcap	application/vnd.tcpdump.pcap
pcf	application/x-font-pcf
pcl	application/vnd.hp-pcl
pclxl	application/vnd.hp-pclxl
pct	image/pict
pcurl	application/vnd.curl.pcurl
pcx	image/x-pcx
pdb	application/vnd.palm
pdf	application/pdf

pfa	application/x-font-type1
pfb	application/x-font-type1
pfm	application/x-font-type1
pfr	application/font-tdpfr
pfx	application/x-pkcs12
pgm	image/x-portable-graymap
pgn	application/x-chess-pgn
pgp	application/pgp-encrypted
pic	image/pict
pict	image/pict
pkg	application/octet-stream
pki	application/pkixcmp
pkipath	application/pkix-pkipath
plb	application/vnd.3gpp.pic-bw-large
plc	application/vnd.mobius.plc
plf	application/vnd.pocketlearn
pls	audio/x-scpls
pml	application/vnd.ctc-posml
png	image/png
pnm	image/x-portable-anymap
pnt	image/x-macpaint
portpkg	application/vnd.macports.portpkg
pot	application/vnd.ms-powerpoint
potm	application/vnd.ms-powerpoint.template.macroenabled.12
potx	application/vnd.openxmlformats-officedocument.presentationml.template
ppam	application/vnd.ms-powerpoint.addin.macroenabled.12
ppd	application/vnd.cups-ppd
ppm	image/x-portable-pixmap
pps	application/vnd.ms-powerpoint
ppsm	application/vnd.ms-powerpoint.slideshow.macroenabled.12
ppsx	application/vnd.openxmlformats-officedocument.presentationml.slideshow
ppt	application/vnd.ms-powerpoint
pptm	application/vnd.ms-powerpoint.presentation.macroenabled.12
pptx	application/vnd.openxmlformats-officedocument.presentationml.presentation
pqa	application/vnd.palm
prc	application/x-mobipocket-ebook
pre	application/vnd.lotus-freelance
prf	application/pics-rules
ps	application/postscript
psb	application/vnd.3gpp.pic-bw-small
psd	image/vnd.adobe.photoshop
psf	application/x-font-linux-psf
pskcxml	application/pskc+xml
ptid	application/vnd.pvi.ptid1
pub	application/x-mspublisher
pvb	application/vnd.3gpp.pic-bw-var
pwn	application/vnd.3m.post-it-notes
pya	audio/vnd.ms-playready.media.pya
pyv	video/vnd.ms-playready.media.pyv
qam	application/vnd.epson.quickanime
qbo	application/vnd.intu.qbo
qfx	application/vnd.intu.qfx

qps	application/vnd.publishare-delta-tree
qt	video/quicktime
qti	image/x-quicktime
qtif	image/x-quicktime
qwd	application/vnd.quark.quarkxpress
qwt	application/vnd.quark.quarkxpress
qxb	application/vnd.quark.quarkxpress
qxd	application/vnd.quark.quarkxpress
qxl	application/vnd.quark.quarkxpress
qxt	application/vnd.quark.quarkxpress
ra	audio/x-pn-realaudio
ram	audio/x-pn-realaudio
rar	application/x-rar-compressed
ras	image/x-cmu-raster
rcprofile	application/vnd.ipunplugged.rcprofile
rdf	application/rdf+xml
rdz	application/vnd.data-vision.rdz
rep	application/vnd.businessobjects
res	application/x-dtbresource+xml
rgb	image/x-rgb
rif	application/reginfo+xml
rip	audio/vnd.rip
ris	application/x-research-info-systems
rl	application/resource-lists+xml
rlc	image/vnd.fujixerox.edmics-rlc
rld	application/resource-lists-diff+xml
rm	application/vnd.rn-realmedia
rmi	audio/midi
rmp	audio/x-pn-realaudio-plugin
rms	application/vnd.jcp.javame.midlet-rms
rmvb	application/vnd.rn-realmedia-vbr
rnc	application/relax-ng-compact-syntax
roa	application/rpki-roa
roff	text/troff
rp9	application/vnd.cloanto.rp9
rpss	application/vnd.nokia.radio-presets
rpst	application/vnd.nokia.radio-preset
rq	application/sparql-query
rs	application/rls-services+xml
rsd	application/rsd+xml
rss	application/rss+xml
rtf	application/rtf
rtx	text/richtext
s	text/x-asm
s3m	audio/s3m
saf	application/vnd.yamaha.smaf-audio
sbml	application/sbml+xml
sc	application/vnd.ibm.secure-container
scd	application/x-msschedule
scm	application/vnd.lotus-screencam
scq	application/scvp-cv-request
scs	application/scvp-cv-response
scurl	text/vnd.curl.scurl
sda	application/vnd.stardivision.draw
sdc	application/vnd.stardivision.calc

sdd	application/vnd.stardivision.impress
sdkd	application/vnd.solent.sdkm+xml
sdkm	application/vnd.solent.sdkm+xml
sdp	application/sdp
sdw	application/vnd.stardivision.writer
see	application/vnd.seemail
seed	application/vnd.fdsn.seed
sema	application/vnd.sema
semd	application/vnd.semd
semf	application/vnd.semf
ser	application/java-serialized-object
setpay	application/set-payment-initiation
setreg	application/set-registration-initiation
sfd-hdstx	application/vnd.hydrostatix.sof-data
sfs	application/vnd.spotfire.sfs
sfv	text/x-sfv
sgi	image/sgi
sgl	application/vnd.stardivision.writer-global
sgm	text/sgml
sgml	text/sgml
sh	application/x-sh
shar	application/x-shar
shf	application/shf+xml
sid	image/x-mrsid-image
sig	application/pgp-signature
sil	audio/silk
silo	model/mesh
sis	application/vnd.symbian.install
sisx	application/vnd.symbian.install
sit	application/x-stuffit
sitx	application/x-stuffitx
skd	application/vnd.koan
skm	application/vnd.koan
skp	application/vnd.koan
skt	application/vnd.koan
sldm	application/vnd.ms-powerpoint.slide.macroenabled.12
sldx	application/vnd.openxmlformats-officedocument.presentationml.slide
slt	application/vnd.epson.salt
sm	application/vnd.stepmania.stepchart
smf	application/vnd.stardivision.math
smi	application/smil+xml
smil	application/smil+xml
smv	video/x-smv
smzip	application/vnd.stepmania.package
snd	audio/basic
snf	application/x-font-snf
so	application/octet-stream
spc	application/x-pkcs7-certificates
spf	application/vnd.yamaha.smaf-phrase
spl	application/x-futuresplash
spot	text/vnd.in3d.spot
spp	application/scvp-vp-response
spq	application/scvp-vp-request
spx	audio/ogg

sql	application/x-sql
src	application/x-wais-source
srt	application/x-subrip
sru	application/sru+xml
srx	application/sparql-results+xml
ssdl	application/ssdl+xml
sse	application/vnd.kodak-descriptor
ssf	application/vnd.epson.ssf
ssml	application/ssml+xml
st	application/vnd.sailingtracker.track
stc	application/vnd.sun.xml.calc.template
std	application/vnd.sun.xml.draw.template
stf	application/vnd.wt.stf
sti	application/vnd.sun.xml.impress.template
stk	application/hyperstudio
stl	application/vnd.ms-pki.stl
str	application/vnd.pg.format
stw	application/vnd.sun.xml.writer.template
sub	text/vnd.dvb.subtitle
sus	application/vnd.sus-calendar
susp	application/vnd.sus-calendar
sv4cpio	application/x-sv4cpio
sv4crc	application/x-sv4crc
svc	application/vnd.dvb.service
svd	application/vnd.svd
svg	image/svg+xml
svgz	image/svg+xml
swa	application/x-director
swf	application/x-shockwave-flash
swi	application/vnd.aristanetworks.swi
sxc	application/vnd.sun.xml.calc
sxd	application/vnd.sun.xml.draw
sxg	application/vnd.sun.xml.writer.global
sxi	application/vnd.sun.xml.impress
sxm	application/vnd.sun.xml.math
sxw	application/vnd.sun.xml.writer
t	text/troff
t3	application/x-t3vm-image
taglet	application/vnd.mynfc
tao	application/vnd.tao.intent-module-archive
tar	application/x-tar
tcap	application/vnd.3gpp2.tcap
tcl	application/x-tcl
teacher	application/vnd.smart.teacher
tei	application/tei+xml
teicorpus	application/tei+xml
tex	application/x-tex
texi	application/x-texinfo
texinfo	application/x-texinfo
text	text/plain
tfi	application/thraud+xml
tfm	application/x-tex-tfm
tga	image/x-tga
thmx	application/vnd.ms-officetheme
tif	image/tiff

tiff	image/tiff
tmo	application/vnd.tmobile-livetv
torrent	application/x-bittorrent
tpl	application/vnd.groove-tool-template
tpt	application/vnd.trid.tpt
tr	text/troff
tra	application/vnd.trueapp
trm	application/x-msterminal
tsd	application/timestamped-data
tsv	text/tab-separated-values
ttc	application/x-font-ttf
ttf	application/x-font-ttf
ttl	text/turtle
twd	application/vnd.simtech-mindmapper
twds	application/vnd.simtech-mindmapper
txd	application/vnd.genomatix.tuxedo
txf	application/vnd.mobius.txf
txt	text/plain
u32	application/x-authorware-bin
udeb	application/x-debian-package
ufd	application/vnd.ufdl
ufdl	application/vnd.ufdl
ulw	audio/basic
ulx	application/x-glulx
umj	application/vnd.umajin
unityweb	application/vnd.unity
uoml	application/vnd.uoml+xml
uri	text/uri-list
uris	text/uri-list
urls	text/uri-list
ustar	application/x-ustar
utz	application/vnd.uiq.theme
uu	text/x-uuencode
uva	audio/vnd.dece.audio
uvd	application/vnd.dece.data
uvf	application/vnd.dece.data
uvg	image/vnd.dece.graphic
uvh	video/vnd.dece.hd
uvi	image/vnd.dece.graphic
uvm	video/vnd.dece.mobile
uvp	video/vnd.dece.pd
uvs	video/vnd.dece.sd
uvt	application/vnd.dece.ttml+xml
uvu	video/vnd.uvvu.mp4
uvv	video/vnd.dece.video
uvva	audio/vnd.dece.audio
uvvd	application/vnd.dece.data
uvvf	application/vnd.dece.data
uvvg	image/vnd.dece.graphic
uvvh	video/vnd.dece.hd
uvvi	image/vnd.dece.graphic
uvvm	video/vnd.dece.mobile
uvvp	video/vnd.dece.pd
uvvs	video/vnd.dece.sd
uvvt	application/vnd.dece.ttml+xml

uvvu	video/vnd.uvvu.mp4
uvvv	video/vnd.dece.video
uvvx	application/vnd.dece.unspecified
uvvz	application/vnd.dece.zip
uvx	application/vnd.dece.unspecified
uvz	application/vnd.dece.zip
vcard	text/vcard
vcd	application/x-cdlink
vcf	text/x-vcard
vcg	application/vnd.groove-vcard
vcs	text/x-vcalendar
vcx	application/vnd.vcx
vis	application/vnd.visionary
viv	video/vnd.vivo
vob	video/x-ms-vob
vor	application/vnd.stardivision.writer
vox	application/x-authorware-bin
vrml	model/vrml
vsd	application/vnd.visio
vsf	application/vnd.vsf
vss	application/vnd.visio
vst	application/vnd.visio
vsw	application/vnd.visio
vtu	model/vnd.vtu
vxml	application/voicexml+xml
w3d	application/x-director
wad	application/x-doom
wav	audio/x-wav
wax	audio/x-ms-wax
wbmp	image/vnd.wap.wbmp
wbs	application/vnd.criticaltools.wbs+xml
wbxml	application/vnd.wap.wbxml
wcm	application/vnd.ms-works
wdb	application/vnd.ms-works
wdp	image/vnd.ms-photo
weba	audio/webm
webm	video/webm
webp	image/webp
wg	application/vnd.pmi.widget
wgt	application/widget
wks	application/vnd.ms-works
wm	video/x-ms-wm
wma	audio/x-ms-wma
wmd	application/x-ms-wmd
wmf	application/x-msmetafile
wml	text/vnd.wap.wml
wmlc	application/vnd.wap.wmlc
wmls	text/vnd.wap.wmlscript
wmlsc	application/vnd.wap.wmlscriptc
wmv	video/x-ms-wmv
wmx	video/x-ms-wmx
wmz	application/x-msmetafile
woff	application/x-font-woff
wpd	application/vnd.wordperfect
wpl	application/vnd.ms-wpl

wps	application/vnd.ms-works
wqd	application/vnd.wqd
wri	application/x-mswrite
wrl	model/vrml
wsdl	application/wsdl+xml
wspolicy	application/wspolicy+xml
wtb	application/vnd.webturbo
wvx	video/x-ms-wvx
x32	application/x-authorware-bin
x3d	model/x3d+xml
x3db	model/x3d+binary
x3dbz	model/x3d+binary
x3dv	model/x3d+vrml
x3dvz	model/x3d+vrml
x3dz	model/x3d+xml
xaml	application/xaml+xml
xap	application/x-silverlight-app
xar	application/vnd.xara
xbap	application/x-ms-xbap
xbd	application/vnd.fujixerox.docuworks.binder
xbm	image/x-xbitmap
xdf	application/xcap-diff+xml
xdm	application/vnd.syncml.dm+xml
xdp	application/vnd.adobe.xdp+xml
xdssc	application/dssc+xml
xdw	application/vnd.fujixerox.docuworks
xenc	application/xenc+xml
xer	application/patch-ops-error+xml
xfdf	application/vnd.adobe.xfdf
xfdl	application/vnd.xfdl
xht	application/xhtml+xml
xhtml	application/xhtml+xml
xhvml	application/xv+xml
xif	image/vnd.xiff
xla	application/vnd.ms-excel
xlam	application/vnd.ms-excel.addin.macroenabled.12
xlc	application/vnd.ms-excel
xlf	application/x-xliff+xml
xlm	application/vnd.ms-excel
xls	application/vnd.ms-excel
xlsb	application/vnd.ms-excel.sheet.binary.macroenabled.12
xlsm	application/vnd.ms-excel.sheet.macroenabled.12
xlsx	application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
xlt	application/vnd.ms-excel
xltm	application/vnd.ms-excel.template.macroenabled.12
xltx	application/vnd.openxmlformats-officedocument.spreadsheetml.template
xlw	application/vnd.ms-excel
xm	audio/xm
xml	application/xml
xo	application/vnd.olpc-sugar
xop	application/xop+xml
xpi	application/x-xpinstall
xpl	application/xproc+xml

xpm	image/x-xpixmap
xpr	application/vnd.is-xpr
xps	application/vnd.ms-xpsdocument
xpw	application/vnd.intercon.formnet
xpx	application/vnd.intercon.formnet
xsl	application/xml
xslt	application/xslt+xml
xsm	application/vnd.syncml+xml
xspf	application/xspf+xml
xul	application/vnd.mozilla.xul+xml
xvm	application/xv+xml
xvml	application/xv+xml
xwd	image/x-xwindowdump
xyz	chemical/x-xyz
xz	application/x-xz
yang	application/yang
yin	application/yin+xml
z	application/x-compress
Z	application/x-compress
z1	application/x-zmachine
z2	application/x-zmachine
z3	application/x-zmachine
z4	application/x-zmachine
z5	application/x-zmachine
z6	application/x-zmachine
z7	application/x-zmachine
z8	application/x-zmachine
zaz	application/vnd.zzazz.deck+xml
zip	application/zip
zir	application/vnd.zul
zirz	application/vnd.zul
zmm	application/vnd.handheld-entertainment+xml

附录 B　　常用函数参数速查表

附录 B 包括了常用的函数、装饰器、方法等的参数表，这些表的内容与正文部分中的表相同，将它们集中在此以方便开发服务器过程中快速查阅。

表 3.2　FastMCP 构造函数常用参数表

参数	描述	数据类型	默认值	备注
name	FastMCP 服务器可读名称	str | None	None	
host	要监听的主机地址	str | None	None	
port	要监听的端口	int | None	None	
log_level	设置 FastMCP 服务器日志级别，用于筛选输出的日志信息	Literal[“DEBUG”, “INFO”, “WARNING”, “ERROR”, “CRITICAL"]	“INFO”	
debug	是否开启调试模式	bool	False	优先级高于 log_level
json_response	设置 HTTP POST 请求相应格式为 SSE（False）或 json（True）	bool	False	共同决定 FastMCP 服务器工作模式（仅适用 HTTP 模式，其它模式下这两个参数无效）
stateless_http	启用无状态/有状态 HTTP 模式	bool	False	
sse_path	客户端建立 SSE 长连接的端点路径	str | None	None	适用于 SSE 模式
message_path	客户端向服务器发送请求消息的端点路径	str | None	None	
streamable http_path	Streamable HTTP 模式下的统一通信端点路径	str | None	None	适用于 Streamable HTTP 模式
lifespan	生命周期钩子，一个用于控制服务器启动和关闭逻辑的异步上下文管理器函数	AsyncContextManager | None	None	
version	FastMCP 服务器版本	str | None	None	参数缺省则默认为 FastMCP 库版本
instructions	描述服务器用途、功能和交互方式，帮助客户端理解	str | None	None	
auth	用于保护基于 HTTP 传输的身份验证提供程序	OAuthProvider | TokenVerifier | None	None	详见《MCP 服务器的身份验证》
include_tags	仅暴露具有至少一个匹配标签的组件	set[str] | None	None	v2.8.0 版本新增 标签管理详见§3.4
exclude_tags	隐藏具有任何匹配标签的组件		None	
on_duplicate_tools	如何处理重复的工具注册	Literal["error", "warn", "replace"]	“warn”	详见§3.5.12
on_duplicate_resources	如何处理重复的资源注册		“warn”	详见§3.6.10
on_duplicate_prompts	如何处理重复的提示词		“warn”	详见§3.7.7

	注册			
tools	工具函数列表	list[Tool|Callable] | None	None	与@mcp.tool 装饰器功能相同，**大多数情况下可缺省该参数**
icons	FastMCP 服务器图标	list[Icon] | None	None	v2.13.0 版本新增 详见§5.3.1
website_url	提供有关服务器更多信息的网站的 URL，显示于客户端应用程序中	str | None	None	v2.13.0 版本新增
mask_error_details	是否隐藏错误细节	bool	False	工具组件的错误管理详见§3.5.8
strict_input_validation	是否启用严格认证，控制如何验证工具输入参数	bool	False	v2.13.0 版本新增
include_fastmcp_meta	是否在组件响应中包含 FastMCP 元数据	bool	True	v2.11.0 版本新增
tool_serializer	工具序列化器，将 Python 对象转为 MCP 协议消息	ToolResultSerializerType | None	None	通常使用默认的（序列化为 JSON），要更换序列化格式时可用自定义的，详见§3.5.13

注：此表中浅灰色标明的参数即为新版本 FastMCP 建议移到 run()方法的参数。FastMCP v3 版本对旧写法会报错，错误为 TypeError 并附带迁移提示。蓝灰色标明的参数为 FastMCP v3 中不再支持写在 FastMCP 构造函数的参数，其中部分参数已不再支持设置，仍支持设置的参数会在报错同时附带迁移建议。

表 3.7　@mcp.tool 装饰器常见参数表

参数	描述	数据类型	默认值	备注
name	通过 MCP 服务器公开显示的工具名称	str | None	None	
title	工具的可读标题	str | None	None	
description	通过 MCP 服务器公开的工具描述	str | None	None	此参数覆盖函数注释（文档字符串），建议此参数缺省
tags	标签，用于对工具分类	set[str] | None	None	标签管理详见§3.4
enabled	启用或禁用工具	bool	True	工具启用/禁用详见§3.5.3
icons	工具的图标列表	list[Icon] | None	None	v2.13.0 版本新增 详见§5.3.2
meta	关于工具的可选元信息，会被传递给 MCP 客户端（**主要面向人类**）	dict[str, Any] | None	None	v2.11.0 版本新增
exclude_args	参数排除列表，列表内的工具参数不会被暴露	list[str] | None	None	v2.14.0 版本起弃用
output_schema	自定义输出模式	dict[str, Any] | None	None	v2.10.0 版本新增
annotations	可选的 ToolAnnotations 对象或字典，用于添加有关工具的额外注释	ToolAnnotations | dict | None	None	v2.2.7 版本新增

	（主要面向程序）			

注：annotations 参数支持的属性未在表中列出，在下面的补充说明部分详细介绍。

表 3.8　@mcp.tool 装饰器 annotations 参数的参数表

参数	描述	数据类型	默认值	备注
title	工具的可读标题	str	None	
readOnlyHint	指示工具是否为仅读取而不做修改	bool	False	
destructiveHint	对于非只读工具，提示其更改是否具有破坏性	bool	True	
idempotentHint	指示重复的相同调用是否与单次调用效果相同	bool	False	
openWorldHint	指定工具是否与外部系统交互	bool	True	

表 3.11　媒体辅助类构造函数参数表

参数	描述	说明
path	文件路径（字符串或 Path 对象）——MIME 类型根据文件扩展名自动检测	**这两个参数必须选择其一提供，不能同时提供**
data	原始字节数据——必须提供 format 参数以指定 MIME 类型	
format	可选的格式覆盖参数（例如 “png”、“wav”、“pdf”）	提供 data 参数时必须提供 提供 path 参数时可以提供
name	当使用 data 参数时，为 File 对象指定的可选名称	提供 data 参数创建 File 对象时建议提供
annotations	可选的 MCP 注释，附加额外的元数据信息	

注：需要注意不同情况下参数的必要性不同。

表 3.13　Tool.from_tool()方法参数表

参数	描述	说明
tool	要转换的工具	**这是唯一必需的参数**
name	新工具的可选名称	建议给出，此参数缺省时新旧工具重名
description	新工具的可选描述	
transform_args	一个包含 ArgTransform 对象的字典，每个对象对应你想要修改的一个参数	
transform_fn	一个可选函数，调用时将替代父工具的逻辑	
output_schema	控制输出架构和结构化输出。传入 None 以移除原始工具的自定义输出模式	
tags	新工具的可选标签集合	
annotations	新工具的可选 ToolAnnotations 集合	
serializer	一个可选函数，用于序列化新工具的结果	

meta	控制工具的元信息。传入 None 以移除元信息，传入任意字典以设置元信息，或不传入以继承父工具的元信息	

注：此表中与@tool 装饰器参数表中同名参数用法相同，但缺省时继承原始工具同名参数值。

表 3.14 ArgTransform 类参数表

参数	描述
name	参数的新名称
description	参数的新描述
default	参数的新默认值
default_factory	一个函数，调用该函数将为参数生成默认值。这对于需要在每次工具调用时生成的参数（如时间戳或唯一 ID）非常有用
hide	是否向 LLM 隐藏该参数
required	参数是否为必需，通常用于将可选参数变为必需参数
type	参数的新类型

注：①只能在 hide=True 时使用 default_factory，因为动态默认值无法在客户端的 JSON schema 中表示。②只能对未声明默认值的参数设置 required=True。

表 3.15 @mcp.resource 装饰器常见参数表

参数	描述	数据类型	默认值	备注
uri	资源的唯一标识	str		**必须提供此参数**
name	通过 MCP 服务器公开显示的资源名称	str | None	None	参数缺省则使用 uri 作为资源名称
mime_type	指定内容类型。FastMCP 通常会推断默认类型（如 text/plain 或 application/json），但对于非文本类型，明确指定更佳	str | None	None	
title	资源的可读标题	str | None	None	
description	通过 MCP 服务器公开的资源描述	str | None	None	此参数覆盖函数注释（文档字符串），建议此参数缺省
tags	标签，用于对资源分类	set[str] | None	None	标签管理详见 § 3.4
enabled	启用或禁用资源	bool	True	
icons	资源的图标列表	list[Icon] | None	None	v2.13.0 版本新增 详见 § 5.3.2
meta	关于资源的可选元信息，会被传递给 MCP 客户端（**主要面向人类**）	dict[str, Any] | None	None	v2.11.0 版本新增
annotations	可选的 Annotations 对象或字典，用于添加有关资源的额外注释（**主要面向程序**）	Annotations | dict | None	None	v2.2.7 版本新增

注：①灰色标注的参数是与工具组件参数有区别的，未标注的与工具组件参数同名的参数用法相同；②annotations 参数支持的属性未在表中列出，在下面的补充说明部分详细介绍

表 3.17　@mcp.resource 装饰器 annotations 参数的参数表

参数	描述	数据类型	默认值	备注
readOnlyHint	指示资源是否为只读的。如果为 true，资源是只读的，不会修改其环境	bool	True	
idempotentHint	指示资源是否是幂等的，即重复读取资源是否会产生额外影响。如果为 true，重复读取资源对其环境不会产生额外影响	bool	True	

表 3.21　@mcp.prompt 装饰器常见参数表

参数	描述	数据类型	默认值	备注
name	通过 MCP 服务器公开显示的提示名称	str ｜ None	None	参数缺省则以函数名称作为提示名称
title	提示的可读标题	str ｜ None	None	
description	通过 MCP 服务器公开的提示组件的描述	str ｜ None	None	此参数覆盖函数注释（文档字符串），建议此参数缺省
tags	标签，用于对资源分类	set[str] ｜ None	None	标签管理详见 § 3.4
enabled	启用或禁用提示	bool	True	
icons	资源的图标列表	list[Icon] ｜ None	None	v2.13.0 版本新增 详见 § 5.3.2
meta	关于提示的可选元信息，会被传递给 MCP 客户端（主要面向人类）	dict[str, Any] ｜ None	None	v2.11.0 版本新增